Formal Logic 1

Christopher R. Walker

October 24, 2026

1. **What Is Logic?**
   Required Reading: Magnus et al., 2025, Chapters 1-5 (pp. 2-48)
   Digging Deeper: Smith, 2020, Chapters 1-4, 6–7, 11 (pp. 1-39, 44–58, 94-104)
   Focus: informal presentation of *validity*, *soundness*, *entailment*, *satisfiability*; distinguishing *deductive* from *inductive* arguments; translating ordinary English into truth-functional logic (TFL); understanding the meaning of TFL connectives;

2. **Truth and Grammar in TFL**
   Required Reading: Magnus et al., 2025, Chapters 6-11 (pp. 49-84)
   Digging Deeper: Smith, 2020, Chapters 8-13, 18-19 (pp. 59-120, 148-172)
   Focus: *sentences* in TFL, *parse trees*, connective *scope*, types of *ambiguity*; understanding *truth-functional* connectives; building *truth tables* for complex sentences;

3. **Semantic Strategies in TFL**
   Required Reading: Magnus et al., 2025, Chapters 12-15 (pp. 85-108)
   Digging Deeper: Smith, 2020, Chapters 14-16 (pp. 120-137)
   Focus: testing logical properties with truth tables; efficiency strategies when using truth tables; using partial tables for validation;

4. **Natural Deduction in TFL**
   Required Reading: Magnus et al., 2025, Chapter 16-17 (pp. 110-141)
   Digging Deeper: Smith, 2020, Chapters 20-22 (pp. 172-211)
   Focus: presentation of the Fitch-style *proof* system; *introduction* and *elimination* rules for TFL connectives;

5. **Proofs in TFL**
   Required Reading: Magnus et al., 2025, Chapter 18 (pp. 142-161)
   Digging Deeper: Smith, 2020, Chapter 17 (pp. 145-147)
   Focus: proof strategies; working backward from the conclusion; working forward from the premises; building and exploiting *subproofs*;

6. **Additional Rules**
   Required Reading: Magnus et al., 2025, Chapters 19-21 (pp. 162-181)
   Digging Deeper: Smith, 2020, Chapter 23 (pp. 211-215)
   Focus: extending the basic system with additional rules;

7. **Basic TFL Metalogic**
   Required Reading: Magnus et al., 2025, Chapter 22 (pp. 182-188)
   Digging Deeper: Smith, 2020, Chapter 24 (pp. 216-228)
   Focus: connecting *proof theory* to the semantics of *truth-functional* logic;

8. **An Informal Survery of First-Order Logic**
   Required Reading: Magnus et al., 2025, Chapter 23-27 (pp. 190-242)
   Digging Deeper: Smith, 2020, Chapters 25-27, 31 (pp. 228-258, 293-298)
   Focus: translating simple *universal* and *existential* sentences; *predicate* notation; nested *quantifiers*; the *identity* predicate; *definite descriptions*;

9. **Grammar of FOL**
   Required Reading: Magnus et al., 2025, Chapter 28-29 (pp. 243-256)
   Digging Deeper: Smith, 2020, Chapters 28-30 (pp. 258-289)
   Focus: *sentences* and *scope ambiguities* in first-order logic;

10. **FOL Semantics**
    Required Reading: Magnus et al., 2025, Chapter 30-35 (pp. 258-295)
    Focus: assessing the truth of FOL sentences relative to an *interpretation*; testing validity via *counter-models*

11. **FOL Proofs**
    Required Reading: Magnus et al., 2025, Chapters 36-41 (pp. 297-335)
    Digging Deeper: Smith, 2020, Chapters 32-42, Appendix (pp. 299-412)
    Focus: introduction and elimination rules for universal and existential quantifiers; identity rules; connecting FOL proof theory to interpretations;

1 What Is Logic?

Logic is a method for systematically evaluating arguments of various types. It can be used flexibly as a tool for identifying the strengths and weaknesses of the various perspectives in a debate; as a way to sort through the options in a mystery or puzzle; or as the bedrock of a hypothetico-deductive model for science, which compares experimental results to theoretical expectations in order to revise the theory or to design addititional experiments.[1]

Although this course is a study in *formal* logic, the main concepts are best introduced informally. Only by understanding these concepts can we begin construct the formal definitions that we will use as the basis of our two logics. And only by understanding these concepts can we even hope to the point of the extensive formal apparatus we will build over the next many weeks.

An *argument* is a sequence of sentences. It comprises some *premises* and a *conclusion*.

> My shoes are either in the den or in the bedroom.
>
> My shoes are not in the den.
>
> ∴ My shoes are in the bedroom.

The conclusion in this *inference* (i.e. "one-step" argument) is introduced with a special symbol ∴, an *inference marker*, that can be pronounced "so".

In real-world examples, the conclusion can occur anywhere.

> My shoes are either in the den or in the bedroom. They must be in the bedroom, because they are not in the den.

But when we turn to formal logic, conclusions will be found at the end.

Informal arguments will often use special words to indicate which sentences are meant to be premises and which is the conclusion.

conclusion indicators	**premise indicators**
so, therefore, hence, thus, accordingly, consequently, ...	*since, because, given that, ...*

Table 1: Words commonly used to indicate premises and conclusions in ordinary English arguments.

The premises and conclusions of these informal arguments are *sentences*, but not just any type of sentence will do. We are not interested in commands or questions, but only *declarative* sentences. These are the sentences that we can assign a *truth value*; sentences that we can call *true* or *false*.

[1] See Smith, 2020[p. 17] for a slightly longer description of the so-called *hypothetico-deductive* model for science.

In arguments like the one outlined above, the conclusion is a *consequence* of the premises; whenever the premises are true, the conclusion is also true. When the consequence of an argument follows in this way from its premises, we will say that the argument is *valid*. Otherwise, it is *invalid*.

Although we will keep the discussion informal for the time being, note that we can talk about arguments by representing their premises and conclusions using capital letters (with or without subscripts). Using this convention, we can quickly define this notion of consequence (*entailment*):

Definition 1.1 (Informal Entailment)**:** The sentences $A_1, \ldots, A_n$ *entail* the sentence B if and only if B is never true when all of $A_1, \ldots, A_n$ are true.[a]

[a]From now on we will use "iff" to mean "if and only if".

We need to flesh out "never" a bit more in our formal definition, but this gives us a sense of what we mean by an valid argument and its components.

Definition 1.2 (Informal Validity)**:** An argument is *valid* iff its premises entail its conclusion. Otherwise, the argument is *invalid*.

An invalid argument can be detected by noting a *counter-example*: an example of an argument with the same form where the premises are all true but the conclusion is false.

Arguments can be valid in a number of different ways. These depend on the limits we place on the types of counter-examples that we will accept as invalidating.

nomologically valid: must respect the laws of nature;

conceptually valid: must respect conceptual relations between words;

Instead of focusing on these or other more contingent notions of validity, we will be instead focused on a somewhat more narrow notion of *formal validity*.

Many conceptually valid and nomologically valid arguments cannot be *formally* validated. As the presentation goes on, we will more and more center our notion of *logical validity* on formal validity.

We understand *formal* as pertaining to the *form* of arguments. [2] Thus the replacement of sentences with letters in arguments known to be valid is our first step to *formal logic*.

The argument on the left is valid while the one on the right is not. [3]

Either X or Y.	Either X or Y.
Not Y.	Not Y.
$\therefore$ X.	$\therefore$ Z.

Note that the dependence on the *forms* of arguments is fundamental here. We can have both valid

[2]Not, for example, as a synonym for *fancy*.
[3]Unless we make clear the premise that $X \rightarrow Z$

arguments with untrue conclusions and invalid arguments whose premises and conclusions are all true. Without knowing actual content of the sentences X, Y and Z, we can assess the arguments below as valid, based entirely on the meaning of the other words being used: *and*, *or* and *not*.

Both X and Y.
$\therefore$ Z.

Either X or Y.
Not Y.
$\therefore$ X.

Moreover, these arguments are simple–indeed, minimal. In both cases, the conclusion is justified by exactly the premise(s) provided, no more and no less. We can use simple, rock-solid entailments of this sort as the basis of a system of rules for building larger arguments (*proofs*).

> One way of demonstrating that an inferential leap from some premises to a given conclusion is valid is by breaking down the big leap into smaller inference steps, each one of which is clearly valid. Smith, 2020[p29, channeling Aristotle]

In the following complex argument (sort of a "proto-proof") we use several *inference steps* (marked with $\therefore$), each depending on a simple rule and one or more of the sentences that came before it, some of which are themselves conclusions.

To make the argument into a proof, we want to decorate it a little bit more, most conspicuously by adding *line numbers* and *justifications* for each inference step.

(1)	Either X or Y.	assumed
(2)	If X, then Z.	assumed
(3)	Not Y.	assumed
(4)	If Z and X, then W.	assumed
(5)	X.	from 1 by "or"
(6)	Z.	from 2 and 5 by "if...then"
(7)	Z and X.	from 6 and 5 by "and"
(8)	W.	from 4 and 7 by "if...then"

Smith, 2020 has a more extensive example, borrowed from Lewis Carroll, that helps to illustrate how rules for the simple inference steps can be put to use in such a fully-justified proof. Consider the following informal argument that he labels (**D**):

D Anyone who understands human nature is clever; every true poet can stir the heart; Shakespeare wrote *Hamlet*; no one who does not understand human nature can stir the heart; no one who is not a true poet wrote *Hamlet*; so Shakespeare is clever. Smith, 2020[p31]

Just two rules are needed to prove Carroll's argument, specifically.

Rule U:
Any/every B is C.
a is B.
$\therefore$ a is C.

Rule V:
No one who is not A is B.
$\therefore$ Any B is A.

Like before, we take each inference step one at a time and list the lines that we used to justify it. But now we can also add specific rule to our justifications, allowing inference steps only when such a rule is available (from Smith, 2020[p31]).

(1)	Anyone who understands human nature is clever.	assumed
(2)	Every true poet can stir the heart.	assumed
(3)	Shakepeare wrote *Hamlet*.	assumed
(4)	No one who doesn't understand human nature can stir the heart.	assumed
(5)	No one who is not a true poet wrote *Hamlet*.	assumed
(6)	Anyone who wrote *Hamlet* is a true poet.	from 5 by V
(7)	Shakespeare is a true poet.	from 6 and 3 by U
(8)	Shakespeare can stir the heart.	from 2 and 7 by U
(9)	Anyone who can stir the heart understands human nature.	from 4 by V
(10)	Shakepeare understands human nature.	from 9 and 8 by U
(11)	so Shakespeare is clever.	from 10 and 1 by U

Returning to more concrete examples, we can see that many arguments in ordinary English have assumptions that are silent or otherwise taken for granted. So the following argument about my dog is correct.

It is a perfect day for fetch.
$\therefore$ Bandit is happy.

But it is missing an important premise. Only once sentence (*) has been added do we have an argument that is deductively valid.

It is a perfect day for fetch.
(*) If it is a perfect day for fetch, then Bandit is happy.
$\therefore$ Bandit is happy.

Definition 1.3 (Enthymeme)**:** An argument with unsaid assumptions is called an *enthymeme*.

Logic is exceptional for detecting invalid arguments but cannot help at all with assessing the truth of premises.

> The important thing to remember is that validity is not about the *actual* truth or falsity of the sentences in the argument. It is about whether it is *possible* for all the premises to be true and the conclusion to be not true at the same time. (Magnus et al., 2025, p15)

This important point becomes a little more clear when reflecting on the contrast between *valid* inferences and *sound* inferences. A valid argument is just a matter of following the rules.

Definition 1.4 (Valid Argument)**:** An argument is *valid* iff all of the inference steps from its premises to its conclusion are allowed by the relevant rules of logic.

But a sound argument requires that we do not make any untrue assumptions.

Definition 1.5 (Sound Argument)**:** An argument is *sound* iff (i) it is valid; **and** (ii) all of its premises are true.

Our formal logics will have plenty to say about valid arguments, but will remain mostly mute on the subject of sound arguments. Additionally, formal logic is limited to the assessment of *deductive* arguments. It does not help with the assessment of the many *inductive* arguments that populate both empirical sciences and everyday experience.

The sun has risen every day so far.
$\therefore$ The sun will rise tomorrow.

While this is a perfectly reasonable inference to make, it is not a deductivley valid argument. What do we mean by this?

Definition 1.6 (Deductively Valid Argument)**:** A *deductively valid argument* is an argument where the conclusion follows directly from the premises via only a series of deductively valid steps: *inference steps* such as those informally demonstrated above, where the conclusion follows from the premise(s) in *every situtation*.

We will not discuss *inductive arguments* here, except to note that they are not deductively valid. But we will need to eventually get a handle on what we mean by "situation". First, we consider what kinds of inference steps are appropriate to a deductive system of sentences.

Beyond valid inference steps, we can also define some other properties of sentences and relations between them:

Definition 1.7 (Some logical properties of sentences)**:** Sentences and collections of sentences can be assigned labels based on their truth in some or all situations:

1. Sentences that are always true are *necessary truths*.
2. Sentences that are never true are *necessary falsehoods*.
3. Sentences that are neither necessary truths nor necessary falshoods are *contingent sentences*; they are sometimes true and sometimes false.
4. Pairs of sentences that have the same truth value in every situation are *necessarily equivalent*.
5. Collections of sentences that are all true together in some situation are *jointly possible*;

those that are not are *jointly impossible*.

Negative results play an important role in the processes of logical deduction. Most blatantly, if a sentence is not true, then its negation must be. The fact that this rule works exceptionlessly is an important part of classical logic. In a similar vein, we repeat the following rule from Smith, 2020, p. 13.

Definition 1.8 (The Invalidity Principle)**:** An inference step with actually true premises and an actually false conclusion must be invalid.

Thus when we find a counter-example to an argument, the argument is not valid.

One ancient approach to logical proofs is to show that assuming the opposite of a conclusion will lead to a contradiction. We will see how the basic treatment of negation leans heavily on this inference when we start to explore the syntax and semantics of actual logics next week.

Definition 1.9 (Reductio Ad Absurdum)**:** If $\phi_1 \dots \phi_n$ are taken as premises and the addition of ψwould result in a contradiction, then $\phi_1 \dots \phi_n$ entail not-ψ.

People often struggle taking reductio ad absurdum (RAA) seriously when they are first introduced to it. One naive reading would take the following to be an acceptable argument that Elvis is still alive.

John lives in Amsterdam.

It is not the case that John lives in Amsterdam.

$\therefore$ Elvis is still alive.

No one likes this result, not even proponents of RAA. The most compelling complaint is that Elvis' ongoing survival likely has nothing at all to do with John's residence in the Netherlands. Still, like democracy, this is the worst outcome except for all the rest. Most logicians are willing to accept this result, despite its oddity, simply because the simplicity offered by the overall system is worth preserving.

And it's worth noting that these examples are arguably just edge cases to begin with. Indeed, by our existing definition of RAA the premises $\phi$and $\neg\phi$are already inconsistent. This has a bunch of implications which suggest that this premise set simply never comes up for serious consideration.

1. *John lives in Amsterdam* entails *It is not the case that John does not live in Amsterdam.*
2. *John lives in Amsterdam* and *John does not live in Amsterdam* together entail *Elvis is not alive.*

Nevertheless, we will see RAA used all the time in service of formal proofs.

So far we showed how to use capital letters as stand-ins for sentences (i.e. as *schematic variables* in logical arguments and described this is as fundamental to formal logics. Now we will discuss this process–often called *symbolization* or *translation*–in more detail.

As we pull apart arguments attempting to ascertain their form, a basic distinction that emerges is between *content expressions* and *topic neutral terms*. So far, we have represented the former using capital letters (A, B, C, X, Y, Z) and the latter with their ordinary English counterparts (*and, not, either...or, if...then*. The important thing to note is that formal logic is concerned primarily with the behavior of topic-neutral terms, abstracting away the content expressions into mostly inert symbols. *Logical validity*[4] is just deductive validity by virtue of topic-neutral terms alone. This is what *purely logical schemas* represent.

Consider again the two mini-arguments presented above, which we will give simple names for reference.

And Rule:
Both X and Y.
$\therefore$ Z.

Or Rule:
Either X or Y.
Not Y.
$\therefore$ X.

Thinking of **And Rule** and **Or Rule** as *proto-schemas*, we can glean a few informal observations:

Observation 1.1 (Properties of Argument Proto-Schemas)**:** Argument schemas have some regular properties

- Schemas are intended to be a somewhat "loose fit" with the English expressions that they are intended to symbolize. The same logical form might apply to a whole range of different (but related) sentences of ordinary English.
- Schemas depend on the radical abstraction of content expressions.
  - Letters are used to symbolize the content expressions of the object language.
  - The meaning of the expressions that these symbols represent is dropped or simplified dramatically.
- The choice of content symbol is relatively unimportant, we just need to keep straight "which ones are which."

The last point is worth lingering on. We can paraphrase it by saying that schemas depend on the equivalence of substitution instances. In our rules, we will use greek letters (ϕ, ψ, χ, ...) to represent arbitrary expressions (mostly sentences):

[4]and *logical entailment*

And Rule:

ϕandψ

$\therefore$ ψ

Or Rule:

ϕorψ

notϕ

$\therefore$ ψ

By choosing a metalanguage variable in this role, we are stressing and important property of our schemas, namely that they can be applied to all of the sentences that share the form in question. Recall that we can describe a argument using sentence letters to reveal its deeper formal structure. For example, we can stipulate that each of the following inference steps is valid in virtue of its form alone.

A	A or B	not-(A and B)
If A, then C	not-A	A
$\therefore$ C	$\therefore$ B	$\therefore$ not-B

The logic of sentences that we define is called *Truth-Functional Logic* (TFL) or *Propositional Logic* (PL), depending on the author. We will use TFL but take the two names to refer to one and the same logic.

I mentioned that logic is concerned with arguments built out of declarative sentences. However, there are good reasons use a different name for the so-called sentences of formal logic. I repeat here just a few, as listed by Smith, 2020[pp54-55]

Observation 1.2 (TFL sentences are not like sentences of English)**:**

1. Some grammatical English declarative sentences lack an interpretation and thus cannot be assigned a truth-value based on their sense.
2. Some grammatical English sentences have more than one interpretation.
3. Some TFL sentences can be used to express more than one grammatical English sentence.
4. Some English sentences cannot be fully interpreted without consideration of the context.
5. Surely there is a sense in which Aristotle and Frege are making many of **the same** arguments that we will consider: arguments that we would surely be mistaken to take as sentences of ancient Greek, late nineteenth century German or contemporary English
6. TFL sentences cannot be distinguished except in their truth or falsity.

Nevertheless, we will not adopt a specialized term (such as *proposition*) for TFL sentences, noting only that we are using *(TFL) sentence* as a technical term, not as a synonym for the ordinary English concept *sentence*.

Recall that by limiting ourselves to sentence letters, we are marking off an explicit boundary in our logic between those arguments that depend in some way on the "content" of their sentences and those that depend on form alone.

When we look at Predicate Logic (Quantificational Logic) later this semester, we will see how to incorporate the contributions of names and properties so that we can encode **some** of the contributions that (sub-sentential) content makes to the rules of inference. But, for now, we will focus on just sentences and the forms of the arguments that they appear in.

> At best, TFL will help us to understand arguments that are valid due to their form.
> Magnus et al., 2025, p30

In the examples above, we have some sentences that contain *sentence letters* themselves. For example: *If A, then B*. When we use capital letters to stand for sentences (or subsentences) in this way, we are assuming that the sentences represented are somewhat simple and can be assessed as *true* or *false*. We will call sentences of this sort *atomic sentences*. They cannot be broken down into parts and we cannot "peek" inside them to support our inferences.

When we use letters to represent sentences in this way, we will often provide a *symbolization key* to help decode the arguments.

A: It is a perfect day for fetch.

C: Bandit is happy.

Of course, these devices are *ad hoc* since TFL is a formal logic and the actual contents of its sentences is immaterial to the analysis.

We can build complex sentences from simpler ones (including atomic sentences) with the help of a few *connectives*. As the name suggests, the connectives of TFL are *truth-functional*. In other words, we will interpret the truth of a complex sentence as a function of the truth of its component subsentences based on the meanings of its connectives and the order that they are introduced.

Consider the simple one-place connective $\neg$. We can understand this connective as a function from a truth-value to its opposite. So, for example, when A is true, $\neg A$ is false. We can summarize the basic TFL connectives as in the following table.

Function	**Symbol**	**Paraphrase**
Negation	$\neg$	'It is not the case that ...'
Conjunction	$\wedge$	'Both ... and ...'
Disjunction	$\vee$	'Either ... or ...'
Conditional	$\rightarrow$	'If ... then ...'
Biconditional	$\leftrightarrow$	'... if and only if ...'

Table 2: Basic TFL Connectives

The symbolization of sentences is subtle and not always a simple preservation of the surface forms found in the English version. Consider each of the connectives in turn.

R: A recession is inevitable.

P: A recession can be prevented.

If P is part of our symbolization key, we might want to symbolize R as $\neg P$ to capture the relationship between the sentences, even though *not* and *it is not the case that* do not occur anywhere in the sentence. Here is where the concept of *paraphrase* comes into play. We will want to lean pretty heavily on the flexibility of paraphrase, especially when dealing with words that have antonyms or other 'lexical' negations with *un-*, *non-*, *de-*, and so on.

We see a similar separation of English forms and TFL forms in our treatment of conjunctions. If we want to symbolize *Barbara is a medical doctor and a philosopher*, we need to depart from the English form and treat it as a conjunction of sentences ($M \wedge P$) instead of as one sentence (*Barbara M and P*, perhaps). That is, we need to mention Barbara in each of the sentences, not just the once. Our symbolization key would be:

M: Barbara is a medical doctor.

P: Barbara is a philosopher.

Note again that the semantics of TFL is bare bones. We will bundle under conjunction several English connectives than have slightly different implications in their full meaning: *and*, *but*, and *although* in the defintion, but many more in practice. In addition to collapsing these English connectives to one in TFL, we will also ignore the implications of sentence order.

A: John left.

B: Mary got upset.

Although *A and B* and *B and A* have different implications about the order of events–and perhaps even about cause–in TFL $A \wedge B$ says the exact same thing as $B \wedge A$. But the interaction of the operators is important. The following have a different meaning in TFL, even though each could be the correct symbolization of the English sentence schema *It is not the case that A and B*.

$$\neg(A \wedge B) \qquad\qquad (\neg A \wedge B)$$

We need to take care in the order we apply the connectives.

With these preliminaries out of the way, we can now provide informal definitions for each of the basic TFL connectives by way of paraphrase. We will use lower-case Greek letters ϕ,ψ,χ,... to represent arbirtary TFL sentences.

Definition 1.10 (Negation)**:** A sentence can be symbolized as $\neg\phi$if it can be paraphrased as "It is not the case that ϕ."

Definition 1.11 (Conjunction)**:** A sentence can be symbolized as $\phi\wedge\psi$if it can be paraphrased as "Both $\phi$and ψ", as "ϕ, but ψ" or as "Although ϕ, ψ".

Definition 1.12 (Disjunction)**:** A sentence can be symbolized as $\phi \vee \psi$if it can be paraphrased as "Either ϕor ψ".

Definition 1.13 (Conditional)**:** A sentence can be symbolized as $\phi \rightarrow \psi$if it can be paraphrased as "If ϕ, then ψ" or as "ψonly if ϕ".

Definition 1.14 (Biconditional)**:** A sentence can be symbolized as $\phi \leftrightarrow \psi$if it can be paraphrased as "ϕif and only if ψ".

2 Truth and Grammar in TFL

So far, despite some basic symbolization as stand-ins for sentences, our discussion has been entirely informal: our arguments are built out of ordinary English sentences; our symbolizations are likewise parphrases of English.

This week we will take our first steps to *formal logic* by describing the *syntax* of Truth-Functional Logic (TFL). We will learn what we mean by truth-functional when we discuss the *semantics* of TFL. For the time- being we will concentrate on the *form* in formal.

First and foremost, we want to know what "counts" as a sentence in TFL. We can start with a few definitions about the things that might be sentences: *expressions*.

Definition 2.1 (Basic TFL Expression)**:** A *basic expression* in TFL is any of the following symbols:

1. atomic sentence letters: A, B, C
2. connectives: $\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$
3. brackets: (,)

We will refer to the collection of basic expressions in TFL as its *alphabet*.

Note that the atomic sentences can be made both *enumerable* and *potentially infintite* with the addition of subscripts and/or superscripts: A_4, B^2, C_1^2, etc.

Definition 2.2 (TFL Expression)**:** An *expression* in TFL is any sequence of basic TFL expressions.

Many of these expressions will be nonsense. So we are really only interested in some of these expressions: the *sentences* of TFL. There are potentially infinitely many of these, so we will have to define the sentences *recursively* using rules. That is, we will need to explain how to construct sentences from expressions.

Definition 2.3 (TFL Sentence)**:** A *sentence* in TFL is defined by the following rules:

1. Every atomic sentence letter is a sentence.
2. If $\phi$is a sentence, then $\neg\phi$is a sentence.
3. If $\phi$and $\psi$are sentences, then $(\phi \wedge \psi)$ is a sentence.
4. If $\phi$and $\psi$are sentences, then $(\phi \vee \psi)$ is a sentence.
5. If $\phi$and $\psi$are sentences, then $(\phi \rightarrow \psi)$ is a sentence.

6. If $\phi$and $\psi$are sentences, then $(\phi \leftrightarrow \psi)$ is a sentence.

7. Nothing else is a sentence.

A sentence is also called a *well-formed formula* (*wff*).

This is called an *inductive* defininition: it uses rules to build infinitely many complex expressions from a small number of basic ones. Inductive defiitions form the backbone of most systems of deductive logic, both in syntax and semantics. Notice that the last clause ensures that there are many TFL expressions that are not TFL sentences. But of course every TFL sentence is a TFL expression.

Before diving into more detailed issues of TFL syntax, we should pause to comment on some of the differences between the connectives of TFL and their english analogues.

First, TFL $\neg$is *logical*. Specifically, it obeys three important inter-related laws:

LEM: $\phi \vee \neg\phi$
LNC: $\neg(\phi \wedge \neg\phi)$
DN: $\neg\neg\phi \leftrightarrow \phi$

We will explore these laws in more detail later. For now it suffices to say that they mean that every sentence is either true or false, never anything in between. And if A is true, then not-A is false. English "not" does not work like this and can be applied to all sorts of English expressions, not just to sentences.

Second, in English the order of two sentences connected by "and" is important. We illustrated this last week with the following example:

A: John left.
B: Mary got upset.

Suffice it to say, *A and B* can express something different than *B and A* ijn English. Typically this will be something about the order of the events described in A and B, but also sometimes has to do with causal relations between them. The TFL connective $\wedge$, on the other hand, is *commutative*. $A \wedge B$ always means the same thing as $B \wedge A$.

Third, English "or" is often understood with an *exclusive* reading: *either A or B, but not both*. But TFL $\vee$is understood in an *inclusive* way: *A or B, and maybe both*.

Finally, the rules of grammar guarantee that there is no structural ambiguity in the sentences of TFL. Each sentence can be decomposed to discover exactly one truth-functional meaning. English, of course, is wildly ambiguous. We will see a demonstration of this in just a moment.

The rules in 2.3[Definition] allow us to identify a few important components of complex sentences.

Definition 2.4 (Main Logical Operator)**:** The *main logical operator* of a sentence is the last connective applied in its construction.

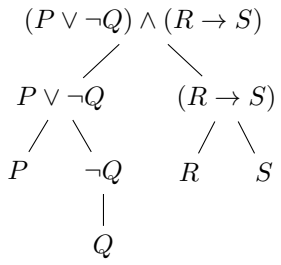
Given our definition of TFL sentences, we know that there is a unique main logical operator for any (non-atomic) sentence. Looked at from the other direction, we can say that the *scope* of a connective is just the new sentence that its rule makes.

Definition 2.5 (Connective Scope)**:** The *scope* of a connective is the sentence for which it is the main logical operator.

When checking to see whether some expression is a sentence, we will need to work backward through these rules: break the expression apart into component parts until we are left with only atomic sentence letters. Doing this is usually straight-forward and involves finding the main logical operator (if any) in the expression under review. Indeed, our bracketing conventions make the main logical operator and its scope almost immediately apparent. But as we get better at reading complex expressions, we will treat some brackets more and more as optional or only occasionally obligatory. For the time being, we will adopt the conventions in Magnus et al., 2025 by omitting outermost brackets and sometimes substituting [, and] for clarity and readability.

One strategy that we can use to check the well-formedness of a formula is a *parse tree*. The idea is that we can *recusrively* break the formula down based on the grammar rule for the main connective. We can illustrate this process visually in the form of a tree, but traditionally we use a tree with its branches pointed down.[5]

Consider the following example of a parse tree.



One of the important properties of parse trees is that each is *uniquely* associated with a well-formed formula (wff), written at the top (the *root*). Likewise, every one of its *nodes* and *leaves* is labeled with a well-formed formula (wff), i.e. a TFL sentence. Of course, this is just because parse trees are built with a specific process that finds the main connective and unfolds it by applying the relevant rule in reverse.

[5]Smith, 2020 also introduces a notion of *construction history* (p74) that "grows" upward and serves as a stepping-stone to parse trees. But for our purposes, this seems redundant.

Observation 2.1 (Parse Tree Uniqueness)**:** Every TFL sentence has a unique parse tree.

Another useful property of parse trees is that we can read important structural information directly off of them. For example, evey *node* in a parse tree is decorated with a *subformula* of wff. Indeed, the subformulas are also exhaustively enumerated by the node decorations.

Observation 2.2 (Nodes are subformulas)**:** Every node in a parse tree for a sentence $\phi$of TFL is decorated with a subformula of ϕ.

Since the *scope* of the main connective of a (sub)formula is just the (sub)formula itself, we can define this notion via parse trees as well.

Definition 2.6 (Scope of a Connective)**:** The *scope* of a connective is the subformula decorating the node where it is first introduced, reading from the *leaves* of its parse tree up.

In ordinary English, we can find two distinct types of *ambiguity*. *Lexical ambiguity* is when on word can mean more than one thing. A popular example is the word *bank* which can be either a financial institution or the edge of a river. *Structural ambiguity*, on the other hand, is when a sentence can have more than one meaning based on how the parts work together. A popular example is:

(T) The astronomer saw his student with a telescope.

The sentence in **(T)** can have two different readings: one where the astronomer has the telescope and one where the student does. A closely related concept is *vagueness* which describes terms that are imprecise and often difficult to apply, such as "tall" or "honorable". For the purposes of our logics, we will assume an omniscience about these terms that belies this vagueness: in our models we will just suppose that we know for each of these terms exactly when they apply. Like other content terms, their precise meanings are not important to our rules.

Since Frege set out the preliminaries of propositional logic in the late nineteenth century, logicians have made an important distinction between two different kinds of meaning that an expression (say a singular term) can have. On the one hand, it picks out something in the world (or domain, or whatever) that we are taking our sentence to be *about*. Frege called this type of meaning *reference* (*Bedeutung* in the orginal German). On the other hand, there is what the expression *means* in some more abstract way (that is mostly *mental*, but also partly *social*). This aspect of the meaning of an expression is called *sense* (*Sinn*).

The touchpoint between our ordinary English content and the symbolization of TFL sentences is found in the *symbolization key* that we construct for a given argument. We can think of such a key as a *glossary* of the content expressions used in the logic. Recall that these expressions in truth- functional logic are limited to sentences; and that, following Frege, we take the *reference* of these sentences to be just their *truth values*. Nevertheless, we can set these values by taking the "real world" truth value of the sentences expressed in the symbolization key. In other words, each entry can be thought of as just a *gloss* of the *sense* of the content symbols used in the logic. But

importantly, what we care about is the reference that the gloss fixes, not the sense itself. We will see how this works in a moment.

As intimated above, TFL sentences are **never** structurally ambiguous. And we can make efforts in formulating our symbolization keys to minimize, if not elminate, both vagueness and lexical ambiguity. That said, all three forms of ambiguity are rampant in ordinary language and our efforts at symbolizing complex arguments from English will need to proceed with due care. Unlike TFL, English does not have bracketing rules. We will refer to the structural ambiguities that can produce more than one TFL sentence as *scope ambiguities*.

Consider the following sentence and symbolization key:

(P) The professor is literate or not dumb and lazy.

A: The professor is literate.
B: The professor is dumb.
C: The professor is lazy.

We can identify a few scope ambiguities, offering plenty of opportunities for the professor to be illiterate, dumb or lazy but not, as it turns out, all three. Notice how the bracketing helps to distinguish between the different *readings* of the English sentence being symbolized.

$(A \vee \neg B) \wedge C$
$A \vee \neg(B \wedge C)$
$A \vee (\neg B \wedge C)$

In our informal discussion, it was convenient for us to mix together the sentences of the logic we were discussing and the sentences about that logic. Now that we are dealing with formal logic, we will need to be a bit more careful. We will also need to be clear about when we are using English in an ordinary way versus when we are using it to talk about a language, whether English or TFL. To this end, a few more conventions and definitions are in order.

The most common indicator of level-mixing is quotation marks. Consider the following examples.

A: "Aristotle" is not the original Greek spelling.
B: Aristotle taught Alexander the Great.

Whereas B is about an English word, A is about the ancient philosopher. This is an importnant distinction: the former is called *use*; and the latter is called *mention*. Speaking about the phenomenon more generally, we can adopt the following definitions.

Definition 2.7 (Object Language and Metalanguage)**:** When talking about a language such, we call the language that we are talking about the *object language* and the language that we are using to talk about it the *metalanguage*.

For the purpuses of truth-theoretic logic (TFL), our object language is made of TFL sentences and

our metalanguage is English mixed with greek letters and a few other textual decorations mixed in for precision and convenience.

Since TFL and our metalanguage do not use quotes, we will rarely need to worry about them. The most important point to make here is that we will use English as our metalanguage when discussing TFL (and later FOL). But we will make some adjustments to simplify our presentation. As we have already seen in 2.3[Def], we will use Greek lower case letters "ϕ", "ψ","χ", etc as *metavariables* for arbitrary TFL sentences. Upper case Latin letters "A", "B", "C", etc will continue to be used as atomic sentence letters. After now, we will rarely mention these variables, even in the metalanguage. So the quotation marks will almost always be omitted. [6]

One more special feature of our English++ metalanguage is the conclusion marker ($\therefore$), which we've seen already in the informal discussion of arguments. We can use someshorthand to formalize the notation in our metalanguage as well. For instance, we can describe a sequence of premises and conclusion that comprise an argument as:

Definition 2.8 (Symbolized Argument)**:** $\phi_1, \ldots, \phi_n \therefore \psi$

But we will try to strike a healthy balance in the metalanguage between too much verbiage and too much formalism.

TFL is *truth-functional* as a result of its *semantics*. Put simply, we evaluate the *truth* of a complex sentences as a *function* of the truth of their component parts. Ultimately this is a complex function of the truth of all of its unique atomic sentences. We can define the notion with an adaptation of the definition in Magnus et al., 2025, p. 72

Definition 2.9 (Truth-Functional)**:** A TFL connective is *truth-functional* iff the truth value of the sentence comprising its scope (i.e. the sentence for which it is the main logical operator) is uniquely determined by the truth values of the sentence(s) it connects.

Consider our informal definition of $\neg\phi$.

> A TFL sentence of the form $\neg\phi$is true just in case $\phi$is false.

We can see right away how this sort of principle works in practice. The rule is simple. But actually working with TFL sentences within the system outlined by TFL requires a substantial shift in how we think about these 'sentences'. Indeed, even this first definition hints at some important features of (classical) logic that are different than those in ordinary English. We will discuss these in more detail later, but one is that every TFL sentence is either true or false; if it is not true, then it is false; if it is not false, then it is true. This rule (the Law of the Excluded Middle) is exceptionless in TFL. We will formulate an axiom to that effect later.

> PL languages are not mere *shorthand code* for bits of English; their whole point is to be usefully tidied-up *replacements* for the messy vernacular. (Smith, 2020, p. 83)

[6] When they are not, we consider them to be part of the mention.

Recall that the process of symbolization for TFL loses a lot of the force or affect that we often take for granted in ordinary language. This list of sentences taken from Magnus et al., 2025, p. 73 all have the same truth-value and can be symbolized with the same TFL sentence ($A \wedge B$), despite a variety of shaded meanings.

Example 2.1: Sentences that can be symbolized as $A \wedge B$:

1. Dana is a logician and Dana is a nice person.
2. Although Dana is a logician, Dana is a nice person.
3. Dana is a logician despite being a nice person.
4. Dana is a logician and Dana is a nice person.
5. Dana is a nice person but also a logician.
6. Dana's being a logician notwithstanding, he is a nice person.

The collapse of these diverse sentences to a single symbolization in TFL happens precisely **because** its semantics is truth-functional. To see this, it will help to look at the important terms one at a time: *truth*, *truth-value* and *function*.

So far we have used "truth" to mean something like a metalogical property that applies to English sentences. On this informal usage, we can fall back on whatever folk definition we like. This one from American Heritage Dictionary will do as well as any.

Definition 2.10 (Informal Truth)**:** *noun*[a]

1. Conformity to fact or actuality.
   "Does this story have any truth?"
2. Reality; actuality.
   "In truth, he was not qualified for the job."
3. The reality of a situation.
   "The truth is, she respects your work."

[a]The American Heritage® Dictionary of the English Language, 5th Edition

We use this notion in understanding our symbolization keys. On our informal treatment, the following key tells us to consider the truth of A and B to be a matter of John and Mary's feelings about each other "in reality."

A: John loves Mary.

B: Mary loves John.

But in TFL, what the key really says is that A has the same *truth value* that the English sentence "John loves Mary" does in reality. We are using *truth value* here as a technical term.

Definition 2.11 (Truth Value)**:** A truth value is one of two values: *True*) and *False* (symbolized in the metalogic as T and F, respectively. Every atomic sentence in TFL is assigned one or the other of these values and every complex sentence in TFL can be assigned one on the basis of its atomic parts.

Recall from algebra that a *function* takes one or more inputs (*arguments*) and produces an output (a *result*). In TFL both the arguments and the output of truth-functional connectives are truth values. Put differently, the symbol for a TFL sentence is best understood **as** its truth value. The meanings of the English sentences provided in the symbolization key are just guides to assigning those values properly. Meaning plays no role in TFL, only truth. This is an important point that takes some time to understand.

Definition 2.12 (Truth Function)**:** A *truth function* is a function which takes one or more truth values as its *argument(s)* and returns a truth value as its output.

One consequnce of the truth-functional treatment of sentences and connectives is that we can now fully define the semantics of a TFL connective on the basis of how the truth of its complex sentence is determined by the truth of the sentences it connects. We do this with the help of truth tables. Truth-tables exhaustively spell out the consequences of assigning truth-values to the components of a sentence. Note that characteristic truth tables use the metalogical variables $\phi$and ψbecause the rules can be applied to arbitrary TFL sentences, not just to atomic ones.

Recall that we earlier highlighted three generalizations about classical negation.

LEM: $\phi \vee \neg\phi$
LNC: $\neg(\phi \wedge \neg\phi)$
DN: $\neg\neg\phi \leftrightarrow \phi$

We can summararize the aggregate of these rules (which, as we will see later, really are all interrelated) by saying that TFL is *bivalent*.

Definition 2.13 (Bivalency)**:** A truth-functional language $\mathcal{L}$ is *bivalent* (i.e. *two-valued*) iff every sentence $\phi$of $\mathcal{L}$ is associated with **exactly one** of the two truth values, T and F, in every situation.

Definition 2.14 (Semantics of $\neg$)**:** For any TFL sentence ϕ, $\neg\phi$is true iff $\phi$is false.

We can flesh out the formal definitions for our major connectives by considering what would happen if we assigned either of the possible truth values to the sentences that they apply to. We will call this presentation a *characteristic truth table*. For the negation operator $\neg$, the table is simple, because it "connects" just one sentence. Let's call it ϕ.

Definition 2.15 (Charateristic Truth Table: Negation)**:** The semantics of the negation operator ¬is captured by the following characteristic truth table:

ϕ	$\neg\phi$
T	F
F	T

There is only one argument: the sentence negated; and that sentence has only two possible truth values: T and F.

Definition 2.16 (Semantics of ∧)**:** A TFL sentence of the form $\phi\wedge\psi$is true iff both $\phi$and $\psi$are true.

For the two-place connectives, there are two arguments: one for each of the sentences connected; and each of those sentence has only two possible truth values: T and F. If we vary them independently for each sentence, we get four total possibilities.[7]

Definition 2.17 (Charateristic Truth Table: Conjunction)**:** The semantics of the connective ∧is captured by the following characteristic truth table:

ϕ	ψ	$\phi\wedge\psi$
T	T	T
T	F	F
F	T	F
F	F	F

Definition 2.18 (Semantics of ∨)**:** A TFL sentence of the form $\phi\vee\psi$is true just in case either $\phi$is true or $\psi$is true (or both).

Definition 2.19 (Charateristic Truth Table: Disjunction)**:** The semantics of the connective ∨is captured by the following characteristic truth table:

[7]Note that the TTFF and TFTF ordering is conventional for TFL truth tables. More generally, propositions to the left are *chunked* and those on the right are *interleaved*. More on this later.

ϕ	ψ	$\phi \vee \psi$
T	T	T
T	F	T
F	T	T
F	F	F

The semantics of conditionals is straight-forward, but not entirely without controvery. We will present the definition now, but postpone discussion of the shortcomings.

Definition 2.20 (Semantics of $\rightarrow$)**:** A TFL sentence of the form $\phi \rightarrow \psi$is true just in case either $\psi$is true or $\phi$is false.

Definition 2.21 (Charateristic Truth Table: Conditional)**:** The semantics of the connective '$\rightarrow$' is captured by the following characteristic truth table:

ϕ	ψ	$\phi \rightarrow \psi$
T	T	T
T	F	F
F	T	T
F	F	T

Definition 2.22 (Semantics of $\leftrightarrow$)**:** A TFL sentence of the form $\phi \leftrightarrow \phi$is true just in case $\phi$and $\psi$are either both true or both false.

Definition 2.23 (Charateristic Truth Table: Biconditional)**:** A TFL sentence of the form $\phi \leftrightarrow \phi$is true just in case $\phi$and $\psi$are either both true or both false. The semantics of the connective '$\leftrightarrow$' is defined by the following characteristic truth table:

ϕ	ψ	$\phi \leftrightarrow \psi$
T	T	T
T	F	F
F	T	F
F	F	T

Instead of depending on our knowledge of reality to assign truth-values to atomic sentencs as we do in the informal assessment of truth in our symbolization tables, we can simply assign values to each of the atomic sentence letters used in our logic. Recall that we think of a symbolization key as

a glossary of the *senses* of the atomic sentences in our logic. But TFL expects their *references* (i.e. the truth values themselves). We will define a *valuation* as the specification of the truth values for all the atomic sentences in a given instance of TFL.

Definition 2.24 (Valuation)**:** A *valuation* is any assignment of truth values to the atomic sentence letters of TFL. On a valuation, each atomic sentence letter is assigned exaclty one truth value.

The completeness of a TFL valutation is refered to as *determinacy*.

Definition 2.25 (Determinacy)**:** For any TFL language, once we have stipulated the senses of its atoms–fixed their truth conditions–each of these atoms will be determinately true or determinately false in any possible situation. The atoms of a TFL language are to be interpreted so that they have determinate truth values, exactly one of T and F, in any situation. (modified from Smith, 2020, p. 84-5)

But, of course, ordinary English often lacks such determinacy. Nevertheless, we will see that it is an important aspect of classical logical systems. One such aspect is the resulting determinacy of **all** the sentences of TFL, both atomic and complex.

Observation 2.3 (Valuations uniquely determine all truth)**:** Note that determinacy is contagious via rule application.

1. A valuation of all the atomic sentences of any TFL sentence ϕ *uniquely determines* a resulting valuation of ϕ.
2. A valuation of all the atomic sentences of any TFL language $\mathcal{L}$ *uniquely determines* a resulting valuation of $\mathcal{L}$, however complex.

We can take a deterministic valuation to be a stand-in for what we were earlier calling a *situation*. With this meaning, a situation would just be the state of affairs defined by the truth values as they are set in the valuation. A truth table for such a valuation is easy to construct because we only need to consider one option for each sentence: the actual truth value that is assigned to it under the valuation. It will have just one row.

A	B	$(A$	$\wedge$	$B)$	$\rightarrow$	A
F	T	F	F	T	**T**	F

The truth value (F) of the entire formula $((A \wedge B) \rightarrow A)$ is bolded in the column under the main connective. This formula is *vacuously true*, because its antecedent is false.

But full truth tables are *exhaustive* in the sense that every possible valuation for a given set of sentences can be represented as a row in the table, with 2^n such rows for a TFL sentence with

n different atomic sentence letters. How this works in practice is best demonstrated by example. Consider the TFL sentence $(A \wedge B) \rightarrow A$. We can see how the truth of the whole depends on the truth of its parts by working through each of the possible valuations "from the inside out".

First, set up the table with all of the possible atomic truth values.

A	B	$(A$	$\wedge$	$B)$	$\rightarrow$	A
T	T					
T	F					
F	T					
F	F					

Then add the atomic truth values to the appropriate columns on the right side of the table.

A	B	$(A$	$\wedge$	$B)$	$\rightarrow$	A
T	T	T		T		T
T	F	T		F		T
F	T	F		T		F
F	F	F		F		F

Now we can evaluate the truth of the sentences with connectives, starting with the "innermost" ones (i.e. the ones that connect only atomic sentences). Here we start with the $\wedge$connective.

A	B	$(A$	$\wedge$	$B)$	$\rightarrow$	A
T	T	T	**T**	T		T
T	F	T	**F**	F		T
F	T	F	**F**	T		F
F	F	F	**F**	F		F

We continue the process until we reach the main logical operator (i.e. the outermost connective). The truth value assigned at this column is the truth value for the entire sentence.

A	B	$(A$	$\wedge$	$B)$	$\rightarrow$	A
			ϕ		$\rightarrow$	ψ
T	T	T	T	T	**T**	T
T	F	T	F	F	**T**	T
F	T	F	F	T	**T**	F
F	F	F	F	F	**T**	F

Notice how we can use the metalogical sentence variables as substitutes for more complex TFL sentences. Here they are really being used as *logical* sentence variables, representing not arbitrary sentences but these specific two:

ϕ $A \wedge B$
ψ A

Returning to our example from last week about the literacy of a certain professor, we can see how this approach becomes complicated fast. Recall that the three readings we found for our example were:

ϕ $(A \vee \neg B) \wedge C$
ψ $A \vee \neg(B \wedge C)$
χ $A \vee (\neg B \wedge C)$

We can explore the different truth conditions for these sentences by expressing each of them in the form of a truth table. Now that we have three atomic sentences (A, B, and C), we will need $2^3 = 8$ rows to express all of the possible valuations relevant to this argument. These valuations are straight-forward, if time-consuming, to lay out no matter how many atomic sentences occur in our argument.

Now "possible" can mean a few different things, depending on how it is interpreted. Consider the following symbolization key.

A: John has his cake.
B: John ate his cake.

In one sense, we can just list out all of this possibilities. In that case, $A \wedge B$ will be true when both A and B are true. We can say that it is *c-possible* that both A is true and B is true, since this is one of the possibile combinations of true and false required by the truth-table method.

On the other hand, we all know that you can't AB. Once you've eaten the cake (B), it is gone ($\neg$A). Although treating both A and B as true is combinatorily possible, the situtation that it describes is not *realizable* and so we can say that it is not *r-possible*.

We will not lean heavily on these definitions except to note that when we talk about possible situations in TFL, we mean that they are c-possible.

Definition 2.26 (Complete Truth Table)**:** The complete truth table for a TFL sentence $\phi$is a truth table with each c-possible valuation; that is, for each possible combination of truth value assignments to all the unique atomic sentences found in ϕ.

We can construct a complete truth table for a TFL sentence with n unique atomic sentences using the following procedure:

1. Write each atomic sentence letter and the TFL sentence across the top, assuming a column for each of its connectives and atomic sentences.

2. Insert 2^n rows.

3. Populate the first column (the first n columns are the unique atomic sentence letters) by adding 'T' to the first half ($2^n/2$) of the rows and then false for the remaining half.
4. For each subsequent column $i <= n$, fill in the portion of the column directly to the right of each uninterrupted series of Ts (or Fs) by filling in the first half with 'T' and the second half with 'F'.
5. If done correctly, the last column of atomic letters (column n) should alternate 'T' then 'F' $2^n/2$ times.
6. Fill in the remaining columns according to the parse tree and connective semantics as above.

Now that we know how to build a truth table for three atomic sentences, we can build truth tables knowing that they are complete. First, set up the table with all of the possible atomic truth values.

A	B	C	$(A$	$\vee$	$\neg$	$B)$	$\wedge$	C
T	T	T						
T	T	F						
T	F	T						
T	F	F						
F	T	T						
F	T	F						
F	F	T						
F	F	F						

Copy the truth values of the atomic sentences into the approriate columns on the right side of the table.

A	B	C	$(A$	$\vee$	$\neg$	$B)$	$\wedge$	C
T	T	T	T			T		T
T	T	F	T			T		F
T	F	T	T			F		T
T	F	F	T			F		F
F	T	T	F			T		T
F	T	F	F			T		F
F	F	T	F			F		T
F	F	F	F			F		F

Apply the connective rules up the parse tree, starting with the atomic sentences.[8]

[8]Starting here, we will use the convention from Smith, 2020 where multiple new steps are indicated in the same table by numbering along the bottom.

A	B	C	$(A$	$\vee$	$\neg$	$B)$	$\wedge$	C
T	T	T	T	T	F	T	**T**	T
T	T	F	T	T	F	T	**F**	F
T	F	T	T	T	T	F	**T**	T
T	F	F	T	T	T	F	**F**	F
F	T	T	F	F	F	T	**F**	T
F	T	F	F	F	F	T	**F**	F
F	F	T	F	T	T	F	**T**	T
F	F	F	F	T	T	F	**F**	F
				2	1		3	

The truth of the main sentence is provided for each valuation in the bolded column. Similarly, the truth conditions for the other two sentences are expressed in the bolded columns of the following two complete truth tables.

A	B	C	A	$\vee$	$\neg$	$(B$	$\wedge$	$C)$
T	T	T	T	**T**	F	T	T	T
T	T	F	T	**T**	T	T	F	F
T	F	T	T	**T**	T	F	F	T
T	F	F	T	**T**	T	F	F	F
F	T	T	F	**F**	F	T	T	T
F	T	F	F	**T**	T	T	F	F
F	F	T	F	**T**	T	F	F	T
F	F	F	F	**T**	T	F	F	F
			1	**6**	5	2	4	3

A	B	C	$(A$	$\vee$	$(\neg$	B	$\wedge$	C
T	T	T	T	**T**	F	T	F	T
T	T	F	T	**T**	F	T	F	F
T	F	T	T	**T**	T	F	T	T
T	F	F	T	**T**	T	F	F	F
F	T	T	F	**T**	F	T	F	T
F	T	F	F	**F**	F	T	F	F
F	F	T	F	**T**	T	F	T	T
F	F	F	F	**F**	T	F	F	F
			1	**6**	4	2	5	3

If we compare the results in a single collapsed table, it is easy to see that these TFL sentences have different truth conditions; that is, these sentences not true at the same valuations. I've bolded the outliers to emphasize the differences.

A	B	C	ϕ	ψ	χ
T	T	T	T	T	T
T	T	F	**F**	T	T
T	F	T	T	T	T
T	F	F	**F**	T	T
F	T	T	F	F	**T**
F	T	F	F	**T**	F
F	F	T	T	T	T
F	F	F	F	**T**	F

Next lecture, we will discuss the semantic implications of these differences in detail. And we will look at some ways to simplify our process of constructing truth tables.

2.1 Expressive Adequacy

Armed with truth tables and a new found confidence in working through them, we can begin to explore some fundamental questions about the logic itself. Are these all of the truth-functional connectives that can be defined? Do we need them all or will just a few suffice? These are important topics in *metalogic*, the major focus of Logic 2. But we shouldn't be afraid to think through them a little here, using our new toolbox.

First, notice that we don't really need both $\wedge$**and** $\vee$.

Observation 2.4 ($\wedge$and $\vee$are mutually-definable with $\neg$)**:** The connectives $\wedge$and $\vee$could each be used in a definition of the other.

1. The sentence $\phi \wedge \psi$can be expressed without $\wedge$as $\neg(\neg\phi \vee \neg\psi)$.
2. The sentence $\phi \vee \psi$can be expressed without $\vee$as $\neg(\neg\phi \wedge \neg\psi)$.

We can see this by comparing them in truth tables.

ϕ	ψ	$\phi\wedge\psi$	$\neg$	$(\neg\phi$	$\vee$	$\neg\psi)$
T	T	**T**	**T**	F	F	F
T	F	**F**	**F**	F	F	T
F	T	**F**	**F**	T	F	F
F	F	**F**	**F**	T	T	T

ϕ	ψ	$\phi\vee\psi$	$\neg$	$(\neg\phi$	$\wedge$	$\neg\psi)$
T	T	**T**	**T**	F	F	F
T	F	**T**	**T**	F	F	T
F	T	**T**	**T**	T	F	F
F	F	**F**	**F**	T	T	T

Similarly, we can dispense with $\rightarrow$as long as we have $\vee$and $\neg$.

Observation 2.5 ($\rightarrow$can be defined with $\vee$and $\neg$)**:** The sentence $\phi \rightarrow \psi$can be expressed without $\rightarrow$as $\neg\phi \vee \psi$.

ϕ	ψ	$\phi \rightarrow \psi$	$\neg\phi$	$\vee$	ψ
T	T	**T**	F	**T**	T
T	F	**F**	F	**F**	F
F	T	**T**	T	**T**	T
F	F	**T**	T	**T**	F

Finally, we can dispense with $\leftrightarrow$as long as we have (i.e. can define) $\wedge$and $\rightarrow$.

Observation 2.6 ($\leftrightarrow$can be defined with $\rightarrow$and $\wedge$)**:** The sentence $\phi \leftrightarrow \psi$can be expressed without $\leftrightarrow$as $(\phi \rightarrow \psi) \wedge (\psi \rightarrow \phi)$. Note that we can further reduce this, eliminating even $\wedge$and $\rightarrow$as before.

ϕ	ψ	$\phi \leftrightarrow \psi$	$\phi \rightarrow \psi$	$\wedge$	$\psi \rightarrow \phi$
T	T	**T**	T	**T**	T
T	F	**F**	F	**F**	T
F	T	**F**	T	**F**	F
F	F	**T**	T	**T**	T

Taken together, these results suggest that we could do without the connectives $\wedge$, $\rightarrow$, and $\leftrightarrow$, building our system entirely from $\vee$and $\neg$.

Before seeing just how far we can strip our system down, let's take a look at the first of our questions: is this all of the truth-functional connectives that could be defined? It seems the answer is: sort of.

We can explore a few well-known candidates to get a sense of what I mean here. First, consider the alternative style of disjunction common in ordinary English, *exclusive disjunction* (XOR), which means "either A or B, but not both". Let's suppose that XOR is symbolized in TFL as a connective $\bullet$. It's easy to see that such a connective is definable using $\wedge$, $\vee$and $\neg$.

ϕ	ψ	$\phi \bullet \psi$	$(\phi \vee \psi)$	$\wedge$	$\neg$	$(\phi \wedge \psi)$
T	T	**F**	T	**F**	F	T
T	F	**T**	T	**T**	T	F
F	T	**T**	T	**T**	T	F
F	F	**T**	F	**T**	T	F

Definition 2.27 (Expressively Adequate)**:** A collection of truth-functional connectives is *expresively adequate* iff they can be used collectively to define

Observation 2.7 (TFL is Expressively Adequate)**:** We can define any new possible n-place

truth-functional connective † given only the connectives already available in TFL: $\wedge$, $\vee$and $\neg$.[a]

[a]See Smith, 2020, pp. 115-117 for a more complete justification of this observation.

So we don't need any more connectives. But how few can we get away with? We already know that just $\neg$and $\vee$is expressively adequate for TFL. As is $\neg$and $\wedge$. So two? Can we get away with one? The answer is yes, but we need to make a new one. Actually, there are two options.

Definition 2.28 (Charateristic Truth Tables: Peirce's Arrow and the Sheffer stroke)**:** The semantics of the connectives $\downarrow$ and $\uparrow$ are captured by the following characteristic truth tables:

ϕ	ψ	$\phi \downarrow \psi$
T	T	F
T	F	F
F	T	F
F	F	T

ϕ	ψ	$\phi \uparrow \psi$
T	T	F
T	F	T
F	T	T
F	F	T

Truth tables for the following equivalences are left as an exercise.

	TFL	Peirce's Arrow ($\downarrow$)	Sheffer Stroke ($\uparrow$)
Negation	$\neg\phi$	$(P \downarrow P)$	$(P \uparrow P)$
Disjunction	$\phi \vee \psi$	$(P \downarrow Q) \downarrow (P \downarrow Q)$	$(P \uparrow P) \uparrow (Q \uparrow Q)$
Conjunction	$\phi \wedge \psi$	$(P \downarrow P) \downarrow (Q \downarrow Q)$	$(P \uparrow Q) \uparrow (P \uparrow Q)$

3 Truth Table Strategies

3.1 Simplification Methods

The truth table method is tedious to say the least. Even when the number of atomic sentences makes the overall number of rows manageable, a logician would need to write dozens of T and F symbols to completely fill in a table. We can introduce a number of shortcuts to make the process much more ergonomic.

Observation 3.1 (Methods to Simplify Truth Tables)**:**

1. You do not need to repeat truth values for the atomic letters. They are easy enough to read right off the left column.
2. Since $\phi \vee \psi$is true just in case either ϕor $\psi$is true, we can skip one disjunct if the other is already marked T.
3. Since $\phi \wedge \psi$is false just in case either ϕor $\psi$is false, we can skip one conjunct if the other is already marked F.
4. Since $\phi \rightarrow \psi$is true in case either $\psi$is true or $\phi$is false, we can skip one component if the other is already so marked.

This truth table with four atomic sentences shows how much savings we can get from the simplified approach. Not only does it have 16 rows, but the complexity of the sentence involves quite a few columns too.

P	Q	R	S	$(((S$	$\wedge$	$Q)$	$\wedge$	$\neg$	$\neg$	$R)$	$\vee$	$\neg$	$(\neg$	$(P$	$\vee$	$\neg$	$P)$	$\wedge$	$Q))$
T	T	T	T								T	T						F	
T	T	T	F								T								
T	T	F	T								T								
T	T	F	F								T								
T	F	T	T								T	T						F	
T	F	T	F								T								
T	F	F	T								T								
T	F	F	F								T								
F	T	T	T				T				T	F						T	
F	T	T	F				F				F								
F	T	F	T				F				F								
F	T	F	F				F				F								
F	F	T	T								T	T						F	
F	F	T	F								T								
F	F	F	T								T								
F	F	F	F								T								
							4				3/5	2						1	

We can summarize the steps of the reasoning as follows.

1. The chunking of truth values that we use in constructing the table exhaustively is really paying off here. Since P and Q are the first two sentences in the table, they have long runs of rows where the truth values of both remains unchanged. So whatever the value of $\neg(P \vee P) \wedge Q$ is it will be the same for each four-row block and when we figure it out we can just write it down every four lines. In another lucky break, we know that $P \vee P$ is always just whatever P is. Taking this all together $\neg(P \vee P) \wedge Q$ is true just in case P is false and Q is true: so only for the four rows in the third block.

2. The same reasoning about the chunking means that we can just reverse these values in the columns for $\neg(\neg(P \vee P) \wedge Q)$: making all the F values T, and vice-versa.

3. Since the main connective is $\vee$, we know that only one of the disjuncts needs to be true. The right disjunct is true on 12 rows, so we can just fill those all in for the main connective now. We don't need to know the value of the left disjunct in these cases.

4. We need to compute the truth value of the left disjunct for the remaining 4 rows. Noting the double negation $\neg\neg R$, we can see that the entire expression $(S \wedge Q) \wedge \neg\neg R$ is true just in case all of S, Q and R are. This is just one of the remaining rows. The rest are false.

5. Now we have all we need to fill in the last 4 rows for the main connective. These four truth values are underlined in the truth table.

3.2 Truth-Theoretic Concepts

Now that we have a handle on the truth-functional semantics of TFL, we are better positioned to put meat on the bones of some informal concepts introduced a few weeks ago: necessary truth/falsehood, contingent truth/falsehood, equivalence and joint possibility.

We can use truth tables to define a technical notions that approximate the concepts of *necessary truth* and *necessary falsity*. First, we can think of these concepts in terms of the forms of arguments. Let's start by firming up our definition of *argument* from week 1.

Definition 3.1 (Argument)**:** An *argument* is a sequence of sentences $\phi_1 \dots \phi_n \therefore \psi$. It comprises some *premises*, $(\phi_1 \dots \phi_n)$ and a *conclusion* (ψ), separated by an inference marker $(\therefore)$.

Beware that not all arguments are *valid*. We can make bad arguments and checking to see if an argument is good is a big part of the point of our formal apparatus. We just want to define the kinds of things that we will check for validity. Indeed, eventually we'll just drop the reference to arguments from our definition altogether.

Definition 3.2 (Tautologically Valid)**:** A TFL argument $\phi_1, \dots, \phi_n \therefore \psi$ is *tautologically valid* (i.e. *logically valid*) iff there is no valuation where all of $\phi_1, \dots, \phi_n$ are true but ψ is false.

We can characterize the relationship between the premises and conclusion of a tautologically valid argument as one of *entailment*.

Definition 3.3 (Tautologically Entail)**:** A collection of TFL sentences $\phi_1, \ldots, \phi_n$ *tautologically entails* (i.e. *logically entails*) a TFL sentence ψiff $\phi_1, \ldots, \phi_n \therefore \psi$ is tautologically valid. When this is the case, we will use the symbol $\vDash$instead of $\therefore$.

$$\phi_1, \ldots, \phi_n \vDash \psi$$

Otherwise, we will say that $\phi_1, \ldots, \phi_n$ *does not entail* $\psi$and use the symbol $\nvDash$.

$$\phi_1, \ldots, \phi_n \nvDash \psi$$

We can recast the notion of tautologocal validity to apply not just to arguments, but to individual sentences as well. Naturally, the definition is extremely similar.

Definition 3.4 (Tautology)**:** $\phi$is a *tautology* in TFL iff it is true on every valuation. When a $\phi$is a tautology, we will signify this by using $\vDash$with no premises.

$$\vDash \phi$$

Otherwise $\phi$is not a tautology, which we can signify in the metalogic with $\nvDash$.

$$\nvDash \phi$$

Some well-known examples of a tautologies have come up already:

LEM: A$\vee\neg$A
LNC: $\neg$(A$\wedge\neg$A)
DN: $\neg\neg$A$\leftrightarrow$A

We can confirm that each is invariably true using simplified truth tables

A	A	$\vee$	$\neg$A
T		**T**	F
F		**T**	T

A	$\neg$	$(A$	$\wedge$	$\neg A)$
T	**T**		F	F
F	**T**		F	T

A	$\neg$	$\neg$A	$\leftrightarrow$	A
T	T	F	**T**	
F	F	T	**T**	

Table 3: Simplified truth tables demonstrating **LEM**, **LNC**, and **DN**

Conversely, a sentence that is false on every valuation is a *contradiction*.

Definition 3.5 (Contradiction)**:** The TFL sentence $\phi$is a *contradiction* iff it is false on every valuation.

A	$A \wedge \neg A$	$A \vee \neg A$
T	**F**	**T**
F	**F**	**T**

Table 4: A simplified truth table with a contradition and a tautology

For example, $A \wedge \neg A$ is a contradiction whereas $A \vee \neg A$ is a tautology.

We will not add a symbol like $\models$that we can use for contradictions, but we can leverage the fact that tautologies and contradictions are complements to symbolize it in the metalogic anyway. If $\phi$is a contradiction then $\models\neg\phi$and if $\models\phi$then $\neg\phi$is a contradiction.

A sentence that is neither a tautology nor a contradiction is called *contingent*.

Definition 3.6 (Contingent Sentence)**:** A TFL sentence $\phi$is a *contingent* iff neither $\models\phi$nor $\models\neg\phi$.

Similarly, we can test whether two sentences are equivalent by checking to see if they have the same truth tables.

Definition 3.7 (Equivalence)**:** Two TFL sentences $\phi$and $\psi$are *equivalent* in TFL iff $\phi$and ψhave the same truth value as each other in every valuation.

So we can easily verify that $\neg(A \wedge B)$ is equivalent to $\neg A \vee \neg B$ since they have the same truth value in every row.

A	B	$\neg$	$(A$	$\wedge$	$B)$	$\neg$	A	$\vee$	$\neg$	B
T	T	**F**	T	T	T	F	T	**F**	F	T
T	F	**T**	T	F	F	F	T	**T**	T	F
F	T	**T**	F	F	T	T	F	**T**	F	T
F	F	**T**	F	F	F	T	F	**T**	T	F

Table 5: A complete truth table demonstrating the equivalence of $\neg(A \wedge B)$ and $\neg A \vee \neg B$.

Similarly, we can confirm that $\neg(A \vee B)$ is equivalent to $\neg A \wedge \neg B$.

A	B	$\neg$	$(A$	$\vee$	$B)$	$\neg A$	$\wedge$	$\neg B$
T	T	**F**		T		F	**F**	F
T	F	**F**		T		F	**F**	T
F	T	**F**		T		T	**F**	F
F	F	**T**		F		T	**T**	T

Table 6: A simplified truth table demonstrating the equivalence of $\neg(A \vee B)$ and $\neg A \neg B$.

Being able to show equivalence is a powerful capability. We've seen how we can make use of it in the metalogic by abstracting away to *forms* in our rule schemas. We can also lean on the same inter-substitutability as we try to assess the tautological status of various complex sentences.

Observation 3.2 (Substitution instances and tautologies)**:** If $\vDash\phi$and $\psi$is a substitution-instance of ϕ, then $\vDash\psi$. But the same is not true of $\nvDash\phi$.

We can just as easily use truth table methods to evaluate the logical properties of sets of sentences, including arguments.

Definition 3.8 (Joint Satisfiability)**:** The sentences $\phi_1 \dots \phi_n$ are *jointly satisfiable* in TFL iff some valuation makes them all true. Otherwise, they are *jointly unsatisfiable*.

We can clean up our definition of *entailment* now, too, dropping the references to arguments and the inference marker ($\therefore$), which really are not important to the concept as now understood.

Definition 3.9 (Entailment)**:** The TFL sentences $\phi_1 \dots \phi_n$ *entail* the TFL sentence $\psi(\phi_1 \dots \phi_n \vDash \psi)$ iff no valuation makes $\phi_1 \dots \phi_n$ all true but ψfalse.

Now that we've dropped the references to arguments, we can look for symmetry in entailment: *equivalence*.

Definition 3.10 (Mutual Entailment)**:** The TFL sentences $\phi$and ψ*mutually entail* each other ($\phi\approx\psi$) iff both $\phi\vDash\psi$and $\psi\vDash\phi$.
When $\phi\approx\psi$, we say that $\phi$and $\psi$are *tautologically equivalent*.

And as we've mentioned already, equivalence plays a fundamental role in formal logic.

We've postponed a lot of discussion about the material conditional ($\rightarrow$), the limits of its interpretation, and its relationship to similar metalogical notions like entailment. This is as good a time as any to wade into that quagmire.

Let's start by (re-)defining the major components of a conditional.

Definition 3.11 (Antecedent)**:** If the material conditional $\phi\rightarrow\psi$is a sentence of TFL, then $\phi$is its *antecedent*.

Definition 3.12 (Consequent)**:** If the material conditional $\phi\rightarrow\psi$is a sentence of TFL, then $\psi$is its *consequent*.

We can also define some some basic structural relationships between material conditionals, for example:

Definition 3.13 (Converse)**:** If the material conditional $\phi \rightarrow \psi$is a sentence of TFL, then $\psi \rightarrow \phi$is its *converse*.

Definition 3.14 (Contrapositive)**:** If the material conditional $\phi \rightarrow \psi$is a sentence of TFL, then $\neg\psi \rightarrow \neg\phi$is its *contrapositive*.

Likewise, we have some patterns of valid inference steps that amount to hard-and-fast rules that we can apply without exception.

Observation 3.3 (Modus Ponens (MP))**:** If $\phi$and $\psi$are TFL sentences, then $\psi, \phi \rightarrow \psi \vDash \psi$ and this can be used as an inference step. This mode of inference is called *modus ponens*.

A	B	A→B	A	B
T	T	T	T	T
T	F	F	T	F
F	T	T	F	T
F	F	T	F	T

Table 7: A truth table showing Modus Ponens is valid

Observation 3.4 (Modus Tollens (MT))**:** If $\phi$and $\psi$are TFL sentences, then $\neg\psi, \phi \rightarrow \psi \vDash \neg\phi$ and this can be used as an inference step in TFL. This mode of inference is called *modus tollens*.

A	B	A→B	¬B	¬A
T	T	T	F	F
T	F	F	T	F
F	T	T	F	T
F	F	T	T	T

Table 8: A truth table showing Modus Tollens is valid

Observation 3.5 (Falsehood Condition (FC))**:** If $\psi$and $\psi$are TFL sentences where $\phi$is true and $\psi$is false, then $\phi \rightarrow \psi$ must be false.

Observation 3.6 (Non-Reversability (NR))**:** If $\phi$and $\psi$are TFL sentences, then we cannot infer $\phi \rightarrow \psi$from $\phi \rightarrow \psi$. This is known as the *non-reversability* of the material conditional.

We've been mixing logic and metalogic in this lecture so haphazardly that it's worth pausing here to make an important distinctions.

A	B	A→B
T	F	F

Table 9: A truth table showing the Falsehood Condition is valid

A	B	A→B	B→A
T	T	T	T
T	F	F	T
F	T	**T**	**F**
F	F	T	T

Table 10: A truth table showing Non-Reversability is valid

Observation 3.7 (Entailment and material implication are truth-functionally equivalent)**:** $\phi \rightarrow \psi$iff $\phi \models \psi$

A	B	A→B	A⊨B
T	T	T	T
T	F	F	F
F	T	T	T
F	F	T	T

Table 11: A truth table showing Non-Reversability is valid

Nevertheless, →is a connective of truth-functional logic and ⊨is a symbol of the metalogic that we've built by augmenting English. We should refrain from using one when we mean the other.

3.3 Worries about the Material Conditional

Now we are equipped with the vocabulary necessary to discuss the controversy about material conditionals.

One way to think of these inference rules if pre-theoretically. In other words, the reason that the entailments for **MP**, **MT**, **FC** and **NR** come out the way they do is that we designed the truth table for the material conditional (→) so that they would. In this light, we can view them as *assumptions* about how implication should work, given whatever definition we decide to impose on the material conditional. They are working as intended.

But one might still wonder if these assumptions are correct. Should we abandon one or more of these principles in order to support a more "natural" type of implication?

Most of the heartburn about the material conditional comes from the fact that it is taken to be true whenever its antecedant is false. This has the effect of making many conditionals *vacuously* true: true **only** by virtue of that fact.

> If Lions won the last Super Bowl, then pigs can fly.

We know the truth of both antecedant (they did not) and consequent (they can not), so we can calculate the truth of the conditional (it's true). But this does not "feel" right. One might even argue that there is no situation in which the consequent is true. Nevertheless ...

Let's try to firm up our discomfort with some help from Smith, 2020, pp. 166-167. Recall that our definition of material implication A $\rightarrow$B is equivalent to $\neg$A$\vee$B. So when we allow the inference in (D) we are also allowing the inferences in (M).

D	**M1**	**M2**
$\neg$A	$\neg$A	not A
$\therefore$ $\neg$A$\vee$B	$\therefore$ A$\rightarrow$B	$\therefore$ if A then B

But while (D) seems trivially correct, (M1) is not immediately obvious. And (M2) comes across as just plain wrong.

But the Lions example is not so bad. The example is at least *indicative*, meaning that its truth or falsity is fully settled and not hypothetical or contingent on future events or the attitudes of human judges. We can live with the idea that the conditional sentence is true, if only because its implication will never come to pass. It is indeed *vacuous*.

Suffice it to say, the material conditional just does not imply any sort of *causal connection* between its antecedent and its consequent. That's just not part of how truth-functional logic works. Still, the material conditional is a pretty good appoximation for what is actually said in an indicative *if...then* sentence of ordinary English, even if it leaves out most of the pragmatic nuance. Its shortfalls are problably no worse than those of $\wedge$and $\vee$: ordinary English just behaves differently than TFL and that's OK.

Some evidence from ordinary English supports this acquiescence.

Observation 3.8 (English conditionals are sometimes like A $\rightarrow$B)**:**

1. English tautologies are typically equivalent to their contrapositives.
   a. *If I won the Powerball, then I am rich.*
   b. *If I am not rich, then I did not win the Powerball.*
2. The material conditional A$\rightarrow$B is just like *if A then B* in some simple cases, and even correctly invalidates two common logical fallacies:
   a. affirming the consequent: A$\rightarrow$B,B $\therefore$A; and
   b. denying the antecedent: A$\rightarrow$B,$\neg$B $\therefore\neg$B.
3. Often *if A then B* and *A only if B* are equivalent, such as these examples (from Smith,

2020, p. 155):

a. *Einstein's theory is right only if space-time is curved.*

b. *If space-time isn't curved, then Einstein's theory isn't right.*

c. *If Einstein's theory is right, then space-time is curved.*

To see how indicative conditionals still have problems around the edges, it helps to think about how we negate them. Consider the following symbolization key.

E: God exists.

P: God answers malevolent prayers.

Putting these sentences together into a conditional ($E \to P$) reads something like (G).

G: If God exists, then God answers malevolent prayers.

We can think of two ways of negating this sentence, but both are unsatisfying.

$\neg(E \to P)$	It is not the case that if God exists God answers malevolent prayers.
$E \to \neg P$	If God exists, then God does not answer malevolent prayers.

In the first example ($\neg(E \to P)$), we would have to accept the truth of E (i.e. God exists), otherwise G could not be false and so $\neg G$ could not be true. In the second example ($E \to \neg P$), the atheist is faced with a different conundrum: when E is false (i.e. when God does not exist), then we can conclude **either** P or $\neg P$, since $E \to \neg P$ and $E \to P$ would then **both** be true.

Other the other end of the spectrum, we have *counterfactuals* (i.e. *subjunctive* conditionals). David Lewis, perhaps the best known philosopher of counterfactuals, suggests the following pair of examples to illustrate the difference.[9]

I: If Oswald didn't shoot Kennedy, then someone else did.

S: If Oswald hadn't shot Kennedy, then someone else would have.

(I) basically just says that someone shot Kennedy. Of course, this seems more or less right. In the case of (S), however, the sentence asks us to contemplate a state of affairs that does not actually exist: the one where Oswald did not shoot Kennedy. In **that** state of affairs, Kennedy may or may not have been shot at all, which makes (S) less than a slam-dunk claim. This seems like something we could debate, not something we should just take to be *vacuously true*, even if Oswald **did** shoot Kennedy.

Although the subjunctive/indicative division lines up well with the English grammar conventions in the Oswald example, the alignment is not always consistent (and mode is not always expressed

[9] from Smith, 2020, p 162)

using verbs). A better approach is to think of the states-of-affairs as *possible worlds* where we can check what the truth-value of our counter-facutal sentence **would be** if if it was evaluated in a world where the antecedant was true. This approach, called *intuitionistic logic*, is a type of *modal logic* and dissatisfaction with material implication was a major motive for its formulation. But we will leave a survey of modal logic for Logic 2 and a complete treatment for Modal Logic. For now we just note that possible worlds semantics is not truth functional.

Observation 3.9: The conditional expressed with a possible world semantics ($\phi \Rightarrow \psi$)is not truth-functional.

To see this, we merely need to note that $\phi \Rightarrow \psi$ is true just in case $\psi$is true at the possible world where the sentence is evalauated. Nothing about whether or not $\phi$and $\psi$are *actually true* can help us to figure this out.

Nevertheless, like democracy, the material conditional is the worst option available, except for all the others. We just need to be careful to avoid using it for the treatment of counterfactuals.

As we've already seen, the symbolization of English sentences into TFL leaves much of the linguistic nuance out of the resulting TFL sentence. We can summarize some of the important shortcomings of TFL symbolization as follows:

Observation 3.10 (Shortcomings of TFL Symbolization)**:**

1. TFL does not support the types of inferences depicted in Aristotles syllogisms (quantification).
2. TFL insists on **LEM** (every TFL is either true or false), but some adjectives do not quite work like that.
3. TFL comes with a number of "paradoxes" that make the material conditional unsatisfying for symbolizing English sentences: one the one hand, negating the whole conditional often gives the wrong conclusions about its antecedent sentence; on the other hand, negating the consequent sentence opens up the possibility of contradiction when the antecedant sentence is false.

With the simiplified methods in 3.1, we can much more efficiently evaluate the validity of TFL arguments. Like the validity of individual sentences, testing the validity of arguments depends on identifying the load-bearing sentences in the structure of the argument. Tersely, we look for *bad lines*.

Observation 3.11 (Bad Lines in a Truth Table for Entailment)**:**

1. When the conclusion is true, the line is not bad.
2. When any of the premises is false, the line is not bad.
3. Truth tables with no bad lines depict valid arguments.

Last week, we saw how we can save time by only filling in the parts of our truth tables that matter

to the evaluation of the overall formula (i.e. to its *main operator*). But these are still *complete* truth tables, it's just that many of the cells are silent because they do not matter or are obvious.

We can simplify our approach even more if we take a step back and consider what we are trying to do in building truth tables in the first place. They are not interesting in their own right. But they do give us a clear method for being thorough in our exploration of the various logical possibilities, gven the rules of the system.

One more step back. Why do we want to explore all of the logical possibilities anyway? We want to be thorough in this way because we have defined a number of our logical notions (e.g. *tautology*, *validity*, *entailment*) in terms of these possibilities. When we explore these properties, we want to show that the sentences being evaluated are true (or false) in all of the possibilities. In our parlance from last week, we are looking for *bad lines* in the table.

But what about showing that a sentence *does not* have these properties? Surely it will suffice to just find one bad line, a *counter-example* so to speak. And indeed it will. Thus, we will refer to a such a bad line as a *contervaluation* We can describe the landscape with this table modified from Magnus et al., 2025[Chapter 15].

	yes	**no**	should be (always)
tautology	all	one	true
contradiction	all	one	false
equivalent	all	one	same
unsatisfiable	all	one	false
valid	all	one	true conclusion or false premise
entailment	all	one	true conclusion or false premise

Recall from 3.4 that a TFL sentence is a tautology just in case it is true in all scenarios. Showing that a TFL sentence is not a tautology amounts to finding a scenario where it is not true. Since we know we are trying to find a bad (i.e. false) line, we work backward through the columns with this assumption in mind.

A	B	(A	→	B)	→	(B	→	A)
F	T		T		**F**	T	F	F
4	4		2		1	3	2	3

The TFL sentence $(A \to B) \to (B \to A)$ is false when A is false and B is true. It is not a tautology.

Similarly, we can show that a TFL sentence is not a contradiction by showing that it is sometimes true. So this time we start by assuming a true line.

A	B	(A	→	B)	→	(B	→	A)
T	F	T	F	F	**T**			
4	4	3	2	3	1			

This time we had to decide what to do next, since there are two ways to make the main operator true: let its antecedant be false or let its consequent be true. We chose the former, because that choice (a false material implication) forces our truth value assignment to the atomic letters: A must be true and B must be false. We have found a counter-example. The TFL sentence $(A \to B) \to (B \to A)$ is also not a contradiction.

We can extend this practice to truth tables with multiple TFL sentences in order to find counter-examples for equivalence, satisfiability, validity and entailment. For example, the following partial table demonstrates that $A \wedge B$ and $A \wedge \neg B$ are not equivalent.

A	B	A	$\wedge$	B	A	$\wedge$	$\neg$	B
T	T		**T**			**F**	F	
2	2		1			4	3	

When we set the values of A and B to make $A \wedge B$ true, the truth value of $A \wedge \neg B$ correspondingly becomes false. The two TFL sentences have different truth values in some scenarios. They cannot be equivalent. A similar process can be applied to the remaining logical properties.

4 Introduction to Natural Deduction

For potential tautologies and entailments with n unique atomic sentences, the size of a truth table is 2^n lines. A familar number to computer scientists, but let's pause to consider how fast this number gets very very big.

sentences	lines
1	2
2	4
3	8
4	16
5	32
6	64
7	128
8	256
9	512
10	1,024

sentences	lines
11	2,048
12	4,096
13	8,192
14	16,384
15	32,768
16	655,36
17	131,072
18	262,144
19	524,288
20	1,1048,576

It is not easy to glean insights from such a table and questions about *why* an argument is correct become extremely obscure. We'd really like to get both the result **and** a story about its justification from the same process. 1000+ line truth tables are just a bad machanism to provide that. As much work as the simplified and partial truth tables help us save, we are still left with the impression that the process should be more constrained and rule-based.

We face similar problems with simplifications like *proof trees*, which still have a 2^n search space, despite less writing and better odds of not needing to explore it all (for the simple cases).

Ideally, we'd like to be able to evaluate the semantic properties of an argument in TFL on the basis of formal practices alone. This is the basic idea behind *natural deduction*. We will use *proofs* that allow us to demonstrate the truth of some TFL sentences when we assume the truth of other TFL sentences. In a sense these proofs leverage rules reflecting the intuitions that we put to use last week in quickly working through our partial truth tables to find counter-examples.

4.1 Preliminaries

In the last lecture or two, we have sometimes used greek letters ϕ, $\psi$and χin truth tables and definitions as a stand-in for **any** TFL sentence, not just atomic sentences. But more broadly, we can think of formulas like these as *schemas*. They each represent an entire class of sentences.

We already discussed schemas informally in 1.1, but we can firm up some concepts here before we start to use them in defining the rule system of *natural deduction*. In an argument schema, a given *metalogical variable* always stands for the same thing, that's the whole point of using them. Will need to add a few new definitions.

Definition 4.1 (Substitution Instance)**:** A *substitution instance* of a metalogical schema is any formula that can be made by substituting a well-formed formula of the language for each of the metalogical variables in the schema.

Substitution instances make it possible for us to generalize the notion of *form* into *metalogical schemas*.

Definition 4.2 (Form of a Well-Formed Expression)**:** A formula of a logic has the *form* of a metalogical schema iff the formula is a substitution instance of the schema.

Turning things around, we can make schemas by generating substitution instances of the well-formed formulas of a language.

Definition 4.3 (Basic Schema)**:** A *basic schema* is a metalogical expression that is obtained from a well-formed formula of the relevant logic by replacing each of the content symbols with a consistent metalogical variable (i.e. each symbol A always gets replaced by the same metalogical variable $\phi$and no other symbol B is also replaced by ϕbut by some other variable ψ).

The ones we make from tautologies are *tautological schemas*. When we note that some wff is a substitution instance of a tautological schema, we can say that it is *valid in virtue of its form*. Similarly, when we schematize an argument, we can say that it is a *tautological argument schema* just in case its simplest substitution instance[10] is a tautologically valid argument.

4.2 Defining Proofs and Rules

We begin with a basic definition of proofs (aka *derivations*).

Definition 4.4 (Proof)**:** A *proof* in Truth Functional Logic is a sequence of sentences that:

1. Begin with zero or more *premises*
2. Introduce new sentence only when they can be made by applying *rules* to lines already in the proof
3. End with a *conclusion* (the sentence that we want to prove)

We have two kinds of rules in a Fitch-style natural deduction system:

1. **Introduction Rules:** that allow us to work toward a conclusion containing the operator being introduced; and

[10]This is just the argument that results the from consistent replacement of each metalogical variable with an atomic sentence letter.

2. **Elimination Rules:** that allow us to break down sentences that are already in the proof so that we can get at the parts.

Before we get into the rules themselves, we should pause for a minute and discuss the styles and formats of the proofs themselves. First, we will separate the premises from the rest of the proof with a horizontal line. Second, we will number each line to make it easier to reference later in the proof. Finally, we will mark every line with a *justification*. This will be the rule applied along with the previous lines that it was applied to. We will mark premises with their own justification PR, which just indicates that we take them to be true by assumption.

1	$P \wedge Q$	PR
2	P	$\wedge$E, 1

In this simple proof, we show that we can conclude P if we assume $P \wedge Q$. In other words, $P \wedge Q \dashv P$. The single premise ($P \wedge Q$) is marked (PR) and the conclusion is the sentence on the final line (P).

4.3 Rules of Natural Deduction

The simplest rule in the system is simply that we can repeat lines that we have already seen.

Rule 4.1 (Repitition)**:**

m	ϕ	
$\vdots$	$\vdots$	
n	ϕ	R, m

where $m < n$

Trivially, we could use this to prove $\phi \vDash \phi$.

0	ϕ	PR
0	ϕ	R, 0

For each of the operators, we will want to introduce a pair of rules: one an introduction rule and the other an elimination rule.

4.3.1 Rules for Conjunction

Conjunction has the most straightforward rules. When both conjuncts occur in a proof, we can introduce a conjunction of them. When a conjunction occurs in a proof, we can eliminate it for

either of the conjuncts.

Rule 4.2 (∧Introduction)**:**

m	ϕ	
n	ψ	
	$\phi \wedge \psi$	$\wedge$I, m, n

Rule 4.3 (∧Elimination)**:** There are two rules for ∧elimination, one for each conjunct:

n	$\phi \wedge \psi$		n	$\phi \wedge \psi$	
	ϕ	$\wedge$E, n		ψ	$\wedge$E, n

With just the introduction and elimination rules for ∧we can prove many valid arguments such as $A \wedge B, C \wedge D \therefore A \wedge D$.

1	$A \wedge B$	PR
2	$C \wedge D$	PR
3	A	$\wedge$E, 1
4	D	$\wedge$E, 2
5	$A \wedge D$	$\wedge$I, 3, 4

4.3.2 Rules for Conditionals

The elimination rule (modus ponens) is straightforward so we will start there.

Rule 4.4 (→Elimination)**:**

m	$\phi \rightarrow \psi$	
n	ϕ	
	ψ	$\rightarrow$E, m, n

Adding modus ponens gives us more tools to explore multi-step arguments in proofs:

1	$A \wedge B$	PR
2	$B \rightarrow C$	PR
3	B	$\wedge$E, 1
4	C	$\rightarrow$E, 2, 3

So far, pretty simple. But for the introduction rule, we will need to make use of assumptions that are not included in the premises. We will mark their introduction with a new horizontal line and a justification of AS. We will discuss assumptions in more detail after the introduction rule for conditionals.

Rule 4.5 ($\rightarrow$Introduction)**:**

i	ϕ	AS
j	ψ	
	$\phi \rightarrow \psi$	$\rightarrow$I, i–j

Here is a simple proof using both the introduction and elimination rules for conditionals.

1	$A \rightarrow (B \rightarrow C)$	PR
2	B	AS
3	A	AS
4	$B \rightarrow C$	$\rightarrow$E, 1, 3
5	C	$\rightarrow$E, j, 4
6	$A \rightarrow C$	$\rightarrow$I, 3–5
7	$A \rightarrow C$	$\rightarrow$I, j–6

4.3.3 Subproofs for Conditionals

The resulting sub-sequence in the last proof, visibly set off from the rest of the proof using indentation, is called a *subproof*. When we use the $\rightarrow$introduction rule we collapse the subproof and treat its assumption as the antecedant of the new conditional. At this point, the subproof is *closed*.

Consider the following proof of $A \rightarrow B, B \rightarrow C \therefore A \rightarrow C$.

1	$A \to B$	PR
2	$B \to C$	PR
3	$\quad A$	AS
4	$\quad B$	$\to$E, 1, 3
5	$\quad C$	$\to$E, 2, 4
6	$A \to C$	$\to$I, 3–5

When the subproof is closed, the assumption that A is true is set aside (*discharged*). We only needed it to establish that $A \to C$, which is true regardless of whether A is. We should pause here and take a closer look at this process.

Perhaps the most important point about subproofs is that their components are meant to be used only by the subproofs themselves, not by the larger proof(s) containing them. An open subproof must be closed; we cannot reach inside closed subproofs.

Definition 4.5 (Permissible Line Citations)**:** When we want to justify a rule in a formal proof, we can only cite lines:

1. that occur before the current line
2. that do not occur in a closed subproof

The justification on line (5) below is not allowed (*), because we are trying to use a line in a closed subproof, line (3), to support it.

1	A	PR
2	$\quad B$	AS
3	$\quad B$	R, 2
4	$B \to B$	$\to$I, 2–3
5*	B	$\to$E, 4, 3

Similarly, we want to be careful when we apply rules like 4.5. We can only cite closed subproofs and they cannot be nested too deeply.

Definition 4.6 (Permissible Subproof Citations)**:** When we want to justify a rule in a formal proof, we can only cite subproofs:

1. that occur entirely before the current line
2. that do not occur inside another closed subproof

3. whose last line does not occur inside a nested subproof[a]

[a]The third condition is really just a special case of the second.

For example, in the following proof, we cannot apply the introduction rule at line (7) because the subproof that we want to use to justify it, on lines (3)-(4), itself occurs within a closed subproof. The intuition behind this is that the subproof on lines (3)-(4) depends on the assumption of B on line (2).

1	A	PR
2	$\quad B$	AS
3	$\quad\quad C$	AS
4	$\quad\quad A \wedge B$	$\wedge$I, 1, 2
5	$\quad C \rightarrow (A \wedge B)$	$\rightarrow$I, 3–4
6	$B \rightarrow (C \rightarrow (A \wedge B))$	$\rightarrow$I, 2–5
7*	$C \rightarrow (A \wedge B)$	$\rightarrow$I, 3–4

4.3.4 Rules for Biconditionals

With this basic understanding of subproofs and definitions for $\rightarrow$introduction and $\rightarrow$elimination, the rules for biconditionals are trivial.

Rule 4.6 ($\leftrightarrow$Elimination)**:** There are two rules for $\leftrightarrow$elimination, one for each sentence:

m	$\phi \leftrightarrow \psi$		m	$\phi \leftrightarrow \psi$	
n	ϕ		n	ψ	
	ψ	$\leftrightarrow$E, m, n		ϕ	$\leftrightarrow$E, m, n

Rule 4.7 ($\leftrightarrow$Introduction)**:**

i	ϕ	AS
j	ψ	
k	ψ	AS
l	ϕ	
	$\phi \leftrightarrow \psi$	$\leftrightarrow$I, i–j, k–l

Here we see a proof that uses both the introduction and elination rules for $\leftrightarrow$to show its transitivity.

1	$A \leftrightarrow B$	PR
2	$B \leftrightarrow C$	PR
3	A	AS
4	C	$\leftrightarrow$E, 2
5	C	AS
6	A	$\leftrightarrow$E, 1
7	$A \leftrightarrow C$	$\leftrightarrow$I, 3–4. 5–6

4.3.5 Rules for Disjunction

The last few operators are more challenging to define. The first of these, disjunction ($\vee$), because its elimination requires us to check more than one possibility.

Introduction, however, is quite straightforward.

Rule 4.8 ($\vee$Introduction)**:** There are two rules for $\vee$introduction, one for each sentence:

n	ϕ		n	ψ	
	$\phi \vee \psi$	$\vee$I, n		$\phi \vee \psi$	$\vee$I, n

We can handle the ambiguity involved in disjunction elimination with multiple subproofs, much as we do when intoducing biconditionals.

Rule 4.9 (∨Elimination)**:**

j	$\phi \vee \psi$	
i	$\quad \phi$	AS
j	$\quad \chi$	
k	$\quad \psi$	AS
l	$\quad \chi$	
	χ	∨E, j, i–j, k–l

Now that we have almost a complete set of rules for the basic operators of TFL, we can prove a variety of valid arguments, such as this one demonstrating the distributivity of ∧and ∨.

1	$A \wedge (B \vee C)$	PR
2	A	∧E, 1
3	$B \vee C$	∧E, 1
4	$\quad B$	AS
5	$\quad A \wedge B$	∧I, 2, 4
6	$\quad (A \wedge B) \vee (A \wedge C)$	∨I, 5
7	$\quad C$	AS
8	$\quad A \wedge C$	∧I, 2, 7
9	$\quad (A \wedge B) \vee (A \wedge C)$	∨I, 8
10	$(A \wedge B) \vee (A \wedge C)$	∨E, 3, 4–6, 7–9

4.3.6 Rules for Negation

An attentive reader will probably have noticed that we have not yet mentioned the negation operator (¬). The nature of negation in TFL is wrapped up in two of its core assumptions: *non-contradiction* and *the excluded middle*. We've seen these before:

LEM: $\phi \vee \neg\phi$
LNC: $\neg(\phi \wedge \neg\phi)$
DN: $\neg\neg\phi \leftrightarrow \phi$

The first (**LEM**) says that one of either ϕor $\neg\phi$must be true. The second (**LNC**) says that $\phi$and $\neg\phi$cannot both be true. To accomomdate these assumptions, especially the second, we introduce an additional symbol into our object language for TFL ($\bot$) which we will use when we encounter a contradiction. We know that the negation of **LNC** (a tautology) is a contradiction. In formulating rules for negation, instead of leveraging **LNC**, we can instead leverage its negation, as in **LNC2**.

LNC2: $\phi \wedge \neg\phi \vDash \bot$

There are several ways to think about $\bot$(e.g. as a zero-place connective or as a logical constant), but we will assume that it is just a sentence that is always false, regardless of situation. This makes it easy to use in other rules and does not obscure its use to indicate a contradiction. Given **LNC2**, the elimination rule is simple.

Rule 4.10 (¬Elimination)**:**

m	$\neg\phi$	
n	ϕ	
	$\bot$	¬E, m, n

We will need to wait for some rules that tell use what to do with $\bot$before we can demonstrate ¬elimination in proofs.

Recall reductio ad adsurdum (RAA) from the first lecture (**def:raa**), repeated here:

Definition 4.7 (Reductio Ad Absurdum)**:** If $\phi_1 \ldots \phi_n$ are taken as premises and the addition of ψwould result in a contradiction, then $\phi_1 \ldots \phi_n$ entail not-ψ.

So how do we introduce a negation? We assume the non-negated sentence and use a subproof to show that this assumption leads to a contradiction. The introduction rule for negation references subproofs of this form in its justification.

Rule 4.11 (¬Introduction)**:**

m	ϕ	AS
n	$\bot$	
	$\neg\phi$	¬I, m–n

Now we can demonstrate both negation rules with a proof.

1	$\neg(A \vee B)$	PR
2	$\quad (A \vee B) \wedge (C \wedge \neg D)$	AS
3	$\quad A \vee B$	$\wedge$E, 2
4	$\quad \bot$	$\neg$E, 1, 3
5	$\neg((A \vee B) \wedge (C \wedge \neg D))$	$\neg$I, 2–4

4.3.7 Indirect Proof and Explosion

Of course, the symmetry of negation lets us work in the opposite direction, too. We will call this method: *indirect proof*, since it does not involve the introduction or elimation of any operators outside of the subproof.

Rule 4.12 (Indirect Proof)**:**

m	$\quad \neg\phi$	AS
n	$\quad \bot$	
	ϕ	IP, m–n

Indirect proof is really just a reversal of the $\neg$I rule. IP is convenient for proving double-negative elimination (**DNE**): $\neg\neg$A $\dashv$A

1	$\neg\neg A$	PR
2	$\quad \neg A$	AS
3	$\quad \bot$	$\neg$E, 1, 2
4	A	IP, 2–3

One additional rule can be thought of as $\bot$elimination. A long-standing assumption in classical logic is that proving a contradiction allows us to conclude anything at all (*ex falso quodlibet*). This follows trivially from the fact that we cannot find a bad line in entailments with a pair of contradictions in the premises because there is no situation in which they are all true.

X: $\bot \vDash \phi$for any TFL sentence ϕ

We will refer to this kind of inference step as *explosion*. Like vacuous material implication, not everyone loves this aspect of logic (especially at first). Nevertheless it is effective and widely adopted.

> [O]fficial definitions of entailment–as with many other definitions of logical concepts–are best thought of as rational reconstructions of our messy-pre-theoretic ideas. We try to fit the range of informal desiderata as best we can, balancing considerations of theoretical simplicity and ease of use against the cost of departures from our initial intuitions. (from Smith, 2020, p. 145)

Rule 4.13 (Explosion)**:**

m	$\bot$	
	ϕ	X, m

We can prove the *disjunctive syllogism* rule (A $\vee$B, $\neg$A $\dashv$B) using explosion:

1	$A \vee B$	PR
2	$\not A$	PR
3	$\quad A$	AS
4	$\quad \bot$	$\neg$E, 2, 3
5	$\quad B$	X, 4
6	$\quad B$	AS
7	$\quad B$	R, 6
8	B	$\vee$E, 1, 3–5, 6–7

5 Constructing Proofs

Although the rules are pretty simple, building proofs can get quite complicated. Much like chess, the rules are easy to learn, but yon can get into trouble quickly if you don't adopt some basic straegies to guide your moves.

Observation 5.1 (Proof Strategies for TFL)**:** We can tersely describe the basic strategy for TFL proofs as follows.

1. Work backwards from the conclusion.
    a. Conjunction: what components needed to use the $\wedge$I rule as the last step? i.e. what are the conjuncts?
    b. Conditional: what are the assumption and conclusion of the subproof we need to take the last step? i.e. what are the antecedant and consequent of the conditional?
    c. Negation: what subproof is needed to show a contradiction?
    d. Disjunction: which of the disjuncts looks easiest to prove?
2. Work forwards from the current step.
    a. Conjunction: unpack the conjuncts
    b. Disjunction: what sentence do you need to prove from both disjuncts?
    c. Conditional: how can you prove the antecedant?
    d. Negation: how can you prove a contradiction?
3. Leverage $\bot$
    a. Work toward a contradiction to conclude anything, especially in a subproof
    b. Indirect proof: build a contradictory subproof that assumes the negation

Ideally, we will apply these stragegies in this order, wherever the rules are appropriate to the current state of the proof. The *work backwards* rules in particular are intended as scaffolding to indicate which portions of the overall proof need to be filled in. The rules themselves reflect this partially-completed aspect.

5.1 Working Backward

Suffice it to say, taking account of what we are trying to prove is an important component of successfully constructing proofs. This is the spirit in which the rules for working backward are intended. They offer a road-map for proving the conclusion.

Proving conjunctions amounts to proving each of the conjuncts.

Rule 5.1 (Proving Conjunctions)**:**

	$\vdots$	
m	ϕ	
	$\vdots$	
n	ψ	
$n+1$	$\phi \wedge \psi$	$\wedge$I, m, n

Proving conditionals and negations involves setting up subproofs of the appropriate form.

Rule 5.2 (Proving Conditionals)**:**

m	ϕ	AS
	$\vdots$	
n	ψ	
$n+1$	$\phi \rightarrow \psi$	$\rightarrow$I, m–n

Rule 5.3 (Proving Negations)**:**

m	ϕ	AS
	$\vdots$	
n	$\bot$	
$n+1$	$\neg\phi$	$\neg$I, m–n

Working backward from disjunctions is possible too, although the ambiguity may lead to some backtracking. So this is a last resort.

Rule 5.4 (Proving Disjunctions)**:** There are two possible justifications for a disjunction, i.e. each of its disjuncts:

	$\vdots$	
n	ϕ	
$n+1$	$\phi \vee \psi$	$\vee$I, n

	$\vdots$	
n	ψ	
$n+1$	$\phi \vee \psi$	$\vee$I, n

5.2 Working Forward

Working forward can be guided by the same schematic rules. For conjunction we just unpack both of the conjuncts.

Rule 5.5 (Unpacking Conjunctions)**:**

n	$\phi \wedge \psi$	
$+1$	ϕ	
$n+2$	ψ	

For disjunction we need to think more about what we are trying to do. Suppose that $\phi$and $\psi$are the disjuncts and $\chi$is what we are trying to prove (or a step in that direction). We will need a subproof from each of the disjuncts to χ.

Rule 5.6 (Unpacking Disjunctions)**:**

k	$\phi \vee \psi$	
$k+1$	ϕ	AS
	$\vdots$	
m	χ	
$m+1$	ϕ	AS
	$\vdots$	
n	χ	
$n+1$	χ	$\vee$E, k, $k+1$–m, $m+1$–n

Finally, conditionals and negations are pretty easy to unpack. But they each require a proof (not necessarily via subproof) of a component (the antecedent and the negated sentence, respectively).

Rule 5.7 (Unpacking Conditionals)**:**

m	$\phi \rightarrow \psi$	
	$\vdots$	
n	ϕ	
$n+1$	ψ	$\rightarrow$E, m, n

Rule 5.8 (Unpacking Negations)**:**

m	$\neg\phi$	
	$\vdots$	
n	ϕ	
$n+1$	$\bot$	$\neg$E, m, n

5.3 Leveraging $\bot$

Sometimes the best way to get a subproof to its intended conclusion is to work toward a contradiction. Then you can conclude the subproof with literally anything, which is a powerfully flexible tool. Recall the proof of disjunctive syllogism from the last lecture.

1	$A \vee B$	PR
2	$\neg A$	PR
3	$\quad A$	
4	$\quad \bot$	$\neg$E, 2, 3
5	$\quad B$	X, 4
6	$\quad B$	
7	$\quad B$	R, 6
8	B	R, 1, 3–5, 6–8

Similarly, we can get any sentence we want in the main proof (or current subproof) by assuming its negation and proving a contradiction. We will call this technique *indirect proof*. Its rule looks a lot like the identically named rule of inference from the last lecture.

Rule 5.9 (Indirect Proof)**:**

m	$\quad \neg\phi$	AS
	$\quad \vdots$	
n	$\quad \bot$	
$n+1$	ϕ	IP, m–n

The method of indirect proof lets us prove all sorts of tautologies because it depends on discharging the assumptions of a subproof. Aside from proving a contradiction (explosion), this is the only way we can get "something from nothing" in natural deduction.

1	$\quad \neg(A \vee \neg A)$	AS
2	$\quad\quad A$	AS
3	$\quad\quad A \vee \neg A$	$\vee$I, 2
4	$\quad\quad \bot$	$\neg$E, 1, 3
5	$\quad \neg A$	$\neg$I, 2–4
6	$\quad A \vee \neg A$	$\vee$I, 5
7	$\quad \bot$	$\neg$E, 1, 6
8	$A \vee \neg A$	IP, 1–7

5.4 Strategies in Action

We will introduce some notation now, but postpone precise definitions until Lecture 8 when we will focus our discussion on TFL metalogic. The concepts should be more or less clear in their informal presentation.

1. When ψcan be derived from ϕ, we will say $\phi\vdash\psi$(or $\psi\dashv\phi$) which is pronounced "ϕproves ψ".
2. We will use $\vdash$instead of $\therefore$, whenever appropriate. Specifically, we will use $\phi\vdash\psi$whenever $\phi\therefore\psi$is a (proof-theoretically) valid argument.
3. When $\phi\vdash\psi$and $\phi\dashv\psi$we can say that $\phi$and $\psi$are *interderivable* and express this as $\phi\dashv\vdash\psi$.

The only way to fully understand our proof strategies is to see them used on some actual proofs. The interdreviability of the various formulas related to material implication are a good choice, since that is an important property of TFL and demonstrates most if not all of the strategies discussed earlier in the lecture.

(A $\rightarrow$B)$\dashv\vdash\neg$(A $\wedge\neg$B)
(A $\rightarrow$B)$\dashv\vdash$($\neg$A $\vee$B)

Proving the interderivability of two sentences typically amounts to proving both directions of the derivation, one at a time.

5.4.1 (A $\rightarrow$B)$\vdash\neg$(A $\wedge\neg$B)

For (A $\rightarrow$B)$\vdash\neg$(A $\wedge\neg$B), we will work backward from a negative. We just need to start with a template for a subproof of $\bot$.

1	$A \rightarrow B$	PR
i	$\quad A \wedge \neg B$	AS
	$\quad \vdots$	
j	$\quad \bot$	
n	$\neg(A \wedge \neg B)$	$\neg$I, i–j

We can start trying to get a contradiction by unpacking the conjunction in the subproof. We'll fill that in.

1	$A \to B$	PR
i	$\quad A \wedge \neg B$	AS
	$\quad A$	$\wedge$E, i
	$\quad \neg B$	$\wedge$E, i
	$\quad \vdots$	
j	$\quad \bot$	
n	$\neg(A \wedge \neg B)$	$\neg$E, i–j

We can fill in the missing lines using modus ponens and negation elimination.

1	$A \to B$	PR
2	$\quad A \wedge \neg B$	AS
3	$\quad A$	$\wedge$E, 2
4	$\quad \neg B$	$\wedge$E, 2
5	$\quad B$	
6	$\quad \bot$	$\neg$E, 4, 5
7	$\neg(A \wedge \neg B)$	$\neg$I, 2–6

5.4.2 $\neg$(A $\wedge\neg$B)$\vdash$(A $\to$B)

For the backward case ($\neg$(A $\wedge\neg$B)$\vdash$(A $\to$B)), start by setting up a proof schema for the conditional.

1	$\neg(A \wedge \neg B)$	PR
i	$\quad A$	AS
	$\quad \vdots$	
j	$\quad B$	
n	$A \to B$	$\to$I, i–j

Now we just need to fill in the dots. Proving B from A will depend on how we can use (1). Unpacking it and working forward will be a mess (a negated conjunction is just a disjunction of negations). But perhaps we can proceed by indirect proof, since (1) is a negation. We'll fill in some more with that assumption in mind, opening a new subproof for IP.

1	$\neg(A \wedge \neg B)$	PR
2	$\quad A$	AS
3	$\quad\quad \neg B$	AS
4	$\quad\quad \vdots$	
i	$\quad\quad \bot$	
j	$\quad B$	
n	$A \to B$	$\to$I, 2–j

And we're done if we just use the $\wedge$introduction rule to fill in the missing line.

1	$\neg(A \wedge \neg B)$	PR
2	$\quad A$	AS
3	$\quad\quad \neg B$	AS
4	$\quad\quad A \wedge \neg B$	$\wedge$I, 2, 3
5	$\quad\quad \bot$	$\neg$E, 1, 4
6	$\quad B$	IP, 3–5
7	$A \to B$	$\to$I, 2–6

5.4.3 (A $\to$B)$\vdash$($\neg$A $\vee$B)

Another well-known equivalence is (A $\to$B)$\approx$($\neg$A $\vee$B). Start with (A $\to$B)$\vdash$($\neg$A $\vee$B). To work backward from a conclusion would require choosing a disjunct, so might be a little open-ended. We'll get back to that if we need to. Let's try indirect proof instead.

1	$A \to B$	PR
i	$\quad \neg(\neg A \vee B)$	AS
	$\quad \vdots$	
j	$\quad \neg A \vee B$	
k	$\quad \bot$	$\neg$E, 1, j
n	$\neg A \vee B$	IP, i–k

To get a contradiction we want to prove either ¬A or B to we can say ¬A∨B. If we assume A we can use use modus ponens to unpack (1), so we'll go that route (i.e. we'll prove ¬A). First sketch out the subproof.

1	$A \to B$	PR
2	$\quad \neg(\neg A \vee B)$	AS
i	$\quad\quad A$	AS
	$\quad\quad \vdots$	
j	$\quad\quad \bot$	
k	$\quad \neg A$	¬I, i–j
l	$\quad \neg A \vee B$	∨I, k
m	$\quad \bot$	¬E, 2, l
n	$\neg A \vee B$	IP, 2–m

Then fill it in by applying →elimination to the premise using the new assumption.

1	$A \to B$	PR
2	$\quad \neg(\neg A \vee B)$	AS
3	$\quad\quad A$	AS
4	$\quad\quad B$	→E, 1, 3
5	$\quad\quad \neg A \vee B$	∨I, 4
6	$\quad\quad \bot$	¬E, 2, 5
7	$\quad \neg A$	¬I, 3–6
8	$\quad \neg A \vee B$	∨I, 7
9	$\quad \bot$	¬E, 2, 8
10	$\neg A \vee B$	IP, 2–9

5.4.4 (¬A ∨B)⊢(A →B)

Now the other direction: (¬A ∨B)⊢(A →B). So far we haven't seen explosion or ∨unpacked, but this proof has them both. First, we note that we are trying to prove A →B by adding a template for the appropriate subproof.

1	$\neg A \vee B$	PR
i	$\quad A$	AS
	$\quad \vdots$	
j	$\quad B$	
n	$(A \to B)$	$\to$I, i–j

The premise is a disjunction, so if it's gonna play a role in the proof, we're gonna need to unpack it with $\vee$elimination, so let's set up the subproofs for that now.

1	$\neg A \vee B$	PR
2	$\quad A$	AS
i	$\quad\quad \not{A}$	AS
	$\quad\quad \vdots$	
j	$\quad\quad B$	
k	$\quad\quad B$	
	$\quad\quad \vdots$	
l	$\quad\quad B$	
m	$\quad B$	$\vee$E, 1, i–j, k–l
n	$(A \to B)$	$\to$I, 2–m

Both subproofs follow immediately. The first is a contradiction, so we can apply explosion to get B. The second is just repetition; in fact we don't even need the placeholder line in the template.

1	$\neg A \vee B$	PR
2	$\quad A$	AS
3	$\quad\quad \not{A}$	AS
4	$\quad\quad \bot$	¬E, 2, 3
5	$\quad\quad B$	X, 4
6	$\quad\quad B$	AS
7	$\quad\quad B$	R, 6
8	$\quad B$	∨E, 1, 3–5
9	$(A \to B)$	→I, 2–8

6 Additional Rules

Last week I mentioned that we will add some of the valid arguments we prove to our rule system. We start by defining one that we proved last week, *disjunctive syllogism*.

Rule 6.1 (Disjunctive Syllogism)**:** There are two rules for disjunctive syllogism, one for each disjunct:

m	$\phi \vee \psi$			m	$\phi \vee \psi$		
n	$\neg\phi$			n	$\neg\psi$		
	ψ	DS, m, n			ϕ	DS, m, n	

6.1 Rules for Negation

Recall our oft-repeated tautologies of negation.

LEM: $\phi \vee \neg\phi$
LNC: $\neg(\phi \wedge \neg\phi)$
DNE: $\neg\neg\phi \leftrightarrow \phi$

We can now prove that they are all mutually inter-derivable.

LEM$\dashv\vdash$**LNC**
LEM$\dashv\vdash$**DNE**
LNC$\dashv\vdash$**DNE**

In fact, they are all *theorems* of TFL (i.e. $\vdash\psi$with no premises). So it will be enough to prove that fact, since if $\vdash\psi$then obviously also $\phi\vdash\psi$.

$\vdash$**LEM**
$\vdash$**LNC**
$\vdash$**DNE**

Note that non-contradiction is already implemented as the negation rules. We can add the other two theorems as derived rules.

Rule 6.2 (Excluded Middle)**:**

1		$\neg(\phi \vee \neg\phi)$	AS
2		ϕ	AS
3		$\phi \vee \neg\phi$	$\vee$I, 2
4		$\bot$	$\neg$E, 1, 3
5		$\neg\phi$	$\neg$I, 2–4
6		$\phi \vee \neg\phi$	$\vee$I, 5
7		$\bot$	$\neg$E, 1, 6
8	$\phi \vee \neg\phi$		IP, 1–n

Table 12: Proof of **LEM**

1		$\phi \wedge \neg\phi$	AS
2		ϕ	$\wedge$E, 1
3		$\neg\phi$	$\wedge$E, 1
4		$\bot$	$\neg$E, 2, 3
5	$\neg(\phi \wedge \neg\phi)$		$\neg$I, 1–4

Table 13: Proof of **LNC**

1		$\neg\neg\phi$	AS
2		$\neg\phi$	AS
3		$\bot$	$\neg$E, 1, 2
4		ϕ	IP, 2–3
5		ϕ	AS
6		$\neg\phi$	AS
7		$\bot$	$\neg$E, 1, 6
8		$\neg\neg\phi$	$\neg$I, 6–7
9	$\phi \leftrightarrow \neg\neg\phi$		$\leftrightarrow$I, 1–4, 5–8

Table 14: Proof of **DNE**

i	|	ϕ	AS
j	|	ψ	
k	|	$\neg\phi$	AS
l	|	ψ	
	ψ		LEM, i–j, k–l

Rule 6.3 (Double Negative Elimination)**:**

m	$\neg\neg\phi$	
	ϕ	DNE, m

We define the naturual deduction rules for negation in terms of **LNC**, but the rule system for TFL could be defined using any of these three formulas, treating the other two as derived rules.[11]

6.2 Modus Tollens

Another useful technique, called *modus tollens* lets us conclude that the antecedant of a true conditional is false when its consequent is. The rule works much like disjunctive syllogism.

Rule 6.4 (Modus Tollens)**:**

m	$\phi \to \psi$	
n	$\neg\psi$	
	$\neg\phi$	MT, m

This is another proof that leverages $\bot$.

[11] For details about a **DN**-based system, see Smith, 2020.

1	$\phi \to \psi$	PR
2	$\neg\psi$	PR
3	$\quad \phi$	AS
4	$\quad \psi$	$\to$E, 1, 3
5	$\quad \bot$	$\neg$E, 2, 4
6	$\neg\phi$	$\neg$I, 3–5

6.3 De Morgans Rules

Finally, the *De Morgan rules* tell us how to properly ditribute negation into conjunctions and disjunctions. To justify our rules, we want to prove the following results.

$\neg\wedge \quad \neg(\phi\wedge\psi)\dashv\vdash(\neg\phi\vee\neg\psi)$
$\neg\vee \quad \neg(\phi\vee\psi)\dashv\vdash(\neg\phi\wedge\neg\psi)$

The first one is by far the hardest.

The rest are easily solved by following the proof strategies developed over the last few weeks. This one should be practically trivial by now.

These two alse use the rule for *disjunctive syllogism*.

Once again, we can incorporate these results with a set of explicit rules.

Rule 6.5 (De Morgan Rules)**:** There are four rules for handling negated coordinations forming a square of duals and rule reversals.

We have a pair of rules for negated conjunctions:

m	$\neg(\phi \wedge \psi)$			m	$\neg\phi \vee \neg\psi$		
	$\neg\phi \vee \neg\psi$	DeM, m			$\neg(\phi \wedge \psi)$	DeM, m	

And a dual pair of rules for negated disjunctions:

m	$\neg(\phi \vee \psi)$			m	$\neg\phi \wedge \neg\psi$		
	$\neg\phi \wedge \neg\psi$	DeM, m			$\neg(\phi \vee \psi)$	DeM, m	

1	$\neg(\phi \wedge \psi)$	PR
2	$\quad \neg(\neg\phi \vee \neg\psi)$	AS
3	$\quad\quad \neg\phi$	AS
4	$\quad\quad \neg\phi \vee \neg\psi$	$\vee$I, 3
5	$\quad\quad \bot$	$\neg$E, 2, 4
6	$\quad \phi$	IP, 3–5
7	$\quad\quad \neg\psi$	AS
8	$\quad\quad \neg\phi \vee \neg\psi$	$\vee$I, 7
9	$\quad\quad \bot$	$\neg$E, 2, 8
10	$\quad \psi$	IP, 7–9
11	$\quad \phi \wedge \psi$	$\wedge$I, 6, 10
12	$\quad \bot$	$\neg$E, 1, 11
13	$\neg\phi \vee \neg\psi$	IP, 2–12

Table 15: Proof of $\neg(\phi \wedge \psi) \vdash (\neg\phi \vee \neg\psi)$

1	$\neg(\phi \vee \psi)$	PR
2	$\quad \phi$	AS
3	$\quad \phi \vee \psi$	$\vee$I, 2
4	$\quad \bot$	$\neg$E, 1, 3
5	$\neg\phi$	$\neg$I, 2–4
6	$\quad \psi$	AS
7	$\quad \phi \vee \psi$	$\vee$I, 6
8	$\quad \bot$	$\neg$E, 1, 7
9	$\neg\psi$	$\neg$I, 6–8
10	$\neg\phi \wedge \neg\psi$	$\wedge$I, 5, 9

Table 16: Proof of $\neg(\phi \vee \psi) \vdash (\neg\phi \wedge \neg\psi)$

1	$\neg\phi \vee \neg\psi$	PR
2	$\quad \phi \wedge \psi$	AS
3	$\quad\quad \neg\phi$	AS
4	$\quad\quad \phi$	$\wedge$E, 2
5	$\quad\quad \bot$	$\neg$E, 3, 4
6	$\quad\quad \neg\psi$	AS
7	$\quad\quad \psi$	$\wedge$E, 2
8	$\quad\quad \bot$	$\neg$E, 6, 7
9	$\quad \bot$	DS, 1, 3–5, 6–8
10	$\neg(\phi \wedge \phi)$	$\neg$I, 2–9

Table 17: Proof of $(\neg\phi \vee \neg\psi) \vdash \neg(\phi \wedge \phi)$

1	$\neg\phi \wedge \neg\psi$	PR
2	$\quad \phi \vee \psi$	AS
3	$\quad\quad \phi$	AS
4	$\quad\quad \neg\phi$	$\wedge$E, 1
5	$\quad\quad \bot$	$\neg$E, 3, 4
6	$\quad\quad \psi$	AS
7	$\quad\quad \neg\psi$	$\wedge$E, 1
8	$\quad\quad \bot$	$\neg$E, 6, 7
9	$\quad \bot$	DS, 2, 3–5, 6–8
10	$\neg(\phi \vee \psi)$	$\neg$I, 2–9

Table 18: Proof of $(\neg\phi \wedge \neg\psi) \vdash \neg(\phi \vee \psi)$

7 Proof-Theoretic Concepts and Basic TFL Metalogic

So far our proof-theoretic rule system can be summarized as follows. In the starred cases (*), the rule is taken to close a subproof which contains the premises. Similarly, $\phi \rightarrow \psi$is typically shorthand for a subproof proving $\psi$from ϕ.

Rule Group	**Introduction**	**Elimination**
$\neg$	$\neg\phi,\phi \vdash \bot$	* $\phi,\bot \vdash \neg\phi$
$\wedge$	$\phi,\psi \vdash \phi \wedge \psi$	$\phi \wedge \psi \vdash \phi$; $\phi \wedge \psi \vdash \psi$
$\vee$	$\phi \vdash \psi$	$\psi \vee \phi,\phi \rightarrow \chi,\psi \rightarrow \chi \vdash \chi$
$\rightarrow$	$\phi \ldots \psi \vdash \phi \rightarrow \psi$	
$\leftrightarrow$	$\phi \rightarrow \psi,\psi \rightarrow \phi \vdash \phi \leftrightarrow \psi$	$\phi \leftrightarrow \psi,\phi \vdash \psi$; $\phi \leftrightarrow \psi,\phi \vdash \phi$
DeM ($\neg\wedge$)	$\neg\phi \vee \neg\psi \vdash \neg(\phi \wedge \psi)$	$\neg(\phi \wedge \psi) \vdash \neg\phi \vee \neg\psi$
DeM ($\neg\vee$)	$\neg\phi \wedge \neg\psi \vdash \neg(\phi \vee \psi)$	$\neg(\phi \vee \psi) \vdash \neg\phi \wedge \neg\psi$
DN	$\phi \vdash \neg\neg\phi$	$\neg\neg\phi \vdash \phi$
DS	$\phi \vee \psi,\neg\phi \vdash \psi$	n/a
LEM	$\phi \rightarrow \psi,\neg\phi \rightarrow \psi \vdash \psi$	n/a

Table 19: Proof Rules for TFL Natural Deduction

7.1 Formalizing the Notion of *Proof*

So far I've been talking informally about valid *arguments*. Something like $\phi, \psi \therefore \phi \wedge \psi$ outlines the argument, but we want to be more precise in our discussion of various arguments. First, we will replace the informal $\therefore$marker with a more formal meta-logical operator $\vdash$and its negation $\nvdash$.

Definition 7.1 (Turnstiles)**:** Let $\phi_1 \ldots \phi_n$ and ψbe TFL sentences. We can say $\phi_1 \ldots \phi_n \vdash \psi$ iff there is a proof that ends in $\psi$and has $\phi_1 \ldots \phi_n$ as its premises. We can say $\phi_1 \ldots \phi_n \nvdash \psi$ if there is no such proof.

Definition 7.2 (Proof)**:** A TFL *proof* is a sequence of TFL sentences presented in a spefific format.

A proof is indented to represent its subproofs. The unindented portion of the proof is called the *main proof* and its column is called the *home column*.

A proof begins with the entry of zero or more *premises*, each entered in the home column on its own line along with (i) a line number (ii) the premise rule (PR) used as a justification. As a general practice, we will write (i) to the left of the TFL sentence and (ii) to its right. A horizontal line is drawn underneath the last premise.

Once the premises are added, we can complete the proof by applying one of the following at each step:

1. Remain in the currect column and apply one of the TFL rules to add a new sentence on a the next line, giving it the next line number and justifying it with the rule name.
2. Move one column to the right and add a sentence, justifying it with the assumption rule (AS) and giving it the next line number. This move opens a new *subproof*.
3. Move one column to the left by applying a rule which *closes* a subproof. Give the new line the next line number and justify it using the name of the rule applied.
4. If the most recent line is in the home column, you may end the proof. The final line of a proof is its *conclusion*.

where the rules are subject to the following contstraints:

1. In order to apply, a rule must take its input:
   a. from a previous line;
   b. that does not occur inside a closed subproof.
2. On the line immediately after closing a subproof, typically either:
   a. one of the rules which makes use of that subproof is applied next; or
   b. a new subproof is started so that we can use a rule which expects multiple subproofs.
3. A proof can only end if its final line is in the home column.

When $\Gamma$are its premises and ϕits conclusion, a successful proof can be referenced as $\Gamma \vdash \phi$. The argument that it proves is said to be *valid*. A proof is pure *syntax*. Only the rules of the proof system are relevant to its evaluation.

Of course, as with schemas, one of the things that makes proofs such a powerful part of logic is the fact that they are *truth-preserving under substitution of equivalents*. So if $\phi \vdash \chi$and $\phi \leftrightarrow \psi$, then $\psi \vdash \chi$. Among other things, this means that we can think of a proof as covering a whole collection of related sentences, namely all those that can be made from it with consistent substitutions.

7.2 Proof-Theoretic Concepts

Now we can use the more precise turnstile notation to express a variety of *proof-theoretic* concepts. Many of these will be familiar from exploring analogous *truth-functional* concepts earlier.

First, we note a sentence which can be proven from nothing is a *theorem*.

Definition 7.3 (Theorem)**:** A sentence $\phi$is a *theorem* iff $\vdash \phi$.

Similarly, sentences that can be proven from each other are *interderivable*.

Definition 7.4 (Interderivable)**:** Two TFL sentences $\phi$and $\psi$are *interderivable* iff $\phi\vdash\psi$and $\psi\vdash\phi$. We can write this as $\phi\dashv\vdash\psi$.

We can generalize the notion of consistency from a comparison of two sentences to a property of a collection of them.

Definition 7.5 (Joint Consistency)**:** TFL sentences $\phi_1 \dots \phi_n$ are *jointly inconsistent* iff $\phi_1 \dots \phi_n \vdash \bot$. Otherwise they are *jointly consistent*.

We can summarize the burdens of proof much as we did for truth tables.

proving	**yes**	**no**
theorem?	one	all
inconsistent?	one	all
equivalent?	two	all

Table 20: Metalogical Burdens of Proof

In this table we answer the question of how many proofs are needed to show the property in question. The main takeaway is that, while proving the affirmative is a straightfoward matter of logic, proving the negative requires metalogical reasoning about *all possible proofs*.

A special case of joint inconsistency is when there is only one sentence. In such a case, we can say that the sentence itself is *inconsistent*.

Definition 7.6 (Inconsistent Sentence)**:** A TFL sentence $\phi$is *inconsistent* iff $\phi \vdash \bot$.

Every TFL sentence that isn't inconsistent or a theorem is *contingent*.

Definition 7.7 (Contingent Sentence)**:** A TFL sentence $\phi$is *(proof-theoretically) contingent* if $\phi$is neither a theorem nor inconsistent.

And we can make formal the notion of a *valid argument* discussed informally earlier as provability of the conclusion from the premises.

Definition 7.8 (Valid Argument)**:** The argument $\phi_1 \dots \phi_n \therefore \psi$ is a *valid* iff $\phi_1 \dots \phi_n \vdash \psi$.

7.3 Proof-theoretic vs. Truth-functional

From a metalogical perspective, there remains the question of the relationship between the truth-functional and proof-theoretic approaches to logic. Metalogical proofs will be a major topic in Logic 2, but we will lay out the broadstrokes before wrapping up our module on Truth-Functional Logic.

Consider the how the definitions of the major logical concepts vary between the two systems.

concept	**truth-functional**	**proof-theoretic**
Tautology / Theorem	always T	derived without premises
Contradiction / Inconsistency	always F	can derive $\bot$from it
Contingent	sometimes T, sometimes F	not a theorem or inconsistency
Equivalent / Interderivable	identical main columns	derive mutually
Unsatisfiable / Inconsistent	no line where all T	can derive $\bot$
Satisfiable / Consistent	some line where all are T	cannot derive $\bot$from them
Valid argument	no lines w F ϕ_i and T ψ	can derive $\psi$from $\phi_1 \dots \phi_n$

Note that we have introduced four major notational devices that look remarkably similar. We should pause for a moment to distinguish them all.

- $\rightarrow$ The material conditional ($\rightarrow$) is a *logical* connective. It is the only one of these symbols that can be used in a sentence of TFL.
- $\therefore$ The inference marker ($\therefore$) is an *informal* device used in the metalogic to discuss arguments. $\Gamma \therefore \phi$does not imply that the argument is valid, but does specify its premises (Γ) and conclusion (ϕ).
- $\vDash$ The entailment marker ($\vDash$) is a formal device used to indicate the *truth-theoretic* validity of an argument. $\phi_1 \dots \phi_n \vDash \psi$ says that the conclusion (ψ) is true in every situation where the premises ($\phi_1 \dots \phi_n$) all are.
- $\vdash$ The provability marker ($\vdash$) is a formal device used to indicate the *proof-theoretic* validity of an argument. $\phi_1 \dots \phi_n \vdash \psi$ says that the conclusion (ψ) can be dervied from the premises ($\phi_1 \dots \phi_n$) by the natural deduction rules for TFL.

Nevertheless, there are parallels in how the symbols are used. The most significant parallels (soundness and completeness), taken together, mean that $\phi \vdash \psi$iff $\phi \vDash \psi$. But first consider a few more trivial observations.

When we were working on truth tables in Lecture 3, we showed how material implication and entailment are truth-functionally equivalent (i.e. $\phi \vDash \psi$iff $\vDash \phi \rightarrow \psi$).

A	B	A$\rightarrow$B	A$\vDash$B
T	T	T	T
T	F	F	F
F	T	T	T
F	F	T	T

Table 21: A truth table showing $\phi \rightarrow \psi$iff $\phi \vDash \psi$

But note that we have to view the meaning of T and F in the A$\vDash$B column in a special way: as *good line* and *bad line*, respectively. Truth is not an evaluation we make about potential entailments, instead we talk about validity.

Likewise, we can be explicit about the relationship between *provability* and *inconsistency*.

Observation 7.1 (Proof and Inconsistency)**:** Let $\Gamma = \gamma_1 \dots \gamma_n$ be a set of TFL sentences and ϕ a single TFL sentence. If $\Gamma \vdash \phi$ then the set that results from adding $\neg\phi$ to Γ ($\Gamma \cup \neg\phi$) is inconsistent.

7.4 Soundness and Completeness

Two of the metalogical results that we will not prove here are *soundness* and *consistency*. We will define them and them assume that they are true as we wrap up our discussion of proofs for logical properties.

Soundness says that we do not get any derivations that are not also entailments.

Definition 7.9 (TFL Soundness)**:** A proof system is *sound* iff $\phi \vdash \psi$ implies $\phi \vDash \psi$ for all TFL sentences ϕ and ψ.

Conversely, completeness says that we do not get any entailments that are not also derivations.

Definition 7.10 (TFL Completeness)**:** A proof system is *complete* iff $\phi \vDash \psi$ implies $\phi \vdash \psi$ for all TFL sentences ϕ and ψ.

Knowing that a proof system is sound and complete means that we can demonstrate a logical property using either proof-theoretic or truth-functional appoaches and know that it holds for both. This means we can leverage the assymmetries in the last table and approach the problem in whatever way is easiest. The following table demonstrates such a strategy.

property	**prove present**	**prove absent**
Tautology, Theorem	derive the sentence	find a F line
Contradiction	derive $\bot$	find a T line
Contingency	find T and F lines	prove ϕ or $\neg\phi$
Equivalence	mutually derive	find a line with truth mismatch
Consistency	find a universally T line	derive $\bot$
Validity	derive ψ from $\phi_1 \dots \phi_n$	find a line with $\phi_1 \dots \phi_n$ all T and ψ F

Table 22: Metalogical Proof Strategies for TFL

8 An Informal Survery of First-Order Logic

Recall from our discussion that TFL falls short of accurately depicting some aspects of arguments and inferences in ordinary English. We repeat our ealier summary for convenience.

Observation 8.1 (Shortcomings of TFL)**:** Truth-functional logic has some shortcomings that make it ill-suited to deal with some phenomena from classical logic and ordinary English:

1. TFL does not support the types of inferences depicted in Aristotle's syllogisms.
2. TFL insists on *excluding the middle* – every TFL is either true or false – but some adjectives do not quite work like that.
3. TFL comes with a number of *paradoxes* that make the material conditional unsatisfying for symbolizing English sentences: one the one hand, negating the whole conditional often gives the wrong conclusions about its antecedent sentence; on the other hand, negating the consequent sentence opens up the possibility of contradiction when the antecedant sentence is false.

We will need to wait until future logic courses to deal with (2) and (3)– they are matters for *non-classical logic* and *modal logic*, respectively–but we can start trying to formalize a system to capture Aristotle's syllogism results right away. This system is *first-order logic*, also commonly refered to as *predicate logic*.

8.1 Artistotle and the Ancient Origins of Predicate Logic

Let's start with a review of Aristotle's syllogisms. The system is based on the organization of four *quantifiers* (although that term does not come until later) into a *square of opposition*.

Definition 8.1 (Aristotle's *square of opposition*)**:** Accounting for the the effects negation on *universal* ("every") and *existential* ("some") statements, there are four distinct *quantifiers*. We can lay them out in opposition to their *duals* (i.e. negations) in the following square. Each distinct quantifier is labeled with a vowel (A,E,I,O) so that we can refer to it easily.

A: Every A is B. E: No A is B.

O: Some A is not B. I: Some A is B.

Armed with notations for these four types of sentences, we can lay out the syllogisms (later named by medeival scholars) into three classes, based on how they are connected by the so-called *middle term* (B), which can occur in either the *subject* or *predicate* position of each of the two premise sentences.

Medeival Name	Aristotle Name			
Barbara	(AAA)	All B are C.	All A are B.	So all A are C.
Celarent	(EAE)	No B are C.	All A are B.	So no A are C.
Darii	(AII)	All B are C.	Some A is B.	So some A are C.
Ferio	(EIO)	No B are C.	Some A is B.	So some A is not C.
Cesare	(EAE)	No C are B.	All A are B.	So no A are C.
Camestres	(AEE)	All C are B.	No A are B.	So no A are C.
Festino	(EIO)	No C are B.	Some A is B.	So some A is not C.
Baroko	(AOO)	All C are B.	Some A is not B	So some A is not C.
Datisi	(AII)	All B are C.	Some B is A.	So some A is C.
Disamis	(IAI)	Some B is C.	All B are A.	So some A is C
Ferison	(EIO)	No B are C.	Some B is A.	So some A is not C.
Bokardo	(OAO)	Some B is not C.	All B are A.	So some A is not C.
Camenes	(AEO)	All C are B.	No B are A.	So no A are C.
Dimaris	(IAI)	Some C is B.	All B are A.	So some A is C.
Fresison	(EIO)	No C is B.	Some B is A.	Some A is not C.

Table 23: Artistotle's syllogisms with medeival names

8.2 Predicates and Singular Terms

Recall that the problem with treating these examples in TFL is that TFL does not have a facility for making distinctions between atomic sentences, aside from their truth. More precisely, we cannot capture the relationship between "all men" and "Socrates" that allows us to make the following inference, analogous to *Barbara*.

All men are mortal.

Socrates is a man.

$\therefore$ Socrates is mortal.

Nor can we capture the fact that "is/are mortal" and "is a man" are something like properties that we can apply to things like "all men" and "Socrates". With first-order logic, we get some rudimentary tools to depict these phenomena formally.

Let's start with some a informal review of the phenomena.

Consider first some simple sentences that differ as little as possible.

John is reading.

Mary is reading.

Fred is reading.

Susie is reading.

We can see a pattern here involving two components.

First, there is whatever the expressions "John", "Mary", "Fred" and "Susie" have in common. We can tentatively call these expressions *proper names* (*names*) and note that they refer *directly* to the individuals that they symbolize (which can include any kind of "person, place or thing"). For all intents and purposes, we can think of them as symbolizing *the objects themselves* as opposed to some mental or linguistic intermediates. Names are a special case of *singular terms* which pick out individuals for *reference*: they are special because they refer to **exactly one** item without ambuguity. We will talk more about other singular terms when we discuss quantifiers and variables. Names will be symbolized in FOL using lower case latin letters ($a, b, c, \dots$).

Second, there is the rest of the sentence.

[ ]$_x$ is reading.

We will refer to such "incomplete sentences" as *predicates* and to the "gaps" (here labelled with an x) as *slots*. Note that predicates can be made by introducing slots into multiple locations of sentences, depending on how many are appropriate to the predicate word (i.e. *verb*, *adjective* or *common noun* in question. Consider this one which could have a gap in any of three locations.

[ ]$_x$ gave a book to Mary.
John gave [ ]$_x$ to Mary.
John gave a book to [ ]$_x$.

We will call a verb like "give" a *three-place predicate*, and generally refer to a predicate with n gaps as an *n-place predicate*. When $n > 1$ will will sometimes refer to these predicates as *n-place relations*. Predicates and relations will be symbolized in FOL using capital latin letters ($A, B, C, \dots$).

8.3 Introducing FOL Quantifiers

Notice that one of the singular terms used in the last example is not a name: "a book". This is where the quantifiers come in. Supposing that the gaps in our predicates can themselves be labeled with symbols (*variables*, symbolized by lower-case latin letters $\dots, x, y, z$), then we can tentatively express "John gave a book to Mary" using *quantifiers* to *bind* these variables. Let G symbolize *give*, j and m symbolize "John" and "Mary", respectively, B symbolize "is a book" and x stand for a variable in both G and B that is also *bound* by the quantifier $\exists$.

$$\exists x(B(x) \wedge G(j, x, m))$$

Pronounced in ordinary English, this FOL sentence says "there is an x such that x is a book and John gives x to Mary" which is more or less what we understand the original English sentence to mean.

We will discuss binding and variables in more detail as we formalize the logic. For now, note that there are two distinct *quantifiers* in first-order logic.

Definition 8.2 (FOL Quantifiers (Informal))**:** First-order logic has two distinct quantifiers:

1. **Existential Quantifier (∃x):** expresses the English paraphrase "for at least one x".... English expressions used to indicate this kind of quantification include "there is/are" and "some".

2. **Universal Quantifer (∀x):** expresses the English paraphrase "for all x".... English expressions used to indicate this kind of quantification include "every", "any", "all", and "each". [a]

In first-order logic, every quantifier must be followed by a variable indicating which of the predicate gaps it *binds* in the interpretation of a sentence. Assigning variables to quantifier and gaps is the main mechanism for disabiguating complex English sentences within the FOL framework (that is not already available in TFL). Often it is more art than science.

[a] Beware that these terms are not used with perfect consistently in ordinary English, especially when subjunctive contexts are involved.

In a sense, when we talk about quantifiers, our subject is really quantified sentences: formulas with a subject and more than one predicate. We will build the sentences that use these quantifiers in a consistent and conventional way, from $(Ac \to Bc)$ for universals and from $(Aa \wedge Ba)$ for existentials. The form of these sentences is important to how quantifiers work in first-order logic. Adding first-order quantifiers to the language in this way makes it possible to model a variety of the inferences explored in classical logic by ancient and medievel scholars.

Definition 8.3 (Square of Opposition in FOL)**:** Just these two quantifiers and negation are enough to express all four corners of Aristotle's *square of opposition*.

Every A is B.	No A is B.
$\forall x(A(x) \to B(x))$	$\neg\exists x(A(x) \wedge B(x))$
Not every A is B.	Some A is B.
$\neg\forall x(A(x) \to B(x))$	$\exists x(A(x) \wedge B(x))$

Similarly, by folding in negation, we can represent additional English quantifiers such as *only*.

Only As are Bs.
$\neg\exists x(B(x) \wedge \neg A(x))$

8.4 Domains of Individuals

One question that arises immediately when dealing with a quantifier like ∀is "every what?" To answer this question we make reference to a *domain*.

Definition 8.4 (Domain (Informal))**:** In first-order logic, a *domain* is the collection of objects that we are talking about.

1. in general, *singular terms* refer to objects in the domain
2. every *proper name* must pick out **exactly** one object in the domain;
3. each domain item (*individual*) can be picked out by any number of proper names (including zero).

As we will see when we begin working through the semantics and natural deduction systems for FOL, the choice of domain is a crucial determinant of the behavior of a first-order logic. Ordinary English is typically understood as *fixing* the domain of its quantifying expressions, when we say "Bobby" we are typically clear about which individuals are relevant to our conversation, especially the one that we mean by "Bobby". For us, choice of domain is important, but more art than science.

We will typically define our domains in one of two ways: *extensionally*, where we list all of the individuals; and *descriptively*, where we use ordinary language (or, later, a metalogical formula) to explain the contents of the domain.

Among the effects of the domain on FOL involves the absence of referential material. There are two prominent cases: when a singular term is used, but it does not refer to any objects in the domain; when a predicate is used, but it does not apply to any objects in the domain. An expression of the first kind is *Santa Claus*; an expression of the second is *even prime greater than 3*. Both of these phenomena can result in unexplected results. To amelieroate this we will prohibit names that do not refer and we will give predicates that do not apply to any objects (*empty predicates*) a special treatment.

Definition 8.5 (Empty Predicates)**:** When $\beta$is an empty predicate, any sentence $\forall x(\beta(x) \rightarrow \dots)$ is (vacuously) true.

8.5 Quantifiers and Variable Choice

When we symbolize sentences using names, the identity of the name symbol is enough to indicate that two sentences are "about" the same object.

$$A(c) \wedge B(c)$$

This can be true even when the sentences are separated by any number of others that may or may not contain any examples of the name in question. The name, quite simply, always refers to exactly the same thing just in virtue of being a proper name.

But when a quantifier is used, the specific sentences to which the quantifier should be applied is a more delicate matter. The following sentences all express slightly different things.

If c is A, then c is B.	$A(c) \rightarrow B(c)$
For all x, if c is A, then x is B.	$\forall x(A(c) \rightarrow B(x))$
For all x, if x is A, then x is B.	$\forall x(A(x) \rightarrow B(x))$

8.6 Multi-Place Predicates

Not only do we need to carefully track the variables across predicates, we also need to keep them straight within a single predicate. As we noted above, some predicates (i.e. *n-place predicates*) could have more than one gap to fill when used in FOL sentences. The example we presented for the verb *give* is repeated here.

[ ]$_x$ gave a book to Mary.
John gave [ ]$_x$ to Mary.
John gave a book to [ ]$_x$.

We refer to the number of slots in an n-place predicate as its *arity* [12], and to an n-place predicate as *n-ary*. We can consider the relevant three-place predicate with all its gaps to be something like:

$$[\quad]_x \text{ gave } [\quad]_y \text{ to } [\quad]_z.$$

symbolized in FOL as:

$$G(x, y, z)$$

The order of x, y, z is meaningful and important, but arbitrary. We can assign the labels in the order that they appear in the original sentence. Note that this convention must be used with care, because the following English sentence has the same meaning but presents its gaps in a different order.

$$[\quad]_x \text{ gave } [\quad]_z \; [\quad]_y.$$

The *role* that the terms play in the sentence is more important than their English order when deciding how to assign terms to the FOL predicate.

8.7 Multiple Quantifiers

When more than one quantifier is used, the process of interpretation can get somewhat complicated. There are three major reasons for this:

[12] *unary*, *binary*, *ternary*, *4-ary*, etc.

Observation 8.2 (Complications for the Interpretation of Multiple Quantifiers)**:** When we have more that one quantifier, interpretation is complicated by three main factors:

1. The order that quantifiers occur is relevant to the interpretation of FOL sentences.
2. The specific choice of variables that the quantifiers apply to is relevant to the interpretation of FOL sentences.
3. The TFL connectives, especially negation, support radically different interpretations, depending on how they are taken to interact with quantifiers in FOL sentences.

Let's look at each of these in turn. First, note that changing the order of the quantifiers can change the meaning of the sentence.

$\forall x \exists y L(x, y)$ = Everyone loves someone. (maybe each someone different)
$\exists y \forall x L(x, y)$ = There is someone that everyone loves. (maybe Sara Lee)

Second, the application of a quantifier can vary. We will use '(', ')', '[' and ']' to indicate the so-called *scope* of a quantifier–the portion of the overall sentence to which the quantifier applies. Here we have four distinct quantifiers ($\forall x, \exists z, \exists y, \exists z$) even though the variable z is used to symbolize two of them.

$$\forall x[\exists z(D(z) \wedge O(x, z)) \rightarrow \exists y(\exists z(D(z) \wedge O(y, z)) \wedge F(x, y))]$$

Finally, the interaction of negation with quantifier scope is a critical to the interpretation of quantification phenomena that go beyond the two basic quantifiers. Recall, for example, how we symbolize the square of opposition. The flexible use of negation and scope allows us to express the FOL sentences with whichever of the quantifiers that we prefer.

Every A is B.	No A is B.
No A is not B.	Every A is not B.
$\forall x(A(x) \rightarrow B(x))$	$\neg\exists x(A(x) \wedge B(x))$
$\neg\exists x(A(x) \wedge \neg B(x))$	$\forall x(A(x) \rightarrow \neg B(x))$

Not every A is B.	Some A is B.
Some A is not B.	Not every A is not B.
$\neg\forall x(A(x) \rightarrow B(x))$	$\exists x(A(x) \wedge B(x))$
$\exists x(A(x) \wedge \neg B(x))$	$\neg\forall x(A(x) \wedge \neg B(x))$

8.8 Paraphrasing English for First-Order Logic

As we did for TFL, we can use paraphrase as a stepping stone to complete formalization in FOL. A few rules of thumb might help.

Rule 8.1 (Methods of Paraphrase)**:** When paraphrasing English as a intermediary to first-order logic, some consistent methods can help guide the process:

1. Move quantifiers forward to clarify their scope, using pronouns to mark their slot
   a. Every good boy deserves fudge.
   b. (Every good boy is such that) he deserves fudge.
2. Replace pronouns with variables early, to avoid ambiguity. This is especially helpful when more than one quantifier is used.
   a. (Every good boy is such that) he deserves fudge.
   b. (Every good boy x is such that) x deserves fudge.
3. Set up the subject and predicate as paraphrases before symbolizing.
   a. (Every good boy x is such that) x deserves fudge.
   b. (Every x is such that)(x is a good boy $\rightarrow$x deserves fudge)
4. Formalize the quantifiers once their subject predicates are handled.
   a. (Every x is such that)(x is a good boy $\rightarrow$x deserves fudge)
   b. ($\forall$x)(x is a good boy $\rightarrow$x deserves fudge)
5. Once all of the basic sentences are made of just an n-ary predicate and n names or variables, then we can convert them to FOL sentences, using the symbolization key.
   a. ($\forall$x)(x is a good boy $\rightarrow$x deserves fudge)
   b. ($\forall$x)(Gx $\rightarrow$Fx)

Consider this sequence of examples adapted from Smith, 2020. For all of the examples, we will assume the following extensionally defined interpretation.

Interpretation 8.1 (Interpretation for Examples (1)-(12))**:** The interpretation will have three names.

m: Mariah
n: Naomi
o: Oliver

Its domain will be all people currently living. And its predicates are defined descriptively as follows.

F: ① is a man
G: ① is a woman
L: ① loves ②
M: ① is married to ②
R: ① is the child of ② and ③

8.8.1 Examples with One Quantifier

(1) There is someone who loves Mariah, Naomi and Oliver.
≃ (Somone is such that) they love Mariah, Naomi and Oliver.
≃ (Some x is such that) x loves Mariah, Naomi and Oliver.
≃ (∃x) x loves Mariah, Naomi and Oliver.
≃ (∃x)(x loves Mariah ∧x loves Naomi ∧x loves Oliver)
≃ ∃x(Lxm∧Lxn∧Lxo)

(2) Whoever is loved by Oliver is loved by Mariah too.
≃ (Everyone who is loved by Oliver is such that) Mariah loves them.
≃ (Every x who is loved by Oliver is such that) Mariah loves x.
≃ (Every x is such that) (if x is loved by Oliver then Mariah loves x)
≃ (∀x)(Oliver loves x →Mariah loves x)
≃ ∀x(Lox→Lmx)

(3) Every woman who loves Mariah is loved by Oliver.
≃ (Every woman x who loves Mariah is such that) x is loved by Oliver.
≃ (Every x is such that) (if x is a woman who loves Mariah, then Oliver loves x)
≃ (∀x) (x is a woman and x loves Mariah →Oliver loves x)
≃ ∀x((Gx∧Lxm)→Lox)

(4) Mariah loves some woman who loves Oliver.
≃ (Some woman x who loves Oliver is such that) Mariah loves x
≃ (Some x is such that) (x is a woman who loves Oliver and Mariah loves x)
≃ (∃x) (x is a woman and x loves Oliver ∧Mariah loves x)
≃ (∃x) (x is a woman ∧x loves Oliver ∧Mariah loves x)
≃ ∃x(Gx ∧Lxo ∧Lmx)

(5) Naomi and Oliver love any child of theirs.
≃ (Every x who is a child of Naomi and Oliver is such that) (Naomi and Oliver love x)
≃ (Every x is such that) (if x is a child of Naomi and Oliver then Naomi and Oliver love x)
≃ (∀x) (x is a child of Naomi and Oliver →(Naomi loves x ∧Oliver loves x))

$\simeq \forall x(Rxno \rightarrow (Lnx \wedge Lox))$

(6) Oliver is Mariah's child.
$\simeq$ (Some x is such that)(Oliver is Mariah and x's child)
$\simeq \exists xRomx$

(7) Mariah is married to no one.
$\simeq$ (it is not the case that)(Mariah is married to someone)
$\simeq$ (it is not the case that) ((there is some x such that) Mariah is married to x)
$\simeq (\neg)((\exists x)$ Mariah is married to x)
$\simeq \neg\exists xMmx$

(8) No one who loves Naomi loves Oliver.
$\simeq$ (It is not the case that)(Someone who loves Naomi loves Oliver)
$\simeq \neg$(Some x such that)(x loves Naomi $\wedge$x loves Oliver)
$\simeq \neg(\exists x)$(x loves Naomi $\wedge$x loves Oliver)
$\simeq \neg\exists x(Lxn \wedge Lxo)$

(9) Naomi is a woman everyone loves.
$\simeq$ Everyone loves Naomi $\wedge$Naomi is a woman
$\simeq$ (Every x is such that)(x loves Naomi) $\wedge$Naomi is a woman
$\simeq (\forall x)$(x loves Naomi) $\wedge$Naomi is a woman
$\simeq \forall xLxn \wedge Gn$

(10) If everyone loves Naomi then Oliver does.
$\simeq$ (Every x is such that)(If x loves Naomi then Oliver loves Naomi)
$\simeq (\forall x)$(x loves Naomi $\rightarrow$Oliver loves Naomi)
$\simeq \forall x(Lxn \rightarrow Lon)$

(11) If someone loves Naomi then Oliver does.
$\simeq$ Someone loves Naomi $\rightarrow$Oliver loves Naomi
$\simeq$ (Some x is such that)(x loves Naomi) $\rightarrow$Oliver loves Naomi
$\simeq (\exists x)$(x loves Naomi) $\rightarrow$Oliver loves Naomi
$\simeq \exists xLxn \rightarrow Lon$

(12) If anyone loves Naomi then Oliver does.
$\simeq$ (Everyone is such that) (If they love Naomi then Oliver loves Naomi)
$\simeq (\forall x)$(x loves Naomi $\rightarrow$Oliver loves Naomi)
$\simeq \forall x(Lxn \rightarrow Lon)$

8.8.2 Examples with Multiple Quantifiers

(1) Every man loves someone or other.
$\simeq$ (Every man is such that) he loves someone or other.
$\simeq$ (Every x is such that) (x is man $\rightarrow$x loves someone or other)
$\simeq$ ($\forall$x) (x is man $\rightarrow$(Some y such that)(x loves y))
$\simeq$ ($\forall$x)(x is a man $\rightarrow$($\exists$y)(x loves y))
$\simeq$ $\forall$x(Fx$\rightarrow\exists$yLxy)

(2) Some man loves a woman.
$\simeq$ (Some man is such that)(he loves a woman)
$\simeq$ (Some x is such that)(x is a man $\wedge$x loves a woman)
$\simeq$ ($\exists$x)(x is a man $\wedge$(some woman y is such that)(x loves y))
$\simeq$ $\exists$x(x is a man $\wedge$($\exists$y)(x is a woman $\wedge$x loves y))
$\simeq$ $\exists$x(Fx$\wedge\exists$y(Gx $\wedge$Lxy))

(3) Someone Oliver loves is loved by everyone Naomi loves.
$\simeq$ (Someone Oliver loves is such that) (they are loved by everyone Naomi loves)
$\simeq$ (Some x is such that) (Oliver loves x $\wedge$x is loved by everyone Naomi loves)
$\simeq$ ($\exists$x) (Oliver loves x $\wedge$(Everyone Naomi loves is such that) x is loved by them)
$\simeq$ $\exists$x(Oliver loves x $\wedge$(Everyone y is such that)(Naomi loves y $\rightarrow$x is loved by y))
$\simeq$ $\exists$x(Oliver loves x $\wedge$($\forall$y)(Naomi loves y $\rightarrow$y loves x))
$\simeq$ $\exists$x(Lox $\wedge\forall$y(Lny$\rightarrow$Lyx))

(4) No woman loves every man.
$\simeq$ (It is not the case that) (Some woman is such that) (She loves every man)
$\simeq$ $\neg$(Some x is such that)(x is a woman $\wedge$x loves every man)
$\simeq$ $\neg$($\exists$x)(x is a woman $\wedge$x loves every man)
$\simeq$ $\neg$($\exists$x)(x is a woman $\wedge$(every man is such that)(x loves him))
$\simeq$ $\neg\exists$x(x is a woman $\wedge$(every y is such that)(y is a man $\rightarrow$x loves y))
$\simeq$ $\neg\exists$x(x is a woman $\wedge\forall$y(y is a man $\rightarrow$x loves y))
$\simeq$ $\neg\exists$x(Gx$\wedge\forall$y(Fy$\rightarrow$Lxy))

(5) No woman loves any man.
$\simeq$ (It is not the case that) (Some woman is such that) (She loves some man)
$\simeq$ $\neg$(Some x is such that)(x is a woman $\wedge$x loves some man)
$\simeq$ $\neg$($\exists$x)(x is a woman $\wedge$(some man is such that)(x loves him))
$\simeq$ $\neg\exists$x(x is a woman $\wedge$(some y is such that)(y is a man $\wedge$x loves y))
$\simeq$ $\neg\exists$x(x is a woman $\wedge$($\exists$y)(y is a man $\wedge$x loves y))

$\simeq$ $\neg\exists$x(Gx $\wedge\exists$y(Fy $\wedge$Lxy))

(6) Anyone who is married loves someone they aren't married to.

$\simeq$ (Everyone who is married is such that) (they love someone they aren't married to)

$\simeq$ (Every x is such that) (x is married $\rightarrow$x loves someone x isn't married to)

$\simeq$ ($\forall$x) (x is married $\rightarrow$(some y is such that) (x loves y $\wedge$x isn't married to y))

$\simeq$ $\forall$x(x is married $\rightarrow(\exists$y) (x loves y $\wedge$x isn't married to y))

$\simeq$ $\forall$x(x is married $\rightarrow\exists$y(Lxy$\wedge\neg$Mxy))

(7) A married man only loves women.

$\simeq$ (Every married man is such that)(he only loves women)

$\simeq$ ($\forall$x)(x is a married man $\rightarrow$x only loves women)

$\simeq$ ($\forall$x)((x is a man $\wedge$x is married) $\rightarrow$x only loves women)

$\simeq$ $\forall$x((x is a man $\wedge$x is married) $\rightarrow$(Every y is such that(x loves y $\rightarrow$y is a woman)))

$\simeq$ $\forall$x((x is a man $\wedge$x is married) $\rightarrow(\forall$y)(x loves y $\rightarrow$y is a woman))

$\simeq$ $\forall$x((x is a man $\wedge$(some z is such that)(x is married to z)) $\rightarrow(\forall$y)(x loves y $\rightarrow$y is a woman))

$\simeq$ $\forall$x((x is a man $\wedge(\exists$z)(x is married to z)) $\rightarrow(\forall$y)(x loves y $\rightarrow$y is a woman))

$\simeq$ $\forall$x((Fx $\wedge\exists$zMxz)$\rightarrow\forall$y(Lxy$\rightarrow$y Gy))

(8) No man loves anyone who loves Naomi.

$\simeq$ (it is not the case that)(some man is such that) (he loves anyone who loves Naomi)

$\simeq$ $\neg$(some x is such that) (x is man $\wedge$x loves anyone who loves Naomi)

$\simeq$ $\neg(\exists$x) (x is man $\wedge$(every y is such that)(y loves Naomi $\rightarrow$x loves y))

$\simeq$ $\neg\exists$x(x is man $\wedge\forall$y(y loves Naomi $\rightarrow$x loves y))

$\simeq$ $\neg\exists$x(Fx$\wedge\forall$y(Lyn$\rightarrow$Lxy))

8.9 Basic Inference Steps for First-Order Logic

Based on the inference patterns we see in arguments using them, we can tentatively describe four new valid inference steps to add to the system we developed for truth-functional logic.

Observation 8.3 (Valid Quantificational Inference Steps)**:** There are four valid inference steps that we can take using the quantifiers of first-order logic:

1. **Universal Instantiation:** From a universal sentence, we can infer any particular instance.
2. **Universal Generalization:** From a particular sentence proven true of *an arbitrarily selected* individual, we can infer the universal generalization of that sentence.

3. **Existential Generalization:** From any particular sentence, we can infer the existential generalization of it.
4. **Existential Instantiation:** From an existential sentence, we can infer the existential instantiation of that sentence for a given name, provided:
   a. the name is not already used elsewhere in the proof
   b. the existential sentence does not contain the name

8.10 Identity of Individuals

We still have more to say about singular terms. One open issue is the question of how to keep them distinct. As it stands, we know that a name always refers to the same individual in the domain (and that that individual is indeed in the domain) but we cannot guess whether two given names refer to one individual or two, both alternatives are permitted. Similarly, we have no way of expressing the so-called *count quantifiers* such as *at least two*.

To tackle both of these problems, we will introduce a special two-place predicate (=) that we will interpret as *identity*. In other words, when we write $a = b$, we will take that to mean that a and b both refer to the same individual in the domain. So we can symbolize the simple identity sentence "Cicero is Tully" with the FOL sentence $c = t$. We will call this type of identity *numerical identity*: the two names talk about **the exact same (one) thing**.

We can use the negation of identity sentences to express concepts like "everyone else": $\forall x(x \neq c \rightarrow A(c, x))$, where $x \neq y$ means $\neg(x = y)$. Likewise, when we want to symbolize "only", "at least two" and "at most two", respectively, we can use the FOL sentences:

$$A(c) \wedge \neg\exists x(x = c \wedge A(x))$$
$$\exists x(A(x) \wedge \exists y(A(y) \wedge x \neq y))$$
$$\neg\exists x\exists y\exists z[((A(x) \wedge A(y) \wedge A(z)) \wedge ((x \neq y \wedge x \neq z) \wedge y \neq z)]$$

When we want to express concepts like *exactly 2*, we need to treat it as *at least 2* ***and*** *at most 2*, symbolizing it as a conjuntion of the two.

$$\exists x\exists y[(A(x) \wedge A(y) \wedge x \neq y] \wedge \neg\exists x\exists y\exists z[((A(x) \wedge A(y) \wedge A(z)) \wedge ((x \neq y \wedge x \neq z) \wedge y \neq z)]$$

8.11 Definite Descriptions

Finally, there is a type of expression whose interpretation seems to be sometimes best treated as a singular term and sometimes as a quantifier. Consider the following example.

> The dog is waiting to come in.

There are two important steps in the interpretation of this sentence that center on the use of the

expression "the dog" here. First, we are using it like a singular term to express a proposition about a particular dog that we have in our domain. Second, we are using quantificational elements to clarify which of the domain objects we have in mind: namely, the one and only dog in the domain.

Famously, Bertrand Russell argued that we should merge these two aspects of such expressions (callled *definite descriptions*) by treating "the" as a type of quantifier. We will look at the other appoach first.

Suppose we just stipulate that "the dog" works like a special kind of name, picking out its referent *directly* and operating as a singular term whenever it occurs in a sentence. Then we might be able to symbolize the sentence about our waiting dog as:

$$W(\iota x(D(x)))$$

Here the expression $\iota x(D(x))$ behaves as a singular term, filling the gap in the predicate W. For Russell's approach, we can just treat *the* as an *existential quantifier* with a *uniqueness condition*. We've already seen both of these devices. We can use them together to express our example:

$$\exists x((D(x) \wedge \neg\exists y(x \neq y \wedge D(x))) \wedge W(x))$$

There are problems with both approaches. We will adopt Russell's, since the ι operator introduces *truth gaps*, which are beyond the scope of this introduction.

There are three traditional ways to formulate Russell's formula for definite descriptions.

(R1) $\exists$x(Ax$\wedge\forall$y(Ay$\rightarrow$x=y) $\wedge$Bx)

(R2) $\exists$x$\forall$y((Ay$\leftrightarrow$x=y)$\wedge$Bx)

(R3) (($\exists$xAx$\wedge\forall$x$\forall$y((Ax$\wedge$Ay)$\rightarrow$x=y))$\wedge\forall$x(Ax$\rightarrow$Bx))

As we will see, they are all equivalent.

9 The Grammar of First-Order Logic

9.1 Components of Syntax

Last week, we explored this ins and outs of first-order logic informally. Now we'll start to formalize those ideas, starting with the basic grammar of FOL.

Definition 9.1 (Expressions of First-Order Logic)**:** : We can difine an *expression* of FOL as any sequence of its six kinds of basic symbols:

1. **Predicates:** $A, B, C, \ldots, =$
2. **Names:** $a, b, c, \ldots$
3. **Variables:** $x, y, z, \ldots$
4. **Connectives:** $\neg, \wedge, \vee, \rightarrow, \leftrightarrow$
5. **Quantifiers:** $\forall, \exists$
6. **Brackets:** (,)

The alphabetic symbols (names, predicates and variables) can be used with or without numeric subscripts, for example $A_1, A_2, A_3, \ldots$ or $x_1, x_2, x_3, \ldots$.

We assume these basic expressions have roughly the meaning discussed informally. But, of course, that is not important here. All we want to do with a grammar is to say which expressions are sentences and which are not.

In the metalanguage we will refer to expressions with lower case greek letters.

The most basic basic expressions in FOL are *singular terms*. Expressions of this kind are noteworthy in their semantics: they each *refer* to an individual in the domain. So we will keep them distinct in the syntax.

Definition 9.2 (Term)**:** A *term* in first-order logic is any *name*. A name in FOL can be one of two kinds:

1. **Dummy Names:** lower case a and c, with or without subscripts **Proper Names:** lower case latin letters, but not a or c

Dummy names are placeholders, like variables. But unlike variables, they are full-fledged terms: we assume that they each refer directly to an individual in the domain. Note that on this definition, variables are not terms. This is as it should be, since they do not *refer*. More on this when we get to the semantics. For now, it is noteworthy that variables are used wherever terms are allowed.

They are not terms, but artefacts of the process that introduces quantifiers. Before we talk about that process, let's work through a few more preliminary definitions.

Definition 9.3 (Predicate)**:** An *n-place predicate* in first-order logic is an expression which takes n terms as *arguments* to make an atomic formula.

Terms and predicates are the only kinds of non-logical vocabulary that we will make use of. Everything else is logic.

Definition 9.4 ((Non-)Logical Vocabulary)**:** The *logical vocabulary* of first-order logic includes all of the brackets, quantifiers, operators and connectives. These do not vary between the languages of FOL. They all use the same *syntax*.

The *non-logical vocabulary* of first-order logic includes all of the so-called *content expressions* of the language: *proper names* and *predicates* of every arity. These will vary from language to language.

Even though *variables* and *dummy names* can occur in the slots of predicates, they are *logical vocabulary*. They are only ever used in the context of an associated quantifier.

As with TFL, we can express generalizations about the logic using a consistent metalogical notation.

Definition 9.5 (Metalogical Symbols)**:** We will adopt the following conventions for the use of metalogical variables in describing first-order logic:

α for expressions (in general)
τ for terms (in general)
ν for proper names
δ for dummy names
ξ for variables
β for predicates
ϕ for well-formed formulas

9.2 Well-Formed Formulas

Unlike TFL, FOL needs to define its atomic sentences since they are not mere truth-bearers but have constituent parts.

Definition 9.6 (Atomic Formula)**:** An *atomic formula* of first-order logic is defined as:

1. Any zero-place predicate is an atomic formula.

2. If $\beta$is an n-place predicate and $\tau_1 \dots \tau_n$ are terms, then $\beta\tau_1, \dots, \tau_n$ is an atomic formula.
3. If τ_1 and τ_2 are terms then $\tau_1 = \tau_2$ is an atomic formula.
4. Nothing else is an atomic formula.

As we will see next lecture, we can determine a truth value for everything that we have defined as an atomic formula. Now that we have these basic truth-bearers, we can fold in the components of truth-functional logic. All of the TFL connectives will be included.

Definition 9.7 (Well-Formed Formula)**:** A *well-formed formula* (*wff*) of first-order logic is defined recusively as:

1. Any atomic formula of FOL is a wff of FOL
2. $\bot$is a wff of FOL.
3. If $\phi$is a wff of FOL, then $\neg\phi$is a wff of FOL.
4. If $\phi$and $\psi$are wffs of FOL, then $\phi \wedge \psi$ is a wff of FOL.
5. If $\phi$and $\psi$are wffs of FOL, then $\phi \vee \psi$ is a wff of FOL.
6. If $\phi$and $\psi$are wffs of FOL, then $\phi \rightarrow \psi$ is a wff of FOL.
7. If $\phi$and $\psi$are wffs of FOL, then $\phi \leftrightarrow \psi$ is a wff of FOL.
8. If $\phi(\tau)$ is a wff of FOL containing one or more occurrences of the term $\tau$and $\phi(\xi)$ is the expression which results from replacing τwherever it occurs in $\phi(\tau)$ with the new variable ξ, then $\exists\xi\phi(\xi)$ and $\forall\xi\phi(\xi)$ are wffs of FOL.
9. Nothing else is a wff of FOL.

We've added two new rules for quantified sentences. We will refer to $\forall\xi$and $\exists\xi$as *quantifiers*, to $\forall$and $\exists$as *quantifier symbols*, and to ξas a *variable*. $\forall\xi$is called *universal* and $\exists$is called *existential*.

9.3 Interlude: Free Variables vs Dummy Names

Notice that on this forumulation of the grammar, variables are artefacts of the process that introduces quantifiers. If you've been following along in Magnus et al., 2025, you will notice that this departs from the rules presented there. We will discuss the differences briefly before committing ourselves to the rules in Def 9.7.

There are two important differences in the grammatical rules for formulas. First, Magnus et al., 2025 allow variables to serve as *terms*, making them available in the construction of *atomic formulas*. Second, they therefore allow the construction of sentences which contain *free variables*: variables that are not within the scope of any quantifiers.

Definition 9.8 (Well-Formed Formula (Magnus et al., 2025)**:** A *well-formed formula* (*wff*) of first-order logic is defined recusively as:

1. Any atomic formula of FOL is a wff of FOL
2. $\bot$is a wff of FOL.
3. If $\phi$is a wff of FOL, then $\neg\phi$is a wff of FOL.
4. If $\phi$and $\psi$are wffs of FOL, then $\phi \wedge \psi$ is a wff of FOL.
5. If $\phi$and $\psi$are wffs of FOL, then $\phi \vee \psi$ is a wff of FOL.
6. If $\phi$and $\psi$are wffs of FOL, then $\phi \rightarrow \psi$ is a wff of FOL.
7. If $\phi$and $\psi$are wffs of FOL, then $\phi \leftrightarrow \psi$ is a wff of FOL.
8. If $\phi$is a wff or FOL and $\xi$is a variable, then $\forall\xi\phi$is a wff of FOL
9. If $\phi$is a wff or FOL and $\xi$is a variable, then $\exists\xi\phi$is a wff of FOL
10. Nothing else is a wff of FOL.

Notice how the quantifier rules do not require any particular relationship between the variables in $\phi$and the one added with the quantifier (ξ). It's just that $\exists\xi$will bind ξif there happen to be any free in ϕ. Maybe there are not. We cannot build any formulas containing unbound variables using our rules of syntax. Variables are only ever introduced along with a quantifier that binds them. That makes the following definition from Magnus et al., 2025 more or less moot in our system. **All** of our variables are *bound* by quantifiers, from the moment they are introduced.

Definition 9.9 (Bound and Free Variables)**:** In first-order logic, a variable $\xi$is *bound* iff it occurs in the scope of a quantifier that uses it: $\forall\xi$or $\exists\xi$. Otherwise it is *free*.

But note that we will make reference to expressions nominally containing free variables when we talk about the *scope* of a quantifier. For example, when we talk about $\phi(\xi)$–i.e. an expression in the scope of $\exists\xi$in $\exists\xi\phi(\xi)$–we note that it is an expression with *free* occurrences of ξ. But in the full context, $\xi$is bound by $\exists\xi$. In the rare instances that we talk about such expressions, we will refer to them as *open wffs*.

9.4 Scope and Bound Variables

Given the close alignment of the definitions, we can recycle some of our ideas about operators and scope from TFL.

Definition 9.10 (Main Logical Operator)**:** The main operator in a wff of first-order logic is the one that was introduced last, given the recursive application of the construction rules introduced in Def 9.7.

Definition 9.11 (Scope)**:** The *scope* of an operator in first-order logic is the subformula for which it serves as the main logical operator.

We can analyze these properties using the same toolkit as TFL, with some tweaks to account for quantifiers. First, we can make use of parse trees to dissect the structure of FOL wffs. We will need a new rule that allows us to add dummy names wherever we encounter a quantified sentence.

Definition 9.12 (Standard Parse Tree)**:** Given a wff of first-order logic, we can build its *standard parse tree* by using the following procedure:

1. If the current node is $\neg\phi$add a node below it with ϕ
2. If the current node is $\phi\wedge\psi$, $\phi\vee\psi$, $\phi\rightarrow\psi$, or $\phi\leftrightarrow\psi$, then split the tree with ϕon one branch and ψon the other.
3. If the current node is $\exists\xi\phi(\xi)$ or $\forall\xi\phi(\xi)$, then add a node below and with the instance $\phi(\delta_i)$ where δ_i is the next available dummy name.

Once again, the *main logical operator* is the connective associated with the first rule applied at the top of the tree (i.e. the last rule applied in the construction of the wff). And the subformula that decorates the node above a given rule application is the *scope* of the associated operator. These definitions work exaclty as they do in TFL, except for the addition of rules for $\forall$and $\exists$, which are analogous to $\neg$in their scopal behavior. In Def **??**, the scope of both $\forall\xi$and $\exists\xi$includes $\phi(\xi)$.

9.5 Sentences and Dummy WFFs

We can now identify sentences as a special kind of wff, where all the names are proper and all the variables are bound. Well-formed formulas with dummy names are called *dummy wffs* and those without are called *sentences*.

Definition 9.13 (Sentence)**:** In first-order logic, a wff $\phi$is a *sentence* iff all the terms that ϕcontains are proper names. Otherwise it is called a *dummy wff*.

We will adopt some bracketing simplification for sanity. For example, $((A\wedge B)\wedge C)$ can be written $(A\wedge B\wedge C)$ and $((A\vee B)\vee C)$ can be written $(A\vee B\vee C)$, since it will not effect the meaning of the sentence to do so. We also allow the use of square brackets "[" and "]" as substitutes for parens "(" and ")", to make pairs of brackets easier to spot.

9.6 Ambiguity and Quantifier Scope

The quantifiers in first-order logic bind variables that occupy slots in the structure of FOL atomic formulas that are typically reserved for terms. But the structure of the syllogisms we reviewed last week is more like a comparison of predicates: "All F are G" or "Some F is G". Our solution was to treat the scope of the quantifiers as sentences connecting the two predicates we care about, taking them to use the same variable, so:

$\forall x(Fx \rightarrow Gx)$
$\exists x(Fx \wedge Gx)$

Indeed, by our rules, these sentences were constructed from *instances* containing dummy names:

$Fc \rightarrow Gc$
$Fa \wedge Ga$

This approach gets interesting when we start to use multiple quantifiers and predicates with a greater arity. We saw this informally when we tried to symbolize some examples last lecture. "No woman loves every man" could be symbolized in either of the following two ways, each with a different meaning.

$\neg\exists x(Gx \wedge \forall y(Fy \rightarrow Lxy))$
$\forall y(Fy \rightarrow \neg\exists x(Gx \wedge Lxy))$

These formulas can be paraphrased, respectively, as:

(There is no woman such that)(Every man is such that)(She loves him)
(Every man is such that)(There is no woman such that)(She loves him)

Despite all the syntactic apparatus, the difference really just comes down to the order of the quantifiers: their scope. When the scope of a quantifer α_1 contains another quantifier α_2 and its scope, we say that α_1 takes *wide scope* over α_2.

Now that we have a grammar to work with, we can see how FOL allows us to be precise about some of the ambiguities of language that we just cannot distinguish in a strictly sentence-level logic like TFL. Magnus et al., 2025, Chapter 29 works through some examples that are particularly illustrative.

1. Everything that glitters is not gold.
2. Only young cats are playful.
3. Everyone went to see a movie.
4. The king of France is not bald.

The first sentence has two readings, the basic *every A is B* one and another that more close reflects its idiomatic sense, something like *it is not the case that everything that glitters is gold*.

1. $\forall x(F(x) \rightarrow \neg G(x))$
2. $\neg\forall x(F(x) \rightarrow G(x))$

Note how the difference involves the relative scopes of negation and the universal quantifier.

Similarly, for the second sentence, we get three distinct readings, normally distinguished in ordinary English with patterns of word stress.

1. If an animal is playful, then it is a young cat.
2. If a cat is playful, then it is young.
3. If an animal is young anf playful, then it is a cat.

These can all be easily distinguished via symbolization in FOL, again by taking account of the relative scopes of operators and quantifiers.

1. $\forall x(P(x) \rightarrow (Y(x) \wedge C(x))$
2. $\forall x((C(x) \wedge P(x)) \rightarrow Y(x))$
3. $\forall x((Y(x) \wedge P(x)) \rightarrow C(x))$

In the third sentence, we see how the quantifiers interact.

1. There is a specific movie and everyone saw that movie.
2. Everyone went to some movie or other.

The two readings differ with respect to which quantifier is placed within the scope of the other.

1. $\exists x(M(x) \wedge \forall y(P(y) \rightarrow \S(y, x)))$
2. $\forall y(P(y) \rightarrow \forall x(M(x) \wedge S(y, x)))$

Finally, Russell's treatment of definite desriptions, while it avoids *truth-value gaps*, makes the fourth sentence ambiguous in a way that directly impacts its truth value.

1. There is a king of France and he is not bald. (False)
2. There is no king of France who is bald. (True)

And, once again, the differences come down to which operator or quantifier takes *wide scope*.

1. $\exists x(K(x) \wedge \forall y(K(y) \rightarrow x \neq y) \wedge \neg B(x))$
2. $\neg\exists x(K(x) \wedge \forall y(K(y) \rightarrow x \neq y) \wedge B(x))$

Despite all these structural ambiguities, there are some cases where one quantifier taking wide scope over another is not meaningfully different than the opposite. First, we can always reverse the position of two adjacent quantifiers of the same type.

Observation 9.1 (Adjacent Quantifiers)**:** If two adjacent quantifiers are the same kind (either both universal or both existential), then it does not matter what order they occur in. In other words, $\forall\xi_1\forall\xi_2\phi$ is equivalent to $\forall\xi_2\forall\xi_1\phi$ and $\exists\xi_1\exists\xi_2\phi$ is equivalent to $\exists\xi_2\exists\xi_1\phi$ for all ϕ.

Similarly, we can adjust the scope of a quantifier in many cases where its variable does not occur in the subformulas impacted by the change.

Observation 9.2 (Moving Quantifiers)**:** When ψdoes not contain the variable ξ, the following pairs of wffs are equivalent.

1. $\forall\xi\phi(\xi)\wedge\psi$and $\forall\xi(\phi(\xi)\wedge\psi)$
2. $\exists\xi\phi(\xi)\wedge\psi$and $\exists\xi(\phi(\xi)\wedge\psi)$
3. $\forall\xi\phi(\xi)\vee\psi$and $\forall\xi(\phi(\xi)\vee\psi)$
4. $\exists\xi\phi(\xi)\vee\psi$and $\exists\xi(\phi(\xi)\vee\psi)$
5. $\forall\xi\phi(\xi)\rightarrow\psi$and $\exists\xi(\phi(\xi)\rightarrow\psi)$
6. $\exists\xi\phi(\xi)\rightarrow\psi$and $\forall\xi(\phi(\xi)\rightarrow\psi)$
7. $\psi\rightarrow\forall\xi\phi(\xi)$ and $\forall\xi(\psi\rightarrow\phi(\xi))$
8. $\psi\rightarrow\exists\xi\phi(\xi)$ and $\exists\xi(\psi\rightarrow\phi(\xi))$

Now that we know how to make them, next week we will discuss how to interpret sentences of first-order logic.

10 FOL Semantics

For the semantics of FOL we will want to leverage as much as possible from our treatment of TFL semantics. But FOL sentences are not atomic symbols of truth values, they are constructed from basic FOL expressions using the rules of grammar. We will need to find a way to account for the truth of atomic sentences which rests on the meanings of these basic parts.

We will use *models* to give a semantics to first-order logic. They will be responsible for associating the expressions of first-order logic with a domain of individuals.

Definition 10.1 (Jobs for Models)**:** An *model* for first-order logic will:

1. determine the *truth* of sentences
2. set a non-empty *domain*;
3. assign each name a *reference* in the domain; and
4. assign each predicate an *extension* made from items in the domain.

Before we start formalizing models, let's look at some of the tools we will need. First, a notational convention. We will use $[\![\alpha]\!]$to talk about the reference (or extension) of an expression α. This will be useful in definitions and metalogic, where the semantic objects are often central to the discussions and a full description of them would be quite long-winded indeed.

10.1 The Basic Idea of Sets

We will start by informally introducing a new formal mechanism, sets. For our purposes, a *set* is a collection of objects. The domain is a set. We can symbolize a set of anything (symbols, objects, etc) using curly braces: a, b. We will not dig more deeply into the properties and behaviors of sets, other than to note that they have no order or repeated items. When all of the elements of one set $\Delta$and found in another set Γ, we say that $\Delta$is a *subset* of $\Gamma(\Delta \subset \Gamma)$. When some item x is found in a set Γ, we will say that x is an *element* of $\Gamma(x \in \Gamma)$.[13]

Sets are a technical device that constrain the models of FOL. Like truth values in TFL, the extensions of predicates (i.e. sets) are the closest we can get to their *meaning*. Beyond its subject belonging to the set of all sunsets, all the nuances of meaning in a predicate like "is a sunset" are lost. Where in TFL two sentences that have the same truth value in all situations are equivalent, in FOL two predicates that have the same extension are equivalent too.

[13]Often when we use $x \in \Gamma$, we mean to refer to x not to the relation $x \in \Gamma$, per se. We just want to note that we are talking about an x from Γ.

10.2 Domains, Reference and Extension

First, let's consider the meanings of the various basic expressions of FOL. We can skip brackets, since their meaning straightforwardly relates the rules of grammar and the scope of operators.

Definition 10.2 (Domain)**:** The *domain* of an FOL model is the set of all objects relevant to the model. A domain for first-order logic must include an individual for each of the names in the language, but multiple names may refer to the same individual.

We can define a domain either *extensionally*, by listing all of its individuals, or with a *domain description*.

The object that a term picks out is called its *reference*.

Definition 10.3 (Reference)**:** We take singular terms to *refer* to individuals in the domain. We will call the individual that a name refers to its *reference*. Although many names may pick out the same reference, each name only picks out one.

Two terms can refer to the same object.

Definition 10.4 (Coreference)**:** Two terms τ_1 and τ_2 are *co-referential* iff they have the same reference.

Likewise, we interpret a predicate as its *extension*.

Definition 10.5 (Extension of a Predicate)**:** The *extension* of a predicate in first-order logic is the set of objects to which that predicate applies. If $\beta$is a 1-place predicate, then the extension of $\beta$is a subset of the domain. If $\beta$is an n-place relation, then its extension is a set of n-place *tuples* of items from the domain

We equate predicates on the basis of their extension.

Definition 10.6 (Coextension)**:** Two predicates β_1 and β_2 are *co-extensional* iff they have the same extension.

We won't linger on sets and tuples, just note that they offer convenient notational devices. One way we could represent an FOL interpretation is extensionally: by listing the contents of the domain and the extensions of all the predicates by rote. Tuples make it possible for us to do that for n-place predicates where n¿1.

D: { John, Mary, Fred, Susie }
j: John
m: Mary
f: Fred
s: Susie
P: {Fred,Susie}
E: {Fred,Susie}
L: {⟨Fred,Mary⟩,⟨Mary,Fred⟩,⟨Susie,John⟩,⟨John,John⟩}

We get a quick sense of who is a philosopher (P), who studies epistemology (E), and who loves whom (L). But this FOL symbolization does not let us capture any of the differences in meaning between *philosopher* and *specialist in epistemology*. This has particularly notable consequences when dealing with so-called *empty predicates*: those predicates that do not have any objects in their extensions. On our approach, they all have the same meaning.

10.3 Setting Reference in Descriptive Domains

Now consider this descriptively defined interpretation which is less explicit about the contents of the domain and the extension of the predicate.

D: all of the birds of prey in Chemung County
h: Hank
T: lives at Tanglewood

While the domain refers to a **specific** set of individuals, we don't exactly have a name for every bird in the county. This makes the explicit definition of our predicates much more challenging. How do we talk about all of the birds in the domain who don't have a name? Although there are some tricks for doing so in the metalogic, the short version is that we can't, especially when we start talking about infinite sets.

Things get especially weird when we try to figure out if two objects are *distinct*. The classic example is:

p: Phosphorus
h: Hesperus
v: Venus

All of these are names for the same thing. We will stick to this version of *direct reference* in our understanding of FOL semantics.[14]

[14]

One trick is to assume that the elements of the domain can, in principle, be counted (and in a consistent order). This gives us a way to make unique reference to each of the items in a set: the natural numbers. As in Magnus et al., 2025 we will use numbers to stand for objects in the domain. Names are taken to point *directly* at the objects these numbers represent, and in a *rigid* way: a name always refers to **the same** object.

Complicating matters further, we cannot assume that domains are finite. There are a number of relations that cannot be validated by anything other than an infinite set of individuals. For example, when a *connected*[15] two-place predicate R is also both *transitive*[16] and *irreflexive*[17], this is only possible in an infinite domain. In other words, the following three wffs cannot be all valid together in a finite model.

connected	$\forall x \exists y Rxy$
transitive	$\forall x((Rxy \wedge Ryz) \rightarrow Rxz)$
irrelflexive	$\forall x \neg Rxx$

This certainly complicates any potential search for counter-examples. We will need more powerful validation tools then those appropriate to finite models. Ultimately, we will need to reason about the models themselves.

10.4 Interpretation in First-Order Logic

Definition 10.7 (FOL Model)**:** We define a model for first-order logic as comprising **D** and $[\![\cdot]\!]$ where:

1. **D** is a non-empty *domain of individuals*; and
2. $[\![\cdot]\!]$ is an *interpretation function* which assigns each term a reference and each predicate an extension as follows:

In future logic courses, especially *Modal Logic*, we will see how terms that refer to the same thing have funny behavior in so-called *epistemic contexts*.

The ancients knew that Phosphorus rose in the morning.

Phospherus is Hesperus.

$\therefore$ The ancients knew that Hesperus rose in the morning.

Although substitution of equivalents is typically truth-preserving. This is not a valid inference, but formalizing that proves to be difficult. Ultimately, all three names refer to one item. Our domain should reflect that. But how do we know when two items in the domain are the distinct? One proposal is that they are never the same. Specifically, each item is just itself. If two things are in the domain, then they really are two different things. We will wait until our survey of metalogic in Logic 2 to expand on this point. Suffice it to say, we will assume that the substitution of expressions with the same reference (or extension) will be truth-preserving in FOL, just as it is in TFL. But, technically, this is only correct in an *extensional context* (which is often circularly defined as a context where truth-preservation for the substitution of equivalents is a correct assumption).

[15] Every individual is related to some other individual.

[16] Rxy and Ryz always means Rxz.

[17] No individual is related to itself.

a. $[\![\tau]\!]$is a *reference* in **D** for each term τ;

b. $[\![\phi]\!]$is a *truth-value* for each 0-place predicate ϕ;

c. $[\![\beta]\!]$is a *set of individuals* from **D** for each 1-place predicate β;

d. $[\![\beta]\!]$is a *set of n-tuples of individuals* from **D** for each n-place predicate β, when n¿1

We've already seen how we can explicitly specify these things with sets of objects (and tuples of objects). Using numbers for our domain, we can recast the above example as follows.

D: {1,2,3,4}
j: 1 (John)
m: 2 (Mary)
f: 3 (Fred)
s: 4 (Susie)
P: {3,4}
E: {3,4}
L: {⟨3,2⟩,⟨2,3⟩,⟨4,1⟩,⟨1,1⟩}

The sets are easy enough to read, but the tuple notation for relations is sometimes difficult to scan. Instead, we can diagrammatically represent our interpretation of the relation L.

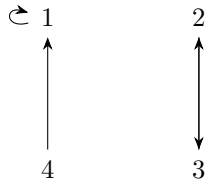


Figure 1: An interpretation of L

10.5 Assessing Truth with the Resources of FOL

Now that we have a handle on interpretations, we can start to talk about how truth works in first-order logic. Recall from our informal survey that we need to account for three types of sentences.

1. **atomic sentences:** sentences built from FOL terms and predicates but not from quantifers or connectives
2. **truth-functional sentences:** sentences whose main operator is truth-functional
3. **quantified sentences:** sentences whose main operator is a quantifier of FOL

Following the convention from Magnus et al., 2025, when a given object is found in the extension of a predicate, we will say that the predicate is *true of* that object. Similarly, we will say that a relation is *true of* a tuple of objects whenever that tuple occurs in the extension of the relation.

10.5.1 Modelling Quantified Sentences

We'll adopt the model from Magnus et al., 2025[Chapter 31] to keep things simple. But we will adopt the notation from Smith, 2020, where we label slots with numbers like ①. It is important to note that ① and 1 are different symbols of the metalanguage. ① is a placeholder for an empty term slot in a predicate. It will be filled by a term or a variable by the grammar. 1 is just the symbol we've decided to use to stand for a particular object in our domain, specifically Aristotle.

D: everyone born before 2000 CE
a: 1 (Aristotle)
b: 2 (Beyonce)
P : ① is a philosopher
R: ① was born before ②

Note that we are not actually giving our names and predicates extensional definitions here. We will assume truth based on real-world conditions. When we construct our truth definition for quantifier-governed sentences, we will want our rules to give some specific results on this interpretation, namely:

"Someone was born before Beyonce" is true
"Some philosopher was born before Aristotle" is true
"Everyone is born before someone" is false

symbolized as:

$\exists x R(b, x)$ is true
$\exists x(P(x) \wedge R(x, a))$ is true
$\forall x \exists y(R(x, y))$ is false

Let's see how we can get these results, starting with some basic rules for the truth of atomic sentences.

10.5.2 Atomic Sentences

We can now offer the following definitions of truth in first-order logic. As noted above, we will depart slightly from Magnus et al., 2025 and use the notation $[\![\alpha]\!]$ to mean the reference (or extension) of the FOL expression α, given a model.

The sentence $\beta\tau_1, \ldots, \tau_n$ is true in an model $\mathcal{M}$ just in case the references of $\beta\tau_1, \ldots, \tau_n$ form an n-tuple that can be found in the extension of β.

Definition 10.8 (Truth for Atomic Sentences)**:** Let $(M) = \langle \boldsymbol{D}, [\![\cdot]\!] \rangle$ be a model for FOL.

$[\![\beta\tau_1, \ldots, \tau_n]\!]_(M) = T$ iff $\langle [\![\tau_1]\!]_{\mathcal{M}}, \ldots, [\![\tau_n]\!]_{\mathcal{M}} \rangle \in [\![\beta]\!]_{\mathcal{M}}$ [a]

[a] From now on, we will drop the $\mathcal{M}$ subscript whenever the choice of model is clear from context.

The sentence $\tau_1 = \tau_2$ is true in an intepretation iff $\|\tau_1\|_I$ and $\|\tau_2\|_I$ are the very same individual in the domain $n \in D$.

Definition 10.9 (Truth for Identity Sentences)**:** $[\![\tau_1 = \tau_2]\!] = T$ iff $[\![\tau_1]\!] = [\![\tau_2]\!]$

10.5.3 Truth-Functional Sentences

For truth-functional sentences, we can just import our connective definitions from TFL.

Definition 10.10 (Truth for Connective-Governed Sentences)**:** The sentential connectives in FOL work just as they do in TFL:

1. $[\![\neg\phi]\!] = T$ iff $[\![\phi]\!] = F$
2. $[\![\phi \wedge \psi]\!] = T$ iff $[\![\phi]\!] = T$ and $[\![\psi]\!] = T$
3. $[\![\phi \wedge \psi]\!] = T$ iff either $[\![\phi]\!] = T$ or $[\![\psi]\!] = T$ (or both)
4. $[\![\phi \rightarrow \psi]\!] = T$ iff either $[\![\phi]\!] = F$ or $[\![\psi]\!] = T$
5. $[\![\phi \leftrightarrow \psi]\!] = T$ iff $[\![\phi]\!] = [\![\psi]\!]$

10.5.4 Quantified Sentences

The definition of truth for atomic sentences (10.8) puts a lot of responsibility on the interpretation function and term expressions. We need to be able to pick out the references for all of our terms in order to check the extensions of the predicates and determine the truth of atomic sentences. But what about the cases where we filled in term slots with something other than a term, like a variable? How do we evaluate the truth of (well-formed) formulas of this type? This is where the dummy names and dummy sentences come in.

Before defining truth for quantified sentences, we need to pause once more and consider the role that names play in definition 10.8. Specifically, we are assuming that each term picks out a specific item in the domain. For names this is straightfoward (by direct reference, the interpretation of a

name just **is** its object). But what about variables? Here, we can leverage the manner in which we built our domain (i.e. as *countable*, and therefore "named" by the natural numbers).

How does this work in practice? One soulution is to use the psuedo-names (numbers) associated with each domain item to talk about possible changes we might make to the interpretation. Specifically, we note that we **could** extend the interpretation by adding a new name and being precise about its reference. Indeed, one assumes that we do this sort of thing in ordinary English all the time (when we learn the name of a person we've seen before, for example).

In order to define our rules for truth, we will lean pretty heavily on the observation above that two equivalent expressions (i.e. two expressions with the same reference or extension) can be substituted for each other in a FOL sentence without changing the truth value of the sentence.

10.5.5 Interlude: Semantics with Open Sentences

As much as possible we want our definition to work at the sentence level, but when we are dealing with free variables, we have to acommodate formulas other than sentences as well.

Recall from Section 9.3 that we have chosen to use a slightly different grammar than Magnus et al., 2025. Specifically, our grammar will not create any *open sentences*: sentences with one or more *unbound variables* (*free variables*). The approach in Magnus et al., 2025 builds quantified sentences from open sentences by adding a quantifier with a variable that matches the free one. They get their open sentences by allowing variables to act like terms in the syntax: a variable can fill any slot in a predicate as though it was a term.

The important point for the semantics is that these formulas (i.e. open sentences) exist. They are built along the way as part of the process for constructing quantified sentences. As such, the semantics in Magnus et al., 2025 can talk about them as though they are well-defined. Indeed, they are.

To see how the semantics of quantified sentences works in Magnus et al., 2025 (which is a common approach in modern predicate logic), we will define open sentences as a type of *subformula*. From that definition, we can explore the standard treatment before we look at the adjustments needed for our official definitions.

Definition 10.11 (Open Sentence)**:** In our first-order logic, only terms and predicates can be used to construct atomic sentences. By definition, each variable is bound by the quantifier that introduces it. Neverthelss, we can think of open sentences as subformulas of quantified sentences.

Let θbe a quantifier symbol ($\forall$or $\exists$) and $\theta\xi\phi$be a quantified sentence of FOL. We define $\phi(\xi)$ as a *subformula* of $\theta\xi\phi$and further stupulate that it:

1. is an open sentence: it contains zero or more free occurrences of the variable ξ(but typically one or more);

2. is not a well-formed formula in our grammar

3. ...

With open sentences at our disposal, we can now present the lightly-modified definition of *substitution instance* from Magnus et al., 2025. In the traditional definition of truth for quantified sentences, we use substitution instances to talk about their *satisfaction*.

Definition 10.12 (Substitution Instance)**:** Let ϕbe a formula of first-order logic. Also let ξbe a variable and τbe a term. We define $\phi(\xi)$ to mean that ϕhas zero or more free occurrences of $\xi$and $\phi(\tau)$ to be the result of replacing all of the free occurrences of ξin $\phi$with τ. The resulting formula is called an *substitution instance* of $\phi$and $\tau$is called its *instantiating name*.

Armed with this notation, we can now talk explicitly about updating our interpretations with exactly the names we need to assess the truth of our quantified sentences. Let's start informally.

Recall that our model contains information about each of its names, specifically which $n \in D$ we are talking about. We can imagine adding more entries of this type by stipulating a name for any of the objects in $\mathbf{D}$. We will call this practice *updating* an interpretation.

Definition 10.13 (Interpretation Update)**:** Let $\mathcal{M} = \langle \mathbf{D}, [\![\cdot]\!] \rangle$ be an FOL model, δbe a name and n be an object in $\mathbf{D}$. Then $[\![\cdot]\!]^{[n/\delta]}$ is just like $[\![\cdot]\!]$ except that it assigns the object n to the name δ.

Then we can talk about whether or not a given object n is true of a predicate β. We will call this unmediated truth of objects in these new sentences *satisfaction* and the new name we add a *dummy name*.

Definition 10.14 (Satisfaction)**:** Let $\mathcal{M} = \langle \mathbf{D}, [\![\cdot]\!] \rangle$ be an FOL model.

Then an object $n \in \mathbf{D}$ *satisfies* $\phi(\xi)$ in $\mathcal{M}$ iff $[\![\phi(\delta)]\!]^{[n/\delta]} = T$, where the *dummy name* δdoes not already occur in $\phi(\xi)$.

We will refer to $\phi(\tau)$ in these cases as a *dummy sentence*.

More broadly, $\phi(\tau)$ is an *instance* of $\exists\xi\phi(\xi)$.

Now that we've done the metalogical heavy lifting in our definition of satisfaction, our truth definitions for quantifier-governed sentences in straight-forward.

Definition 10.15 (Truth for Quantifier-Governed Sentences)**:** Let $\mathcal{M} = \langle \mathbf{D}, [\![\cdot]\!] \rangle$ be an FOL model.

1. $[\![\forall\xi\phi(\xi)]\!] = T$ iff every $n \in \mathbf{D}$ satisfies $\phi(\xi)$

2. $[\![\exists \xi \phi(\xi)]\!] = T$ iff at least one $n \in \mathbf{D}$ satisfies $\phi(\xi)$

10.5.6 Quantified Truth without Open Sentences

We can reframe the definitions in 10.15 in terms of updating the interpretation function. There is at least one way to update $[\![\cdot]\!]$ to make $\exists \xi \phi$true; and $\forall \xi \phi$is true no matter how we update $[\![\cdot]\!]$.

Definition 10.16 (Quantifier Truth and Interpretation Updates)**:** Let $\mathcal{M} = \langle \mathbf{D}, [\![\cdot]\!] \rangle$ be an FOL model.

We define the truth of quantified sentences as follows:

1. $[\![\forall \xi \phi]\!] = T$ iff $[\![\phi]\!]^{[n/\xi]} = T$ for every $n \in \mathbf{D}$
2. $[\![\exists \xi \phi]\!] = T$ iff $[\![\phi]\!]^{[n/\xi]} = T$ for some $n \in \mathbf{D}$

Note how we can formulate this definition without any reference to open sentences like $\phi(\xi)$. We can just talk directly about $[\![\cdot]\!]$. These are, of course, just two ways of saying the same thing. We will adopt this second definition because it aligns better with our syntax and the semantics of open sentences seems incoherent, at best.

10.6 Quantifier Duals

One of the interesting properties of the Square of Opposition [ref?] is that it arranges the quantified sentences into pairs, or *duals*, which differed only in terms of negation. "Every" is a dual of "Not every". It is also a dual of "Every not". These two flavors of duality depend on two different flavors of negation: *external* and *internal*, respectively. Among the interesting details of this arrangement is that the existential and universal quantifiers are doubly dual: they are the result of both internal and external negation.

Observation 10.1 (Duals of Quantified Sentences)**:** The quantified sentences of FOL can be related via the following equivalences.

1. "Some" is synonymous with "Not every not"
2. "Every" is synonymous with "Not some not"

10.7 Metasemantics of FOL

We can summarize the truth semantics of FOL as follows.

Definition 10.17 (Truth in First-Order Logic)**:** Let $(M) = \langle \boldsymbol{D}, [\![\cdot]\!] \rangle$ be a model for FOL. We can assess the truth of FOL sentences as follows.

1. $[\![\beta\tau_1, \ldots, \tau_n]\!] = T$ iff $\langle [\![\tau_1]\!], \ldots, [\![\tau_n]\!] \rangle \in [\![\beta]\!]$
2. $[\![\tau_1 = \tau_2]\!] = T$ iff $[\![\tau_1]\!] = [\![\tau_2]\!]$
3. $[\![\neg\phi]\!] = T$ iff $[\![\phi]\!] = F$
4. $[\![\phi \wedge \psi]\!] = T$ iff $[\![\phi]\!] = T$ and $[\![\psi]\!] = T$
5. $[\![\phi \wedge \psi]\!] = T$ iff either $[\![\phi]\!] = T$ or $[\![\psi]\!] = T$ (or both)
6. $[\![\phi \rightarrow \psi]\!] = T$ iff either $[\![\phi]\!] = F$ or $[\![\psi]\!] = T$
7. $[\![\phi \leftrightarrow \psi]\!] = T$ iff $[\![\phi]\!] = [\![\psi]\!]$
8. $[\![\forall\xi\phi]\!] = T$ iff $[\![\phi]\!]^{[n/\xi]} = T$ for every $n \in \mathbf{D}$
9. $[\![\exists\xi\phi]\!] = T$ iff $[\![\phi]\!]^{[n/\xi]} = T$ for some $n \in \mathbf{D}$

With these truth definitions in place, we can now talk about how to formalize the semantic concepts relevant to first-order logic. As we have thus far, we will try to preserve as much as possible from our truth-functional definitions.

A set of premises Γ*entails* a sentence ψonly if it is never the case that $\Gamma$are all false, but $\psi$is true.

Definition 10.18 (Entailment)**:** The sentences $\phi_i, \ldots, \phi_n$ *entail* the sentence $\psi(\phi_i, \ldots, \phi_n \vDash \psi)$ iff there is no model $\mathcal{M} = \langle \mathbf{D}, [\![\cdot]\!] \rangle$ where $[\![\phi_i]\!] = F, \ldots, [\![\phi_n]\!] = F$ and $[\![\psi]\!] = T$.

A sentence $\phi$is *valid* iff $\phi$is true on every interpretation. We can call such ϕa *validity*.

Definition 10.19 (Validity)**:** A sentence $\phi$is *valid* iff $[\![\phi]\!] = T$ for every model $\mathcal{M} = \langle \mathbf{D}, [\![\cdot]\!] \rangle$.

A sentence $\phi$is a *contradiction* iff $\phi$is false on every interpretation.

Definition 10.20 (Contradiction)**:** A sentence $\phi$is a *contradiction* iff $[\![\phi]\!] = F$ for every model $\mathcal{M} = \langle \mathbf{D}, [\![\cdot]\!] \rangle$.

Similarly, an argument is valid just in case there is no interpretation in which all of its premises are true and its conclusion is false. (i.e. $\phi_i, \ldots, \phi_n \vDash \psi$).

Definition 10.21 (Valid Argument)**:** The argument $\phi_i, \ldots, \phi_n \therefore \psi$ is *valid* in first-order logic

iff there is no model $\mathcal{M} = \langle \mathbf{D}, [\![\cdot]\!] \rangle$ where $elib\phi_i]\!] = T, \ldots, [\![\phi_n]\!] = T$ and $[\![\psi]\!] = F$. Otherwise it is *invalid*.

Two FOL sentences $\phi$and $\psi$are *equivalent* when they are true in exactly the same interpretations (i.e. when both $\phi \vDash \psi$ and $\psi \vDash \phi$).

Definition 10.22 (Equivalent)**:** Two FOL sentences $\phi$and $\psi$are *equivalent* iff $[\![\phi]\!] = [\![\psi]\!]$ for all models $\mathcal{M} = \langle \mathbf{D}, [\![\cdot]\!] \rangle$

The FOL sentences $\phi_1, \ldots, \phi_n$ are *jointly satisfiable* when there is an interpretation that makes them all true.

Definition 10.23 (Joint Satisfiability)**:** The FOL sentences $\phi_1, \ldots, \phi_n$ are *jointly satisfiable* iff $[\![\phi_i]\!] = T$ for all $1 \leq i \leq n$ on some model $\mathcal{M} = \langle \mathbf{D}, [\![\cdot]\!] \rangle$. Otherwise they are *jointly unsatisfiable*.

10.8 Interpretations and Proofs

10.8.1 Proof and Counter-Models

Disproving the semantic properties discussed in the last section amounts to finding an interpretation that conflicts with the conditions in the definition. Those requirements in the are summarized in the following table.

to disprove	**find an interpretation where**
validity	$\phi$is false
contradiction	$\phi$is true
equivalent	$\phi$and ψdiffer in truth value

We will call such a refutory interpretation a *counter-model*. Generalizing to multi-sentence arguments we can provide the following set of interlocking definitions.

Definition 10.24 (Effects of Counter-Models on Arguments)**:** Let $\mathcal{M} = \langle \mathbf{D}, [\![\cdot]\!] \rangle$ be an FOL model.

If $[\![\phi_1]\!] = T, \ldots, [\![\phi_n]\!] = T$ but $[\![\psi]\!] = F$, then:

1. $\phi_1, \ldots, \phi_n \therefore \psi$ is *invalid*; and
2. $\phi_1, \ldots, \phi_n \nvDash \psi$; and

3. $\phi_1, \ldots, \phi_n, \neg\psi$ are *jointly satisfiable*

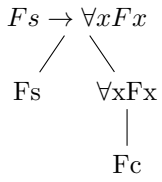
For the examples in the rest of this section, we will assume the following model.

D: {1, 2, 3}
s: 1 (Socrates)
p: 2 (Plato)
3 (Aristotle)
F: {1,3}
G: { }
L: {⟨1,2⟩,⟨2,3⟩, ⟨2,1⟩,⟨3,3⟩}

The same model can be used to show none of the following are valid.

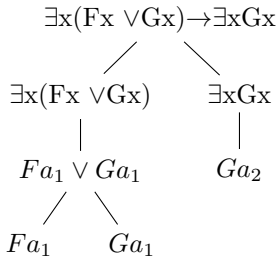
Fs.˙.∀xFx
∃x(Fx ∨Gx).˙.∃xGx
∀x∃yLxy.˙.∃y∀xLxy

First, this model is a counter-model for the validity of $Fm \rightarrow \forall xFx$. To see this we can use the tools introduced in the last two lectures. First, a parse tree reveals the deeper structure, including the specific instances that need to be evaluated for truth.

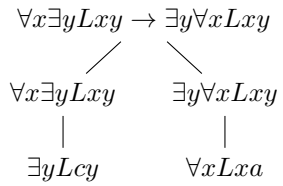


We need to show that Fs is true on a countermodel but that Fc is not, for some ⟦c⟧∈**D**. This is indeed the case since 2 (Plato) is not F in our model.

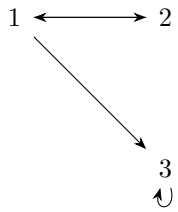
We can evaluate the second example with the following parse tree:



Once again, the antecedant is true. We could assign either 1 or 3 to the name a_1, since both satisfy Fx and hence $Fx \vee Gx$. We just need to find out if anybody is G. Nobody is. The sentence is not valid.



To show the sentence to be invalid we need a true antecendent ($\exists yLcy$) and a false consequent ($\forall xLxa$). A look at the interpretation of L will help.



The antecedent is true: every node has an arrow going out (i.e. everyone loves someone or other). The consequent, however, is false. There is no one that is loved by everyone. 3 is the only individual that loves themselves, and 1 does not love them. This is a counter-model. $\forall x\exists yLxy \rightarrow \exists y\forall xLxy$ is not valid.

10.8.2 Discovering Countermodels

The countermodel in the last section was given to us in advnace of the review. But how can we find one ourselves among the potentially infinite possibilities? The answer is to keep it simple. Start from a *minimal model*: a model with just one individual and no predicates. Then add in details as needed. It's more art than science, so let's look at a simple example. Later we will prove with natural deduction that $\exists y\forall xLxy \vdash \forall x\exists yLxy$. Now, consider this proof via counter-model that $\forall x\exists yLxy \nvDash \exists y\forall xLxy$

If we start with just one individual ($\mathbf{D}$=$\{1\}$) then we are left with only reflexive L=$\{\langle 1,1\rangle\}$ or empty L=$\{\}$. In the first case the conclusion is true so the entailment is valid. In the second case the premise is false so the entailment is valid. This is not a counter-model. But it we are careful we should be able to find a countermodel with two individuals ($\mathbf{D}$=$\{1,2\}$). We want both individuals to be loved, so as to sasify the premise, so we will connect them to each other. We want each individual to be unloved by someone to make the conclusion false, so we will just make that the person themselves (i.e. no one loves themself).

The resulting interpretation for L can be depicted as:

$$1 \longleftrightarrow 2$$

Figure 2: A simple counter-model proving $\forall x \exists y Lxy \nvDash \exists y \forall x Lxy$

10.9 Proving Validity

But proving the opposite (i.e. that there is no counter-model) requires more substantial reasoning about all possible models, since we need to know that our condition persists no matter what interpretation we use. We will look more closely at this kind of metalogical proof in *Logic 2*.

Ultimately, we can summarize the proof requirements for the various semantic properties of FOL in a table, just as we did for TFL.

property	**yes**	**no**
validity	all models	one counter-model
contradiction	all models	one counter-model
equivalent	all models	one counter-model
unsatisfiable	all models	one counter-model
valid argument	all models	one counter-model
entailment	all models	one counter-model

10.10 ** Bonus Material: Properties of Relations **

Finally, we can leverage regular properties of relations to characterize some predicates without fully specifying their extensions. For instance we know that the following argument is valid.

Andy is taller than Bill.
Bill is taller than Charlie.
$\therefore$ Andy is taller than Charlie.

This is because the *taller than* relation is transitive. In other words:

Example 10.1: $eforallx \forall y \forall z[(R(x,y) \wedge R(y,z)) \rightarrow R(x,z)]$

Note that we have defined our relation in the familiar language of first-order logic. A handful of useful properties of relations are thus defined in the following table.

property	fol definition
transitive	$eforallx\forall y\forall z[(R(x,y) \wedge R(y,z)) \rightarrow R(x,z)]$
reflexive	$\forall x(R(x,x))$
symmetric	$\forall x\forall y(R(x,y) \rightarrow R(y,x))$
anti-symmetric	$\forall x\forall y((R(x,y) \wedge x \neq y) \rightarrow \neg R(y,x))$

11 FOL Proofs

Proofs for FOL will be governed by the same rule system as TFL, with the addition of some new rules for dealing with singular terms, predicates and quantifiers. Of course, these rules will take the form of our familiar introduction and elimination rules, plus some derived rules that let us avoid repeating frequently needed subproofs.

11.1 Basic Natural Deduction Rules for FOL

Given the definitions for substitution instances discussed last week, defining elimination rules for $\forall x$ and introduction rules for $\exists x$ is particularly straight-forward. In our rules, we let ξbe a metavariable for FOL variables, τbe a metavariable for FOL terms and δbe a metavariable for FOL dummy names.

With *universal elimination*, we can replace a universally quantified variable with any term.

Rule 11.1 (Universal Elimination ($\forall$E))**:**

n	$\forall\xi\phi(\xi)$	
	$\phi(\tau)$	$\forall$E, n

With *existential introduction*, we can replace any one or more instances of a given name with an existentially-quantified variable.

Rule 11.2 (Existential Introduction ($\exists$I))**:**

n	$\phi(\delta)$	
	$\exists\xi\phi(\xi)$	$\exists$E, n

The two complementary operations, (*existential elimination* and *universal introduction*), are more constrained in their definitions.

For *universal introduction* ($\forall$I), we should only allow the rule to apply when the name in question is *arbitrary*. For the purposes of proofs, this amounts to not being found in premises or unresolved subproofs since their use there makes them non-arbitrary.

Rule 11.3 (Universal Introduction ($\forall$I))**:**

n	$\phi(\delta)$	
	$\forall\xi\phi(\xi)$	$\exists$E, n

But only if δdoes not occur in a premise or an undischarged assumption.

Finally, with *existential elimination*, if we have $\exists\xi\phi(\xi)$ and can prove $\phi(\delta)\rightarrow\psi$via subproof, then we can infer ψ, provided that it does not contain δ.

Rule 11.4 (Existential Elimination ($\exists$E))**:**

n	$\exists\xi\phi(\xi)$	
i	$\quad\phi(\delta)$	AS
j	$\quad\psi$	
	ψ	$\exists$E, n, i–j

But δmust not occur in:

1. any assumption discharged before line i
2. $\exists\xi\phi(\xi)$
3. ψ

11.2 Subproofs and Dummy Names

We should pause for a moment to consider the rules for the use of dummy names (δ) in subproofs. Note that both universal introduction and existential elimination have strict rules about when a dummy name can be used in an inference step. Let's look at some examples of bad inferences (modified from Smith, 2020).

1	$\exists xFx$	PR
2	$\quad Fa$	AS
$**3$	$\quad\forall xFx$	$\forall$I, 1, 2
$**3+1$	$\forall xFx$	$\leftrightarrow$E, 1, 2–$**3$

We cannot infer that everyone is F just from the fact that someone is. This proof mistakenly generalizes Fa by misusing the $\forall$I rule. The name a is introduced in an assumption that has not yet been discharged (i.e. it is an assumption of an open subproof.

Here's another one.

1	$\exists x Fx$	PR
2	$\quad Fa$	AS
3	$\quad Fa$	R, 2
$**4$	Fa	$\leftrightarrow$E, 1, 2–3
$**4+1$	$\forall x Fx$	$\forall$I, $**4$

This time the problem is that we are trying to draw an instance from a closed subproof. But the subject of that instance (a) is entirely hypothetical, a representative example introduced by the assumptions of the subproof. The $\exists$E rule does not allow this kind of inference.

Similarly, we cannot reuse representative names. Once we've introduced δ_i via assumption, we need to later use a new name (δ_{i+1}) if we want to assume a name for a different existential sentence.

1	$\exists x Fx$	PR
2	$\exists x Gx$	PR
3	$\quad Fa$	AS
4	$\quad\quad Ga$	AS
5	$\quad\quad Fa \wedge Ga$	$\wedge$I, 3, 4
6		
$**7$	$\quad \exists x(Fx \wedge Gx)$	$\leftrightarrow$E, 2, 4–6
$**7+1$	$\forall x Fx$	$\leftrightarrow$E, 1, 3–$**7$

The "error" is the reuse of a dummy name in a new subproof (line 4). We cannot assume that the x who Fs is the same as the x who Gs. But the bad inference step is actually on line 7. We are not allowed to eliminate dummy names from dummy sentences when the name is introduced within a closed subproof.

11.3 Proof Strategies for First-Order Logic

Similarly, proofs in FOL will proceed with a strategy based on the one we devised for FOL, with the addition of some new techniques to deal with quantifiers. Quite tersely, we can describe these new techniques as follows.

1. Work forward when dealing with the existential quantifier ($\exists$)
2. Work backward when dealing with universal quantifiers ($\forall$)

Consider this example proving $\forall x(Px \to Q) \vdash \exists x Px \to Q$.

First we set up the proof with premise and conclusion at their respective ends.

1	$\forall$x(Px$\to$Q)	PR
	⋮	
n	$\exists$x Px$\to$Q	

Then we can work backward from the conclusion, a conditional.

1	$\forall$x(Px$\to$Q)	PR
2	$\quad$ $\exists$x Px	AS
	$\quad$ ⋮	
m	$\quad$ Q	
n	$\exists$x Px$\to$Q	$\to$E, 2–m

Now our best move is to work forward from the existential in the subproof, adding a new subproof with an instance as its assumption and Q as its conclusion.

1	$\forall$x(Px$\to$Q)	PR
2	$\quad$ $\exists$x Px	AS
3	$\quad\quad$ Pa	AS
	$\quad\quad$ ⋮	
l	$\quad\quad$ Q	
m	$\quad$ Q	$\leftrightarrow$E, 2, 3–l
n	$\exists$x Px$\to$Q	$\to$E, 2–n

Now we can unpack the universal quantifier inside the subproof, working forward from the first line. We can use a for its instance, which will be helpful, since the conclusion follows by modus ponens.

1	$\forall x(Px \rightarrow Q)$	PR
2	$\quad \exists x\ Px$	AS
3	$\quad\quad Pa$	AS
4	$\quad\quad Pa \rightarrow Q$	$\forall$E, 1
5	$\quad\quad Q$	$\rightarrow$E, 3, 4
6	$\quad Q$	$\leftrightarrow$E, 2, 3–5
7	$\exists x\ Px \rightarrow Q$	$\rightarrow$E, 2–6

After merging the new proof strategies into the ones for TFL in Obs. 5.1, we get the following complete list of proof strategies for first-order logic.

Observation 11.1 (Proof Strategies for FOL)**:** We can tersely describe the basic strategy for FOL proofs as follows.

1. Work backwards from the conclusion.
   a. Conjunction: what components needed to use the $\wedge$I rule as the last step? i.e. what are the conjuncts?
   b. Conditional: what are the assumption and conclusion of the subproof we need to take the last step? i.e. what are the antecedant and consequent of the conditional?
   c. Negation: what subproof is needed to show a contradiction?
   d. Disjunction: which of the disjuncts looks easiest to prove?
   e. Universal quantification: how can we prove an arbitrary instance?
2. Work forwards from the current step.
   a. Conjunction: unpack the conjuncts
   b. Disjunction: what sentence do you need to prove from both disjuncts?
   c. Conditional: how can you prove the antecedant?
   d. Negation: how can you prove a contradiction?
   e. Existential quantification: how can we work from an assumed instance to the conclusion?
3. Leverage $\perp$
   a. Work toward a contradiction to conclude anything, especially in a subproof
   b. Indirect proof: build a contradictory subproof that assumes the negation

11.4 Some Examples of FOL Proofs

Let's looks at a few more examples of proofs in first-order logic.

First, we'll prove $\exists xFx, \forall x(Fx \to Gx) \vdash \exists xGx$.

1	∃xFx	PR
2	∀x(Fx→Gx)	PR
3	⋮	
n	∃xGx	

Working forward from the first line, we start a subproof assuming Fa. And working backward from the last line, we end that subproof with $\exists xGx$.

1	∃xFx	PR
2	∀x(Fx→Gx)	PR
3	Fa	AS
4	⋮	
m	∃xGx	
n	∃xGx	↔E, 1, 3–m

Now we can unpack the universal with an instance using a, get Ga by modus ponens. The existential generalization $\exists xGx$ follows immediately. And we are done.

1	∃xFx	PR
2	∀x(Fx→Gx)	PR
3	Fa	AS
4	Fa→Ga	∀E, 2
5	Ga	→E, 3, 4
6	∃xGx	↔I, 5
7	∃xGx	↔E, 1, 3–6

We can show the validity of $\forall x(Hx \to \neg Gx), \forall x(Fx \to Gx) \vdash \forall x(Hx \to \neg Fx)$ with a more involved proof.

1	∀x(Hx→¬Gx)	PR
2	∀x(Fx→Gx)	PR
3	⋮	
n	∀x(Hx→¬Fx)	

Working backward from the conclusion suggests that we are trying to prove an arbitrary instance $(Ha \rightarrow \neg Fa)$.

1	∀x(Hx→¬Gx)	PR
2	∀x(Fx→Gx)	PR
3	⋮	
m	(Ha→¬Fa)	
n	∀x(Hx→¬Fx)	∀I, m

Let's unpack the premises with that instance in mind.

1	∀x(Hx→¬Gx)	PR
2	∀x(Fx→Gx)	PR
3	Ha→¬Gx	∀E, 1
4	Fa→Ga	∀E, 2
5	⋮	
m	(Ha→¬Fa)	
n	∀x(Hx→¬Fx)	∀I, m

Now we can build a subproof for $Ha \rightarrow \neg Fa$.

1	$\forall$x(Hx$\rightarrow\neg$Gx)	PR
2	$\forall$x(Fx$\rightarrow$Gx)	PR
3	Ha$\rightarrow\neg$Ga	$\forall$E, 1
4	Fa$\rightarrow$Ga	$\forall$E, 2
5	Ha	AS
6	$\vdots$	
l	$\neg$Fa	
m	(Ha$\rightarrow\neg$Fa)	$\rightarrow$E, 5–l
n	$\forall$x(Hx$\rightarrow\neg$Fx)	$\forall$I, m

Now we will use modus ponens and modus tollens with those conditionals we unpacked before. And we are done.

1	$\forall$x(Hx$\rightarrow\neg$Gx)	PR
2	$\forall$x(Fx$\rightarrow$Gx)	PR
3	Ha$\rightarrow\neg$Ga	$\forall$E, 1
4	Fa$\rightarrow$Ga	$\forall$E, 2
5	Ha	AS
6	$\neg$Ga	$\rightarrow$E, 3, 5
7	$\neg$Fa	MT, 4, 6
8	(Ha$\rightarrow\neg$Fa)	$\rightarrow$I, 5–7
9	$\forall$x(Hx$\rightarrow\neg$Fx)	$\forall$I, 8

Things get more difficult when we start to apply more subproof-oriented strategies. Consider this example from Smith, 2020, p. 308.

1	$\exists x Fx \vee \exists x Gx$	PR
2	$\quad \exists x Fx$	AS
3	$\quad\quad Fa$	AS
4	$\quad\quad Fa \vee Ga$	$\vee$I, 3
5	$\quad\quad \exists x(Fx \vee Gx)$	$\exists$I, 4
6	$\quad \exists x(Fx \vee Gx)$	$\exists$E, 2, 3–5
7	$\quad \exists x gx$	AS
8	$\quad\quad Ga$	AS
9	$\quad\quad Fa \vee Ga$	$\vee$I, 6
10	$\quad\quad \exists x(Fx \vee Gx)$	$\exists$I, 7
11	$\quad \exists x(Fx \vee Gx)$	$\exists$E, 7, 8–10
12	$\exists x(Fx \vee Gx)$	$\vee$E, 1, 2–6, 7–11

11.5 Generalizing from Proofs

As with TFL, we can prove the validity of entire classes of arguments in FOL by using *substitution instances*. Indeed, by using metalogical notation in place of FOL formulas, we can generalize our arguments into *schemas*.

Observation 11.2 (FOL wffs can be generalized into metalogical schemas)**:** Since the validity of a formal argument is dependent on its shape alone (i.e. its form), any valid sentence (or argument) can be reformulated as a *schema* by consistently replacing expressions in the sentence with metalogical variables of the appropriate type.

One corollary of this observation is that we can use the same practice with proofs.

Observation 11.3 (Substitution Instances of Proofs are Proofs)**:** For any proof Γin the natural deduction system of first-order logic, a substitution instance Γ' of $\Gamma$is also a proof.

As a result, we can just prove the schema itself. Given all the different ways we can do a consistent substitution, this means we can often prove arguments for formulas that are superficially quite distinct. Consider this example from Smith, 2020, where the justifications are omitted for space, but should be obvious enough.

1	$\forall x Fx$	1	$\forall \xi \phi(\xi)$	1	$\forall z(\exists y Rzy \wedge Hz)$
2	$\forall x(Fx \rightarrow Gx)$	2	$\forall \xi(\phi(\xi) \rightarrow \psi(\xi))$	2	$\forall z((\exists y Rzy \wedge Hz) \rightarrow \forall x Sxz)$
3	Fa	3	$\phi(\tau)$	3	$\exists y Rcy \wedge Hc$
4	$Fa \rightarrow Ga$	4	$\phi(\tau) \rightarrow \psi(\tau)$	4	$(\exists y Rcy \wedge Hc) \rightarrow \forall x Sxc$
5	Ga	5	$\psi(\tau)$	5	$\forall x Sxc$
6	$\forall x Gx$	6	$\forall \xi \psi(\xi)$	6	$\forall z \forall x Sxz$

We will call such a schematic proof a *proof schema*. Just like proofs in the object language, a proof using proof schemas shows the validity of the arguments that they prove. As in TFL, we can convert back and forth between schematic theorems and theorems in the object language. The advantage of the schematic proofs is that they distill the problem as much as possible to a mere matter of form. The relationships that matter are revealed as directly as possible.

11.6 Equivalences between Quantified Sentences

Like conjunction and disjunction, existential and universal quantifiers are mutually-definable with the help of the negation operator. We saw the conversion rules for quantifers in Observation 10.1, but we did not prove them. But we can do so now with natrual deduction. As a reminder:

Observation 11.4 (Duals of Negative Quantified Sentences)**:** The quantified sentences of FOL can be related via negation with the following equivalences:

1. $\neg \forall \xi \phi \leftrightarrow \exists \xi \neg \phi$
2. $\neg \exists \xi \phi \leftrightarrow \forall \xi \neg \phi$

As usual, we will prove these results one conditional at a time. We will express the sentences schematically, but we could easily do the same proof with Fx and Fa in place of $\phi(\xi)$ and $\phi(\delta)$, respectively. [18]

[18] Similalrly for $\phi(\xi)$ and $\phi(\tau)$.

11.6.1 $\neg\forall\xi\phi \leftrightarrow \exists\xi\neg\phi$

1	$\neg\forall\xi\phi(\xi)$	PR
2	$\quad\neg\exists\xi\neg\phi(\xi)$	AS
3	$\quad\quad\neg\phi(\tau)$	AS
4	$\quad\quad\exists\xi\neg\phi(\xi)$	$\leftrightarrow$I, 3
5	$\quad\quad\bot$	$\neg$E, 2, 4
6	$\quad\neg\neg\phi(\tau)$	IP, 3–5
7	$\quad\forall\xi\phi(\xi)$	$\forall$I, 6
8	$\quad\bot$	$\neg$E, 1, 7
9	$\exists\xi\neg\phi(\xi)$	IP, 2–8

1	$\exists\xi\neg\phi(\xi)$	PR
2	$\quad\forall\xi\phi(\xi)$	AS
3	$\quad\quad\neg\phi(\delta)$	AS
4	$\quad\quad\phi(\delta)$	$\forall$E, 2
5	$\quad\quad\bot$	$\neg$E, 3, 4
6	$\quad\bot$	$\leftrightarrow$E, 1, 3–5
7	$\neg\forall\xi\phi$	$\neg$I, 2–6

11.6.2 $\neg\exists\xi\phi \leftrightarrow \forall\xi\neg\phi$

1	$\neg\exists\xi\phi$	PR
2	$\quad\phi(\tau)$	AS
3	$\quad\exists\xi\phi(\tau)$	$\leftrightarrow$I, 2
4	$\quad\bot$	$\neg$E, 1, 3
5	$\not\phi(\tau)$	$\neg$I, 2–4
6	$\forall\xi\neg\phi(\xi)$	$\forall$I, 5

1	$\forall\xi\neg\phi(\xi)$	PR
2	$\quad\exists\xi\phi(\xi)$	AS
3	$\quad\quad\phi(\delta)$	AS
4	$\quad\quad\neg\phi(\delta)$	$\forall$E, 1
5	$\quad\quad\bot$	$\neg$E, 3, 4
6	$\quad\bot$	$\leftrightarrow$E, 2, 3–5
7	$\neg\exists\xi\phi(\xi)$	$\neg$I, 2–6

11.7 Quantifier Conversion Rule

Now that we've proved the schemas valid, we can introduce natural deduction rules for the conversion of quantifiers as follows.

Rule 11.5 (Conversion of Quantifiers (CQ))**:** We can relate the quantifiers of first-order logic, with the following two pairs of complementary rules. One pair for handling a negated existential quantifier:

n	$\forall\xi\neg\phi$		n	$\neg\exists\xi\phi$	
	$\neg\exists\xi\phi$	CQ, n		$\forall\xi\neg\phi$	CQ, n

and one pair for handling a negated universal quantifier:

n	$\exists\xi\neg\phi$		n	$\neg\forall\xi\phi$	
	$\neg\forall\xi\phi$	CQ, n		$\exists\xi\neg\phi$	CQ, n

We will use the rules in a few examples. First, since we use the two kinds of quantifiers differently in quantified sentences, we'll prove $\neg\exists x(Fx \wedge Gx) \vdash \forall x(Fx \to Gx)$.

1	$\neg\exists x(Fx \wedge Gx)$	PR
2	$\forall x\neg(Fx \wedge Gx)$	CQ, 1
3	$\quad Fa$	AS
4	$\quad\quad Ga$	AS
5	$\quad\quad Fa \wedge Ga$	$\wedge$I, 3, 4
6	$\quad\quad \neg(Fa \wedge Ga)$	$\forall$E, 2
7	$\quad\quad \bot$	$\neg$E, 5, 6
8	$\quad \neg Ga$	$\neg$I, 4–7
9	$Fa \to Ga$	$\to$I, 3–8
10	$\forall x(Fx \to Gx)$	$\forall$I, 9

Similarly, working with the contents of a complex expression that is within the scope of a quantifier is challenging. Quantifier conversion rules make it easier to get them to interact in a way that aligns better with the natural deduction rules. Consider this proof of $\forall x(Fx \vee Gx), \neg\forall xFx \vdash \exists xGx$ where line 2 is inert until we apply CQ.

1	$\forall x(Fx \vee Gx)$	PR
2	$\neg\forall xFx$	PR
3	$\exists x\neg Fx$	CQ, 2
4	$\quad\neg Fa$	AS
5	$\quad Fa \vee Ga$	$\forall$E, 1
6	$\quad Ga$	DS
7	$\quad\exists xGx$	$\exists$I, 6
8	$\exists xGx$	$\leftrightarrow$E, 3, 4–7

We can even use CQ twice to reach inside universally quantified sentences in a way reminicent of modus tollens: $\forall x(Fx \rightarrow Gx), \neg\forall xGx \vdash \neg\forall xFx$

1	$\forall x(Fx \rightarrow Gx)$	PR
2	$\neg\forall xGx$	PR
3	$\exists x\neg Gx$	CQ
4	$\quad\neg Ga$	AS
5	$\quad Fa \rightarrow Ga$	$\forall$E, 1
6	$\quad\neg Fa$	MT, 4, 5
7	$\quad\exists x\neg Fx$	$\exists$I, 6
8	$\exists xFx$	$\exists$E, 3, 4–7
9	$\neg\forall xFx$	CQ

Derived rules can simplify to triviality a proof that would be extremely convoluted with just the basic rules. Consider $\neg\forall x\neg Fx \vdash \exists xFx$.

1	$\neg\forall x\neg Fx$	PR
2	$\exists x\neg\neg Fx$	CQ
3	$\quad\neg\neg Fa$	AS
4	$\quad Fa$	DNE
5	$\quad\exists xFx$	$\exists$I, 4
6	$\exists xFx$	$\exists$E, 2, 3–5

11.8 Proofs of Inferences with Quantified Sentences

The natutal deduction system makes it possible to prove the validity of a variety of inferences involving quantified sentences. First, we can show that a universal generalization proves the related existential generalization. $\forall xFx \vdash \exists xFx$, which is the expected result for a non-empty domain. The proof is a trivial application of the rules.

1	$\forall xFx$	PR
2	Fa	$\forall$E, 1
3	$\exists xFx$	$\exists$I, 2

The system starts to get more complex with multiple quantifiers. But the simple cases are still pretty easy to prove. For example: $\exists y\forall xLxy \vdash \forall x\exists yLxy$

1	$\exists y\forall xLxy$	PR
2	$\quad \forall xLxa$	AS
3	$\quad Lba$	$\forall$E, 2
4	$\quad \exists yLby$	$\exists$I, 3
5	$\quad \forall x\exists yLxy$	$\forall$I, 4
6	$\forall x\exists yLxy$	$\exists$E, 1, 2–5

The reverse is invalid, as we proved earlier with a counterexample (see: Fig. **??**).

Finally, consider this example which shows how a universal implication can be factored out of a disjunction, simplifying the formula significantly. $\forall x(Fx \rightarrow Gx) \vee \forall x(Fx \rightarrow Hx) \vdash \forall x(Fx \rightarrow (Gx \vee Hx))$

1	$\forall x(Fx \to Gx) \vee \forall x(Fx \to Hx)$	PR
2	$\quad \forall x(Fx \to Gx)$	AS
3	$\quad Fa \to Ga$	∀E, 2
4	$\quad\quad Fa$	AS
5	$\quad\quad Ga$	→E, 3, 4
6	$\quad\quad Ga \vee Ha$	∨I, 5
7	$\quad Fa \to (Ga \vee Ha)$	→I, 4–6
8	$\quad \forall x(Fx \to (Gx \vee Hx))$	∀I, 7
9	$\quad \forall x(Fx \to Hx)$	AS
10	$\quad Fa \to Ga$	∀E, 2
11	$\quad\quad Fa$	AS
12	$\quad\quad Ha$	→E, 10, 11
13	$\quad\quad Ga \vee Ha$	∨I, 12
14	$\quad Fa \to (Ga \vee Ha)$	→I, 11–13
15	$\quad \forall x(Fx \to (Gx \vee Hx))$	∀I, 7
16	$\forall x(Fx \to (Gx \vee Hx))$	DS, 1, 2–8, 9–15

11.9 Identity

Finally, we will need some rules for handling the identity predicate (=). As usual, these will take to form of introduction and elimination rules for =.

If $\tau$is a term, we can add a sentence at any point that says $\tau = \tau$.

Rule 11.6 (Identity Introduction (=I))**:**

τ=τ	=I

And, as with existential introduction, we can perform *identity elimination* on any one or more instances of the relevant term. Whenever we know that $\tau_1 = \tau_2$ for terms τ_1 and τ_2, we can replace all of the occurences of τ_1 in $\phi(\tau_1)$ with τ_2 and the resulting $\phi(\tau_2)$ will have exactly the same truth value. =E is also known as *Leibniz's Law*.

Rule 11.7 (Leibniz's Law (=E))**:** Since identity is a symmetric relation, we define a pair of rules for the elimination of non-trivial identity sentences like $\tau = \delta$.

m	τ=δ		m	τ=δ	
n	$\phi(\tau)$		n	$\phi(\delta)$	
	$\phi(\delta)$	=E, m, n		$\phi(\tau)$	=E, m, n

As always, the best way to understand the rules is to see them used in proofs.

Here's a trivial one that we could use to prove identity is symmetric: $a = b \vdash b = a$. Recall that $=$ is just a two-place predicate, so line 2 is playing **that** role in the inference to line 3.

2 a=a 3 b=a 1	$a = b$	=E, 1, n

We can also prove that $=$ is transitive. Here we will prove the generalization, not just an arbitrary instance.

1	a=b ∧b=c	AS
2	a=b	∧E, 1
3	b=c	∧E, 1
4	a=c	=E, 2, 3
5	(a=b ∧b=c)→a=c	→I, 1–4
6	∀z((a=b ∧b=c)→a=c)	∀I, 5
7	∀y∀z((a=b ∧b=c)→a=c)	∀I, 6
8	∀x∀y∀z((a=b ∧b=c)→a=c)	∀I, 7

So we can say that $\vdash \forall x \forall y \forall z((a = b \wedge b = c) \rightarrow a = c)$ is a *theorem* of first-order logic.

Definition 11.1 (Theorem of First-Order Logic)**:** A *theorem* in first-order logic is any FOL sentence ϕwhich can be proven without premises (i.e. $\vdash \phi$).

Not all theorems are as straightforward to prove as the symmetry and transitivity of identity, however. Many proceed inirectly and involve multiple layers of subproof. For example, consider this proof of $\vdash \neg\exists x \forall y((Lxy \rightarrow \neg Lyy) \wedge (\neg Lyy \rightarrow Lxy))$, which is short but far from simple.

1	$\exists x \forall y((Lxy \rightarrow \neg Lyy) \wedge (\neg Lyy \rightarrow Lxy))$	AS
2	$\quad \forall y((Lay \rightarrow \neg Lyy) \wedge (\neg Lyy \rightarrow Lay))$	AS
3	$\quad (Laa \rightarrow \neg Laa) \wedge (\neg Laa \rightarrow Laa)$	∀E, 2
4	$\quad Laa \rightarrow \neg Laa$	∧E, 3
5	$\quad \neg Laa \rightarrow Laa$	∧E, 3
6	$\quad\quad Laa$	AS
7	$\quad\quad \neg Laa$	→E, 4, 6
8	$\quad\quad \bot$	¬E, 6, 7
9	$\quad \neg Laa$	¬I, 6–8
10	$\quad Laa$	→E, 5, 9
11	$\quad \bot$	¬E, 9, 10
12	$\bot$	∃E, 1, 2–11
13	$\neg \exists x \forall y((Lxy \rightarrow \neg Lyy) \wedge (\neg Lyy \rightarrow Lxy))$	IP, 1–12

11.10 Basic FOL Metalogic

Without getting too deep into the metalogic, we will make some basic claims about first-order logic.

First, note that the system never fails to provide a truth value for well-formed formulas of the language. Whenever we have a sentence ϕbuilt by the grammar of first-order logic, we know that $[\![\phi]\!] \in \mathbf{T}$. Consider this informal proof adapted from Smith, 2020, pp. 342-343.

Observation 11.5 (Every sentence of FOL has a truth value)**:** Let $(M) = \langle \boldsymbol{T}, \boldsymbol{D}, [\![\cdot]\!] \rangle$ be a model for FOL $\mathcal{L}$. We will say that $\phi \in \mathcal{L}$ just in case $\phi$is a well-formed formula of $\mathcal{L}$. We will say that $[\![\phi]\!] \in \mathbf{T}$ just in case $\phi$is assigned a truth value by $[\![\cdot]\!]$.

Now suppose we have a wff ψ. We know that we can iteratively break ψdown into a parse tree $\mathcal{T}$ by reversing the rule associated with its main operator θ. We also know that the leaves of $\mathcal{T}$ will each be an atomic sentence $\chi(\delta_i)$ with dummy names replacing all the variables. We further know that $[\![\chi(\delta_i)]\!]^{[n/\delta_i]}$ is defined for every χthat can be built by the grammar of $\mathcal{L}$, since it has been updated to account for the dummy names δ_i.

Reversing our course in the parse tree, we know by the rules of FOL semantics that we can compute the truth of a complex sentence built by FOL connectives on the basis of the truth of the sentences that are connected. Since we know the truth value at each of the leaves, computing the truth of any node is just a matter of working up the part of the tree that it *dominates*.

When we reach the top of the tree, the truth value that we compute there will be the truth value of the entire formula.

A more complete proof would walk through the rules in detail: how do each of the rules for a parse tree preserve the FOL syntax? and how do each of the rules for computing FOL truth preserve the FOL semantics? But the informal sketch gives us a good idea how that might proceed.

Definition 11.2 (Validity and Entailment)**:** Let $\mathcal{L}$ be a system of first-order logic. Also let ψbe a sentence of $\mathcal{L}$ and $\Gamma = \{\phi_1, \ldots, \phi_n\}$ be a set of sentences of $\mathcal{L}$. The inference $\Gamma \therefore \psi$ is *valid* iff there is no model $(M) = \langle \mathbf{T}, \mathbf{D}, [\![\cdot]\!] \rangle$ where $[\![\phi_i]\!] = T$ for all $\phi_i \in \Gamma$ but $[\![\psi]\!] = F$.

When $\Gamma \therefore \psi$ is valid, we say that Γ*entails* $\psi (\Gamma \vDash \psi)$.

Similarly, we can define a notions like *tautology* and *consistency* against a backdrop of possible models.

Definition 11.3 (Tautology)**:** We will say that an FOL sentence $\phi$is a *tautology* iff $[\![\phi]\!] = T$ on **every** possible model $(M) = \langle \mathbf{T}, \mathbf{D}, [\![\cdot]\!] \rangle$.

Definition 11.4 (Consistency)**:** We will say that a set of FOL sentences $\Gamma = \{\phi_1, \ldots, \phi_n\}$ is *consistent* iff $[\![\phi_i]\!] = T$ for all $\phi_i \in \Gamma$ on **some** possible model $(M) = \langle \mathbf{T}, \mathbf{D}, [\![\cdot]\!] \rangle$.

Finally note that FOL is both *sound* and *complete*.

Observation 11.6 (Soundness and Completeness of First-Order Logic)**:** The system comprising the syntax and semantics of first-order logic is both *sound* and *complete*

$$ephi_1, \ldots, \phi_n \vdash \psi \text{ iff } ephi_1, \ldots, \phi_n \vDash \psi$$

The observation consists of two parts. First, our system is *sound* in that every provable argument is also a valid argument.

$$\text{if } ephi_1, \ldots, \phi_n \vdash \psi \text{ then } ephi_1, \ldots, \phi_n \vDash \psi$$

Second, our system is *complete* in that every valid argument is also provable.

$$\text{if } ephi_1, \ldots, \phi_n \vDash \psi \text{ then } ephi_1, \ldots, \phi_n \vdash \psi$$

Corollary to this observation is the practical equivalence of several analogous proof-theoretic and model-theoretic concepts.

Definition 11.5 (Properties of First-Order Logic)**:** Since first-order logic is sound and complete, the following pairs of concepts are analogues.

model-theoretic		**proof-theoretic**
valid	iff	provable
validity	iff	theorem
equivalent	iff	interderivable
jointly satisfiable	iff	jointly equivalent

Strategies for proving these concepts in FOL metalogic can be summarized in the usual table.

	yes	**no**
valid (sentence)?	prove $\dashv \phi$	find ϕcounter-model
contradiction?	prove $\phi \dashv \perp$	find $\neg\phi$ counter-model
equivalent?	prove $\phi \dashv \psi$ and $\psi \dashv \phi$	find model where $\phi$and ψdiffer
jointly satisfiable?	model where ϕ_i are all true	prove $\phi_1 \ldots \phi_n \dashv \perp$
valid (argument)?	prove $\phi_1 \ldots \phi_n \dashv \psi$	find model with $\phi_1 \ldots \phi_n$ all true but ψfalse

References

Magnus, P., Button, T., Trueman, R., & Zach, R. (2025). *Forall x: Calgary: An introduction to formal logic*. University of Calgary. https://openlogicproject.org/

Smith, P. (2020). *An introduction to formal logic* (2nd). Independently published.